达尔文哥德尔机

非官方中文研究资料 · 0.3 · 来源与许可逐篇注明

目录

1 达尔文哥德尔机：自我改进智能体的开放式演化 2

1.1 摘要 . . . 2
1.2 1 引言 . . . 3
1.3 2 相关工作 . . . 4
1.4 3 达尔文哥德尔机 . . . 5
1.5 4 实验 . . . 5
1.6 4.1 实验设置 . . . 6
1.7 4.2 基准 . . . 6
1.8 4.3 基线 . . . 7
1.9 4.4 结果 . . . 7
1.10 5 安全讨论 . . . 9
1.11 6 结论与局限 . . . 10
1.12 伦理声明 . . . 10
1.13 可复现性声明 . . . 11
1.14 致谢 . . . 11
1.15 附录目录 . . . 11
1.16 附录 A 补充结果 . . . 11
1.17 附录 B 补充相关工作 . . . 13
1.18 附录 C 算法细节 . . . 14
1.19 附录 D 实验细节 . . . 21
1.20 附录 E 基准细节 . . . 22
1.21 附录 F 发现的最佳智能体 . . . 31
1.22 附录 G 相似目标功能的不同实现 . . . 86
1.23 附录 H 案例研究：解决幻觉 . . . 107
1.24 附录 I 补充安全讨论 . . . 112
1.25 附录 J 补充未来工作方向 . . . 112

参考文献 113

1.26 译者术语表 . . . 128

1 达尔文哥德尔机：自我改进智能体的开放式演化

原作：Jenny Zhang[*,1,2]、Shengran Hu[*,1,2,3]、
Cong Lu[1,2,3]、Robert Lange[†,3]、Jeff Clune[†,1,2,4]. 版本：arXiv:2505.22954v3，2026 年 3 月 12 日. 原文：https://arxiv.org/abs/2505.22954v3. 原作者单位：[1] 英属哥伦比亚大学、[2]Vector Institute、[3]Sakana AI、[4]Canada CIFAR AI Chair. Jenny Zhang 与 Shengran Hu 为共同第一作者，Robert Lange 与 Jeff Clune 为共同资深作者. 联系方式：jennyzzt@cs.ubc.ca、srhu@cs.ubc.ca、conglu@cs.ubc.ca、robert@sakana.ai、jeff.clune@ubc.ca.

本译稿由 ai-self-study contributors 以机器辅助方式译制，非官方译本. 原作采用 CC BY 4.0：https://creativecommons.org/licenses/by/4.0/. 改动包括中文翻译、固定教材版式重排、交叉引用及图表编号适配. 图中英文标注、程序标识符、可执行代码及书目元数据保留原文，以便核对并避免改变程序含义；提示模板与输出样例的自然语言另作完整中文翻译. 各图均标明原作者和许可. 下文“我们”均指原作者，原作的论断、范围与历史时点按原文保留.

注 译者说明：本文演化的是调用冻结基础模型的智能体系统代码、工具与工作流，所报实验没有更新基础模型权重. 在有限编码基准上获得提升，不能作为无限持续自我改进的证明. 原文提出的开放式、自加速前景与实验已经验证的范围须区分. 这些说明不属于原论文正文.

1.1 摘要

当今多数人工智能系统受到人为设计的固定架构限制，不能自主而持续地改进自身. 相比之下，科学方法是累积性的开放系统，每项创新都建立在此前的成果之上，为未来发现创造条件. 人们越来越希望，当前依赖人工推动人工智能进步的过程本身也能实现自动化. 如果能够安全地完成，这种自动化将加速人工智能的发展，让我们更早获得其益处. 这一前景提出了一个问题：人工智能系统如何在不断提高相关问题求解能力的同时，无休止地改进自身？元学习能够自动发现新算法，但受限于一阶改进，以及人为设计的合适搜索空间. 哥德尔机 [136] 提出了一种理论替代方案：一种反复修改自身、且每次修改都可证明有益的自我改进人工智能. 遗憾的是，在实践中证明多数修改具有净收益是不可能的. 我们提出达尔文哥德尔机（Darwin Gödel Machine，DGM），一种新的自我改进系统；它迭代修改自己的代码，由此也改进修改自身代码库的能力，并用编码基准对每次变化进行经验验证. 受达尔文演化和开放式研究启发，DGM 逐步扩大生成的编码智能体档案. 它从档案中抽取智能体，这些智能体通过自我修改产生新的、有趣的版本. 这种开放式探索形成一棵不断生长的、多样且高质量的智能体树，并支持并行探索搜索空间中的多条不同路径. 实验中，DGM 自动改进其编码能力，例如获得更好的代码编辑工具、长上下文窗口管理和同行评审机制，使 SWE-bench 的表现从 20.0% 提高到 50.0%，Polyglot 从 14.2% 提高到 30.7%. 此外，DGM 显著优于不具备自我改进或开放式探索的基线. 所有实验均采取安全防护，如沙箱和人工监督. 总体而言，DGM 朝着自我改进人工智能迈出了重要一步，能够沿着通向无尽创新的路径收集自己的垫脚石. 全部代码开源于 https://github.com/jennyzzt/dgm.

1.2 1 引言

科学进步具有累积性与开放性，每次突破都建立在无数先前认识之上. 同样，最先进的人工智能系统也建立在悠久的创新谱系上. 例如，当今大语言模型 [17] 的骨干 Transformer[154] 并非凭空出现，而是建立在多年的先前创新上，包括循环神经网络 [94, 3, 61, 129] 和注意力机制 [139, 9, 76, 119]. 然而，当今多数系统仍受人为设计的固定架构约束，只能在预设边界内学习，不能自主重写自身源码来改进自己. 因此，人工智能发展的每项进步仍高度依赖人为干预，牵制了进步的速度. 本文研究一种令人关注的可能性：安全地自动搜索越来越好的人工智能. 可以设想，人工智能系统像科学发现本身一样，成为自身发展的动力：建立在过去成果上，递归改进，推动自己获得更先进的能力.

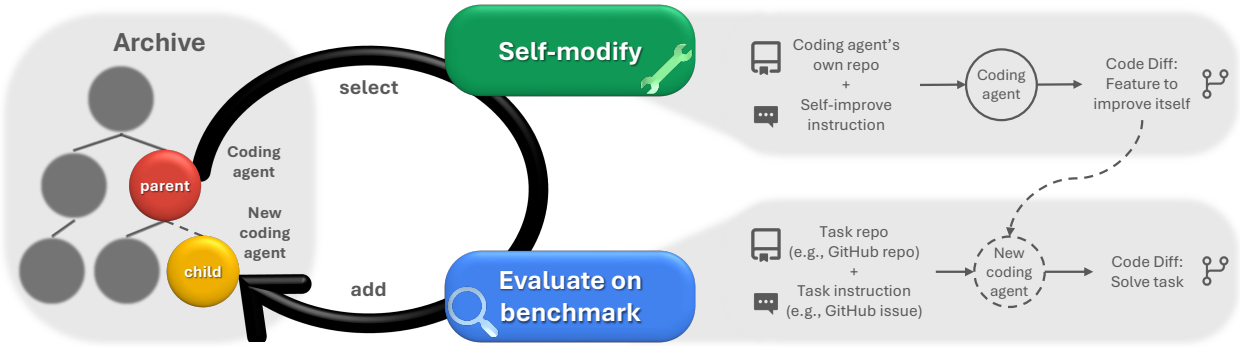


图 1.1: 达尔文哥德尔机. DGM 交替执行自我修改与下游任务评价，迭代构建不断扩大的智能体档案. 通过开放式探索，选择档案中的智能体执行自我修改. 原图：Jenny Zhang、Shengran Hu、Cong Lu、Robert Lange、Jeff Clune，CC BY 4.0；图内英文标注保留.

Schmidhuber[136] 提出了一类具有严格数学基础的、自指的、自我改进问题求解器. 它依赖形式证明来论证代码重写，确保任何自我修改都可证明有益. 但在实践中，如果不对系统作限制性假设，就无法形式证明对人工智能系统的一项修改是否有益. 例如，看似给基于大语言模型的编码智能体更多工具——代码搜索、测试运行器等——会带来益处，但实际影响高度依赖模型训练和任务上下文；针对一种设置优化的测试工具，可能在另一种设置下让智能体困惑. 我们不要求形式证明，而是根据基准经验验证自我修改，使系统能依据观测结果改进和探索. 这类似生物演化：突变和适应并不事先得到验证，而是在产生、试验之后经自然选择保留. 我们也从达尔文演化 [30] 获得启发，研究保留此前发现的智能体库、让其成为后代垫脚石的有效性.

我们提出**达尔文哥德尔机（DGM）**，一种自指、自我改进系统，通过编写和修改自己的代码成为更好的编码智能体. 每次自我修改都要求 DGM 编辑自己的代码库. 我们使用图灵完备的 Python，因此 DGM 在潜力上能够构建任何可计算机器. 我们的框架设想智能体可以重写自己的训练脚本，包括训练新的基础模型（FM）. 但本文没有展示这一点，因为训练基础模型计算量很大，会引入大量额外复杂性，我们将其留作未来工作. 本文聚焦于使用冻结预训练基础模型的编码智能体设计改进，如工具使用与工作流. DGM 交替执行自我修改和评价. 自我修改阶段，从档案中选出的编码智能体生成自己的修改版本；评价阶段，对每个修改后的智能体进行编码基准测试，估计其编码能力，再加入档案. 通过这一循环改善能力后，DGM 在解决编码任务和进行未来自我改进方面都变得更好. 一个关键假设是，编码基准表现提高意味着编码能力更好，从而自我修改与自我改进能力也更好. 此外，DGM 维护

生成智能体的档案，从一个智能体开始，随时间持续积累全部变体. 为支持持续自我改进，DGM 借鉴开放式研究 [157, 44, 43]，积累多样的垫脚石，即有趣但次优、可能促成未来突破的解或特征. 开放式探索鼓励发现超出即时性能收益的新颖且可能有用的自我修改.

我们在两个编码基准 SWE-bench[72] 与 Polyglot[122] 上报告结果. DGM 在 SWE-bench 上从 20.0% 自动提高到 50.0%，在 Polyglot 上从 14.2% 提高到 30.7%. 我们表明，自我改进支持持续进步：DGM 优于始终用同一个基础智能体修改并生成新智能体、却不进行自我改进的基线. 我们还表明，开放式探索与保留全部此前生成智能体的档案，有助于发现更好的编码智能体. DGM 优于不进行开放式探索的基线；后者不积累有趣且彼此不同的垫脚石档案，编码智能体始终建立在自身最新版本上. 总体而言，DGM 朝着能积累先前创新并递归改进的人工智能系统迈出了一步. 我们广泛讨论安全问题，包括沙箱化和自我修改的可追溯性，以保证负责任的实验（见5节）. 推进安全、自指、自我改进模型的可能性，使我们更接近不仅会学习、还会像科学一样沿开放式、自加速轨迹演化的人工智能.

1.3 2 相关工作

开放式. 推动无界创新的一项重大挑战，是设计不断生成新颖且可学习成果的开放式人工智能系统 [149]. Hughes 等 [66] 把开放式刻画为系统从观察者视角生成既新颖又可学习的成果序列的能力. 核心困难是如何组织和探索庞大搜索空间，持续产生对人有趣的成果 [24, 71]. 早期进展来自质量多样性算法、目标导向探索、内在动机和学习进度框架 [124, 36, 86, 117]；近期进展则用大规模基础模型作为人类趣味的代理，并作为跨领域生成和评价新行为的通用引擎 [17, 63, 175]. 但这些方法尚未闭合自指自我改进循环，即下游任务改进还未转化为更强的自我修改能力或进一步创新的加速. 我们希望模拟科学技术的加速过程，新工具和发现会催生更多发现. 如何模仿自然的演化轨迹，使其不仅朝复杂性发展，还朝着越来越强的演化能力发展 [31, 49, 58]?

基础模型智能体的元学习. 许多基于基础模型的智能体由人工设计. 构建模块包括提示工程 [22, 140]、思维链 [160, 165, 62, 54, 92, 108, 171]、自我反思 [143, 165, 102]、多智能体辩论 [183, 91, 74]、记忆 [96, 179, 106]、温度采样 [182] 与检索增强生成 [88]. 人工组合这些模块，会让系统能力受限于设计者的创造力. 近期出现的元学习方法使用基础模型自动优化提示 [44, 42, 75, 23, 170, 168] 和设计智能体模块 [176, 181, 167, 184, 128, 174, 166, 46, 112, 151, 177, 113]. 智能体系统的自动化设计（ADAS）[63] 用固定的元智能体迭代生成下游智能体，在目标基准上评价，并利用反馈改进后续生成. 相比之下，DGM 是单一系统，既解决下游编码问题，又改进自己的实现即代码库，不再需要固定、人工设计的元智能体，从而支持自指改进.

自我改进人工智能. 早期研究者提出了自我改进的理论和概念方案 [50, 134, 136]. 一些实践方法包括以神经网络权重参数化定义的系统 [135, 55, 60, 78, 67, 68, 97, 57]. Metz 等 [105] 开发了一个基于梯度的优化器，利用种群训练 [69] 的变体进行自指元训练. Lange 等 [82] 把这种方案扩展到无梯度学习. Silver 等 [145] 利用自博弈持续演化智能体，在国际象棋与围棋等困难领域取得超人表现. 更接近 DGM 的是近期使用基础模型智能体进行自我改进的方案 [167, 126, 64, 172, 65, 146]. Zelikman 等 [172] 用元智能体生成下游智能体，再依据生成解的元效用更新元智能体. Yin 等 [167] 用同一个系统解决下游任务并递归修改自身. 但下游任务或元效用并不总与自我改进所需能力一致. 在 DGM 中，下游任务提升直接反映自我改进能力增强，因此具有自加速进步的潜力. 最相似的同期工作是 Robeyns 等 [126]，它也让单一智能体递归解决编码问题并修改自己的代码库. 与该工作以及 Zelikman 等、Yin 等

的主要区别在于，DGM 具有开放式探索循环，鼓励超越即时收益的自我修改，避免在次优状态停滞. 附录B还讨论程序合成与达尔文演化的相关工作.

1.4 3 达尔文哥德尔机

哥德尔机是一种理论构想：寻找可证明能改进自身的方法的人工智能 [136]. 本文提出 DGM，尝试实现创造哥德尔机的长期梦想. DGM 放宽了哥德尔机不切实际的要求，即从理论上证明一项变化能改善系统；它改为要求通过实验中的经验证据，展示所提议的新版本提升了表现. 此外，依赖改进的经验证据，可能让 DGM 陷入全部可计算算法这一庞大可能系统空间的局部最优. 为此，DGM 在搜索中维护已发现解的档案，支持开放式探索，而不只演化一个解. 这些原则呼应达尔文演化 [30]（附录B）：从此前发现解的档案选出实体、修改它，并在它以有趣方式变新时保留 [175, 43, 148]. 因此我们把算法称为达尔文哥德尔机（图1.1）.

编码智能体的自指自我改进. DGM 仅以一个编码智能体初始化，并用编码基准评价进展. 编码智能体定义为一个以代码仓库实现、由冻结预训练基础模型驱动、能读写和执行代码的单一系统. 用 Python 等通用图灵完备语言表达的代码，是构建与改进智能系统的强大媒介，因为它可以表示任何可计算过程. 近期工作 [63, 176] 表明，可以通过对代码仓库中实现的提示、工作流与工具等设计进行元学习来改进智能体. 因而，我们把自我改进定义为修改智能体自身模块设计、也就是自身代码的一项编码任务；不包含下一段的开放式探索过程. 关键动机在于，经验证据必须同时反映系统自我改进和解决下游任务的能力. 把 DGM 配置为编码智能体并测试编码能力，观察到的改进不仅展示下游表现增强，也展示进一步自我改进的能力，因为自我改进本质上是一项修改编码智能体自身代码仓库的编码任务.

基于种群的开放式探索. DGM 从一个初始编码智能体开始，建立全部已发现智能体的档案. 每次迭代选出父智能体，让其自我修改并分支产生新智能体. 父代选择大体正比于各智能体的表现分数，反比于具有代码库编辑功能的子代数（附录C.2）. 这偏向表现良好而探索不足、即现有子代较少的智能体，兼顾利用强表现者与探索有希望但采样较少的谱系. 每个智能体保留非零选择概率，确保在计算资源足够时任何改进路径仍有可能. 每个被选父代分析自己的基准评价日志，提出下一项待实现功能，再把提案作为问题描述执行（附录C.3）. 父代将建议功能实现在自身代码库中，生成新编码智能体. 每个新智能体在选定编码基准上被定量评价，估计其编码能力. 只有成功编译且仍能编辑指定代码库的智能体才加入档案，因为只有它们能继续自我修改，其余均被丢弃. 父代选择、自我修改与评价循环不断进行，逐步扩充解的档案. 重要的是，归档的解可能成为在很久之后才带来改进的垫脚石，因此本方法与爬山式智能体设计 [126] 有实质区别. 当前开放式探索过程，即档案维护与父代选择，是固定的，不能由 DGM 修改；我们将其留作未来方向. 附录C.4给出算法伪代码.

1.5 4 实验

第4.1节介绍包括初始编码智能体在内的实验设置. 我们在 SWE-bench[72] 和 Polyglot[122] 上实验（第4.2节）. 每个基准均比较 DGM 与两个基线：无自我改进智能体的 DGM、无开放式探索的 DGM（第4.3节）. 所有实验中 DGM 都优于两个基线，表明自我改进机制与开放式探索对持续性能增益至关重要（第4.4节）. 我们还展示所发现特征能跨模型、基准和任务迁移.

1.6 4.1 实验设置

DGM 以单一编码智能体初始化. 基础智能体围绕冻结基础模型构建，增加工具使用能力 [133, 6]. 基础模型自主决定每一步的动作，使用它认为适合的工具. 基础智能体有两个工具：执行 bash 命令的 Bash 工具，以及查看和编辑目录、整个文件的编辑工具（附录C.1）. 智能体收到一个输入提示，说明目标仓库位置、问题描述和仓库测试框架. 我们故意让基础智能体保持轻量，提供可清楚展示自我改进的简单起点. DGM 运行 80 次迭代，每次生成一个新智能体；SWE-bench 同时运行两次迭代，Polyglot 同时运行四次（附录C.2）. 两个基准中的自我修改均由 Claude 3.5 Sonnet（New）[5] 驱动. 基准评价时，SWE-bench 使用 Claude 3.5 Sonnet（New），Polyglot 使用 o3-mini[116]（附录D.1）.

1.7 4.2 基准

我们在两个流行编码基准上评价 DGM；它们考察编码任务的不同方面，从而验证算法在不同用例中的有效性. SWE-bench 与 Polyglot 均评价人工智能智能体自动解决编码任务的能力. 给定代码仓库和任务指令，智能体需修改仓库以完成任务. 二者均被广泛使用 [178, 173, 163, 20, 51, 47]，要求智能体浏览仓库、理解不同文件中函数的关联，在复杂代码里识别细小错误. SWE-bench 只有 Python 任务，Polyglot 涵盖多种语言. 另一个区别是，SWE-bench 任务可能需编辑多个文件，而 Polyglot 主要需在单个文件从头实现解法；尽管仍需查看其他文件来理解必要修改，但总体编辑文件数更少.

SWE-bench. 为避免在不可解任务上浪费计算，我们使用 SWE-bench Verified[115]，即经人工筛选、所有任务均可解的 SWE-bench[72] 子集. 本文的 SWE-bench 默认指这一 Verified 子集.

Polyglot. Polyglot 包含 C++、Rust、Python 等多种编程语言任务 [122]. 相比最常用且可能已包含于基础模型训练集的 SWE-bench，Polyglot 更小众，更不可能包含于基础模型后训练数据. 此外，Polyglot 主要由其开发者用于评价 Aider[47]. 这提供了比较自动设计智能体与社区代表性编码智能体的机会，后者由人类开发者持续开发，并针对 Polyglot 测试超过一年. Polyglot 还允许检验 DGM 超出 Python、泛化到多语言的能力. 我们遵循 Polyglot 排行榜设置，但有一项关键区别：排行榜报告 pass@2，允许智能体查看一次真实测试反馈；我们报告 pass@1，智能体从不看到真实测试结果.

完整基准评价昂贵（附录E.1），我们因此采用分阶段评价来估计编码表现. 一般而言，越确信某智能体相对强，就用更多任务评价它. 先在 10 项任务上验证基本代码库编辑功能. 只有仍能编辑代码的智能体，才可解决下游编码任务或继续自我修改. 通过初始阶段后，再在扩展的 50 项任务上评价. SWE-bench 任务依据 SWE-bench-verified-mini[60] 选择，后者旨在代表完整基准（附录E.2）；Polyglot 任务为完整基准的随机子集（附录E.3）. 所用大语言模型固有的随机性会造成表现噪声. 较小子集用于估计总体有效性、识别优良候选. SWE-bench 中，若智能体在 60 任务子集上成功率超过 40%，且排在档案前两名，就再在 200 个任务上评价，即原 60 个加 140 个，以更精确衡量真实编码能力；40% 是依据预运行噪声选择的启发式阈值. Polyglot 中，若在 10 任务子集上成功率超过 40%，则在扩展的 50 任务子集上评价.

1.8 4.3 基线

为隔离 DGM 核心组件的贡献，我们比较**无自我改进的 DGM** 与**无开放式探索的 DGM**. 前者在此设置下复现 ADAS[63] 的方法：负责修改编码智能体的元智能体在整个实验中始终固定为基础智能体（原文算法 2）. 这用于考察迭代自我改进在加速更好编码智能体演化中的作用. 后者移除档案，始终修改最新保存的自身版本（原文算法 3）. 若智能体自我修改到丧失编辑代码库所需的基本功能，就无法继续修改自身或解决任何编码任务；因而该基线保留仍维持基本编辑功能的最新版本. 它用于评价档案以及已有充分记录的开放式探索有益原则 [24, 148, 175, 44, 84, 130, 27] 对智能体演化的作用.

除学习型基线外，我们还比较人工设计的开源方案. SWE-bench 采用经核验、即 SWE-bench 团队能复现结果的最佳开源方案（附录E.4）. Polyglot 采用代表性智能体 Aider[47]，它开源且设计为支持多语言和大型代码库编辑（附录E.5）. 为公平比较，在评价 DGM 的相同任务子集上测量解题比例；图1.2中的水平虚线展示这些值.

1.9 4.4 结果

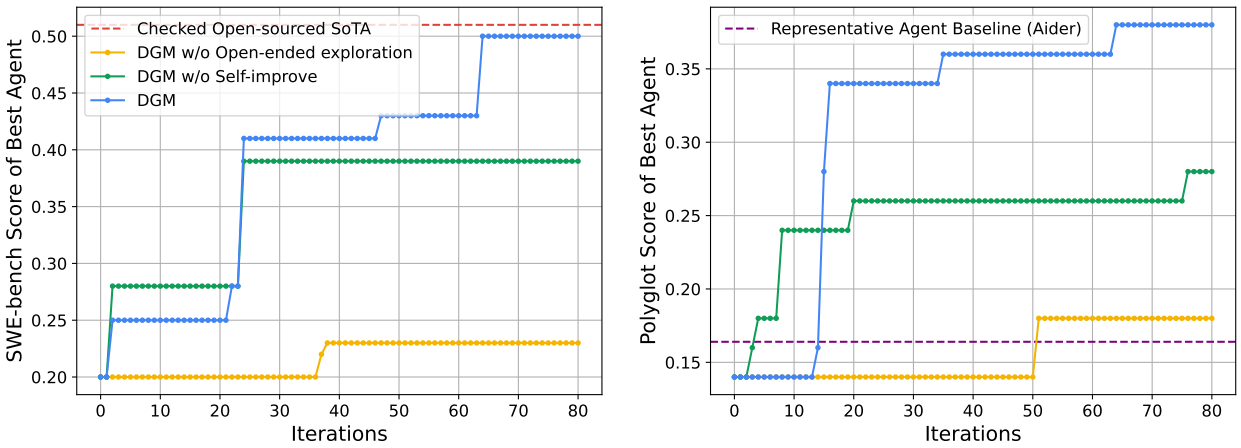


图 1.2: 自我改进与开放式探索使 DGM 持续进步并改善表现. DGM 自动发现越来越好的编码智能体，在左侧 SWE-bench 与右侧 Polyglot 上均取得更好表现. 它优于缺少自我改进或开放式探索的基线，表明两个组件对持续自我改进都至关重要. 分数来自第4.2节所述基准子集. 原图：Jenny Zhang、Shengran Hu、Cong Lu、Robert Lange、Jeff Clune，CC BY 4.0；英文标注保留.

80 次迭代后，编码智能体在 SWE-bench 上从 20.0% 提高到 50.0%，在 Polyglot 上从 14.0% 提高到 38.0%（图1.2）. 因 Polyglot 实验设置仅评价 50 项任务，我们另在完整 Polyglot 基准上评价基础智能体和 DGM 发现的最佳智能体，以更精确估计改进；完整基准上从 14.2% 提高到 30.7%. 这表明 DGM 能自动自我改进，生成更好的编码智能体. 此外，所发现最佳智能体的表现可比于 SWE-bench 经核验、开源、人工设计的最佳方案. Polyglot 中，尽管基础智能体表现低于 Aider，DGM 仍发现了远超 Aider 的智能体. 两个基准中，发现的智能体可比于或优于人工设计智能体. SWE-bench 最佳智能体与 Aider 都经过人类精心塑造，而 DGM 提示了这样的未来：这种创造力可以自动化，通过持续自我改进的自指循环不断演化.

DGM 自动改善工具与基础模型的使用工作流（图1.3）. 例如，它把编辑工具改为支持按行查看文件、通过字

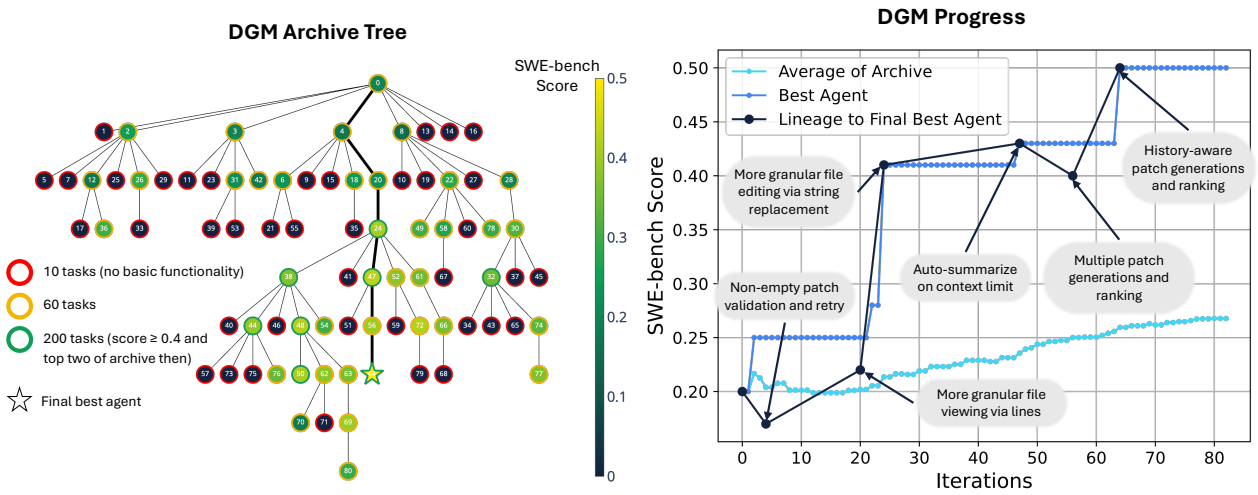


图 1.3: DGM 自动自我改进，成为更好的编码智能体. 左：SWE-bench 运行中生成的智能体档案. 每个节点代表一个编码智能体，节点 0 是基础智能体. 节点颜色表示 SWE-bench 表现即解题百分比，边框颜色表示该智能体评价过的任务数. 边表示哪个智能体自我修改产生了子代. 许多创新路径经过表现较差的节点，而节点 24 等关键创新引发建立于其上的创新爆发；这两点都体现开放式搜索的益处. 右：SWE-bench 上的 DGM 进展图. 浅蓝线是全部具有基本代码库编辑功能智能体的平均分，蓝线是每次迭代时档案中任一智能体达到的最高分，深色线是最终最佳智能体及其前驱的谱系，其中有两次性能下降. 这展示开放式搜索的益处：探索多样且有趣的垫脚石，而非只从当前最好解分支. 原图：Jenny Zhang、Shengran Hu、Cong Lu、Robert Lange、Jeff Clune，CC BY 4.0；英文标注保留.

符串替换精确编辑，而非总是查看或替换整个文件. 工作流改进包括多次尝试解题，用另一基础模型评价并选出最好方案，以及生成后续尝试时考虑此前尝试. 附录F.1与F.2分别展示通往两个基准最终最佳智能体的全部修改.

开放式探索允许以非零概率从档案中任何智能体分支，因此 DGM 能摆脱具有误导性的性能低谷或高峰. 例如 SWE-bench 的第 4 与 56 次迭代中，虽然智能体分数暂时低于父代，DGM 仍沿该路径探索创新，产生超过全部前驱的新智能体. 此外，开放式探索允许尝试相同目标功能的不同实现. 例如，更精细编辑工具的具体实现可以有很大差异，导致显著不同的表现（附录G）. DGM 可以探索多个实现，寻找更适合的方案，而不陷入次优实现.

DGM 在两个基准上均优于无自我改进和无开放式探索的基线. 不更新负责修改编码智能体的元智能体时，无自我改进基线在初期提升智能体，但增益很快趋缓（附录A.1）. 无开放式探索基线只保留最近的智能体，因此表现差的自我修改会增加后续改进难度.

为评价改进的普遍性，我们用不同于优化时使用的基础模型测试基础智能体与运行中发现的最佳智能体. SWE-bench 运行使用 Claude 3.5 Sonnet（New），迁移评价把模型替换为 Claude 3.7 Sonnet[7] 或 o3-mini，在 200 个任务上评价（图1.4左）. 使用 o3-mini 时，基础智能体 23.0%，发现智能体 33.0%；使用 Claude 3.7 Sonnet 时，分别为 19.0% 与 59.5%. 这提示改进可跨基础模型泛化，而非紧密耦合于运行时使用的特定模型. Polyglot 模型迁移结果见附录A.2.

我们进一步研究跨基准和编程语言迁移. 首先，把在一个基准如 SWE-bench 上发现的最佳智能体用于完全留出的另一基准如 Polyglot，反向亦然（图1.4中）. SWE-bench 演化出的最佳智能体在 Polyglot 达到 28.9%，初始基线是 14.2%；Polyglot 演化出的最佳智能体在 SWE-bench 达到 24.5%，原基线是 20.0%. 每个智能体优化时从未访

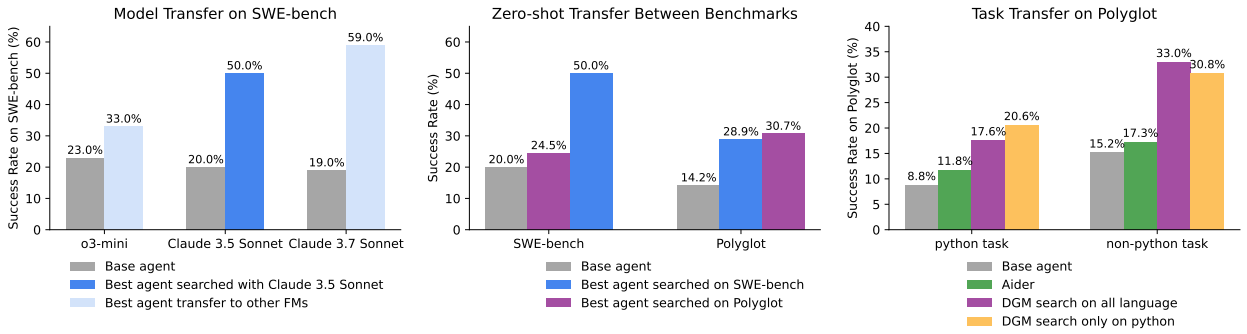


图 1.4: 模型、基准与任务间的迁移. DGM 发现智能体的更好表现可以迁移到不同模型（左）、不同基准（中），以及 Polyglot 中不同编程语言的任务（右），例如从 Python 任务迁移到 C++ 任务. 原图：Jenny Zhang、Shengran Hu、Cong Lu、Robert Lange、Jeff Clune，CC BY 4.0；英文标注保留.

问另一基准，因此这些评价是真正的留出测试. 跨基准一致的增益支持我们关于改进反映通用技能获得、而非过拟合或利用基准特定痕迹的论断. 其次，我们用只在 Polyglot 的 Python 任务上训练的 DGM，再把发现智能体迁移到其他语言. 与全语言训练相比，聚焦 Python 稍微改善 Python 表现，但降低非 Python 表现（图右）. 然而，从 Python 迁移到搜索时未见的语言后，智能体仍达到可比于全语言 DGM 的表现，显著优于基础智能体与 Aider. 这些结果展示改进的稳健性，说明并未过拟合于某一语言. 附录A给出更多结果.

1.10 5 安全讨论

DGM 等自我改进系统朝更自主的人工智能开发迈出一步，符合构建能造福人类的强人工智能这一长期目标 [134, 24, 103, 85]. 但自主修改自身代码的能力带来独特安全问题. 即便提高目标指标，只针对基准优化的修改也可能意外引入漏洞或偏离人类意图的行为 [15]. 尤其当评价基准不能完整刻画安全和稳健等期望属性时，自我改进循环可能在连续世代中放大不对齐. 迭代自我修改还可能导致日益复杂、难解释的内部逻辑，妨碍理解、监督和控制 [141, 8, 52, 45].

针对这些挑战，当前实现与实验有若干防护. 所有智能体执行和自我修改都在隔离沙箱中进行，限制对宿主系统的影响，降低非预期行动风险. 每次沙箱执行有严格时限，减少资源耗尽和无界行为风险. 自我改进当前限于通过修改自己的 Python 代码库来提高特定编码基准表现这一明确领域，限制修改范围. 此外，我们主动监测表现与代码变化，档案提供可追溯的修改谱系以供审查. 目前未发现生成智能体具有有害或恶意行为的证据，自我修改主要聚焦提高编码能力.

反过来，自我改进范式的一项潜在重要益处，是原则上可以用于加强安全和可解释性本身. 我们初步研究如何在人工智能安全场景部署 DGM，开发应对基础模型幻觉的方法（附录H）. 如同学习改进编码能力，如果评价标准包含相应属性 [128]，DGM 可能发现并整合更好的内部防护，或通过修改提高透明度，如纳入类似 Constitutional AI 的原则 [10]. 这提示一条有希望但具挑战的路径：把自我改进用作构建更可信人工智能的工具. 还可研究从一开始引入 Constitutional AI；困难是如何激励系统保留这些准则. 值得探索的一种方案是设置不可修改的系统部分，用于评价并停止其余部分.

由于当前前沿基础模型能力限制以及沙箱等有效缓解措施，DGM 在保持安全研究边界的同时展示自我改进潜力. 附录I进一步讨论更广的安全不确定性. 我们主动加入安全讨论，以提高对自我改进系统新兴前景及相关安全影响的认识，尤其考虑到它们必然会更有能力 [169, 14, 37, 13, 24]. 因此，我们倡导继续研究生成人工智能的算法 [24] 与自我改进系统的安全、有益演化.

1.11 6 结论与局限

我们提出 DGM，这是首个由基础模型驱动、具有开放式探索的自我改进系统，评价基准上的进步可以直接转化为更好的自我改进能力. 我们展示了自动发现更好的工具和基础模型系统，从而提高 SWE-bench 与 Polyglot 表现. 通过自我改进和开放式探索，DGM 表现持续上升，使我们向自加速、自我改进系统更进一步.

DGM 能自主达到与公开可用方案相当的表现，但仍不及 SWE-bench 最佳闭源方案. 运行更久是否持续产生收益并最终超过闭源方案，仍是开放问题. 闭源方案常依赖专家团队精心手工设计的技术. 基础模型在推理等能力上尚未达到这些专家水平，所以 DGM 当前需要大量计算来发现改进. 第4节所展示的一次 SWE-bench 运行约需两周，并产生可观 API 成本（附录E.1）. 我们假设进一步进展需更高效利用计算资源并发展更好的推理能力.

本版本主要由基础模型驱动，因此固有地受其能力限制. 一项令人期待的未来方向是，把自我修改从提示和工作流扩展到计算需求更大的方法，如重写训练脚本、更新基础模型本身. 当前聚焦编码，而人工智能越来越广泛地应用于计算机视觉、创意写作等领域；另一个有希望的扩展是构建能在编码之外改进自身的系统. 本研究的关键假设是编码基准能良好反映自我改进能力，因为自我修改要求编辑自身代码库. 也可设想共同演化目标任务分布 [43, 159]，像真正开放式过程一样，移除自我改进绑定单一目标的约束. 附录J列出更多方向. 继续探索这种强大技术时，必须如第5节所述，把安全放在中心.

总之，DGM 通过能够编辑自身代码库的自我改进系统，朝人工智能开发自动化迈出了重要一步. 计算与推理限制当前约束了完整潜力，但基础模型与基础设施的持续进步可能释放更强、更通用的自我改进. 若能谨慎处理安全问题，自我改进系统与人工智能生成算法 [24] 的未来有巨大潜力，通过开放式演化不断重写或重新训练自身，追求与人类价值一致的更强能力.

1.12 伦理声明

我们确认遵守 ICLR 伦理守则. 本研究在代码编辑智能体这一有限背景下研究自我改进系统，并在标准编程基准上评价. 没有人类受试者，也未收集或处理个人身份信息，因此无需机构伦理审查委员会批准.

安全与误用. 若允许无约束行动，或优化意外引入不安全行为，自我修改系统可能构成安全风险. 所有实验智能体都在隔离沙箱中运行，有严格资源和时限；网络访问受限，不能修改宿主环境. 自我改进范围限于智能体自身 Python 代码库和评价框架. 我们维护完整可审计的代码修改与评价谱系，支持回滚与事后分析. 发现的智能体未部署于真实开发环境. 代码、提示和评价成果的发布计划将排除提升系统访问权限的组件，包含默认沙箱、防护与清楚的预期用途文档.

两用性、下游影响与局限. 更强自主编码智能体可能有两用性，如帮助软件维护，也可能在误用时帮助创建有害代码. 我们认为推进可控、可审计自我改进方法与展示实用防护的研究收益超过风险. 但明确不鼓励安全敏感或

无沙箱部署，并提供具体安全建议（第5节）. 对基准优化的经验关注可能不能覆盖稳健、可解释和广泛社会价值等全部属性. 因而我们把基准增益视为通用人工智能发展的必要但不充分指标，并讨论把安全、推理等目标纳入优化循环的路径.

数据治理、知识产权与许可. SWE-bench Verified 与 Polyglot 由开源仓库与任务构成. 据我们所知，实验遵守了数据集许可和使用条款. 我们没有引入或分发专有代码. 基础模型通过提供者 API、依其服务条款访问；没有提交敏感数据，也没有尝试绕过使用政策. 发布日志将清除 API 密钥及意外出现的敏感字符串.

偏见、公平与平等. 虽然领域是软件代码而非以人为中心的文本，基础模型行为仍可能反映语言或生态系统偏好等偏见，对训练数据中工具覆盖更好的社区带来不均衡收益. 我们通过 Polyglot 多语言评价与跨基准迁移报告部分应对此问题. 未来应诊断有偏失败模式，并纳入更广、由社区推动的任务集.

利益冲突与资助. 作者均无在本文评价产品中的财务利益. 除薪酬或常规研究支持外，资助者与雇主未影响实验设计、分析或发表决定. 外部计算资源或 API 额度在附录中致谢.

1.13 可复现性声明

我们将开源全部代码与完整智能体日志，包括自我修改的完整档案谱系、差异补丁、提示、配置和评价框架. 为支持精确复现，本文提供算法细节与伪代码（第3节、附录C.4）、父代选择和开放式探索设置（附录C.2）、基础模型选择与超参数（附录D.1）、两个基准的任务子集（附录E.2、E.3）、分阶段评价协议与脚本（第4.2节）、最佳发现智能体的实现与补丁（附录F.1、F.2），以及计算与费用估计（附录E.1）. 发布的代码仓库将包含环境规范以及复现全部结果、图与表的脚本.

1.14 致谢

本研究得到 Vector Institute、Canada CIFAR AI Chairs 项目、Schmidt Futures 资助、NSERC Discovery Grant，以及 Rafael Cosman 的慷慨捐赠支持. 安大略省、加拿大政府通过 CIFAR，以及 Vector Institute 的赞助企业部分提供了研究准备所需资源（https://vectorinstitute.ai/partnerships/current-partners/）. 本材料表达的意见、发现、结论或建议均属于作者，并不必然反映资助者观点. 我们还感谢 Aaron Dharna、Ben Norman、Cédric Colas、Sam Devlin 和 Shyam Sudhakaran 的深入讨论和反馈.

1.15 附录目录

原文附录依次为：A 补充结果，B 补充相关工作，C 算法细节，D 实验细节，E 基准细节，F 发现的最佳智能体，G 相似目标功能的不同实现，H 解决幻觉的案例研究，I 补充安全讨论，J 补充未来工作方向. 原文的自动目录在译稿中改为这一索引，全部实质内容按原顺序保留.

1.16 附录 A 补充结果

A.1 SWE-bench 上的基线

A.2 Polyglot 上的跨模型普遍性

除 SWE-bench 上的模型迁移测试（第4.4节、图1.2）外，本节报告 Polyglot 的迁移结果. Polyglot 的 DGM

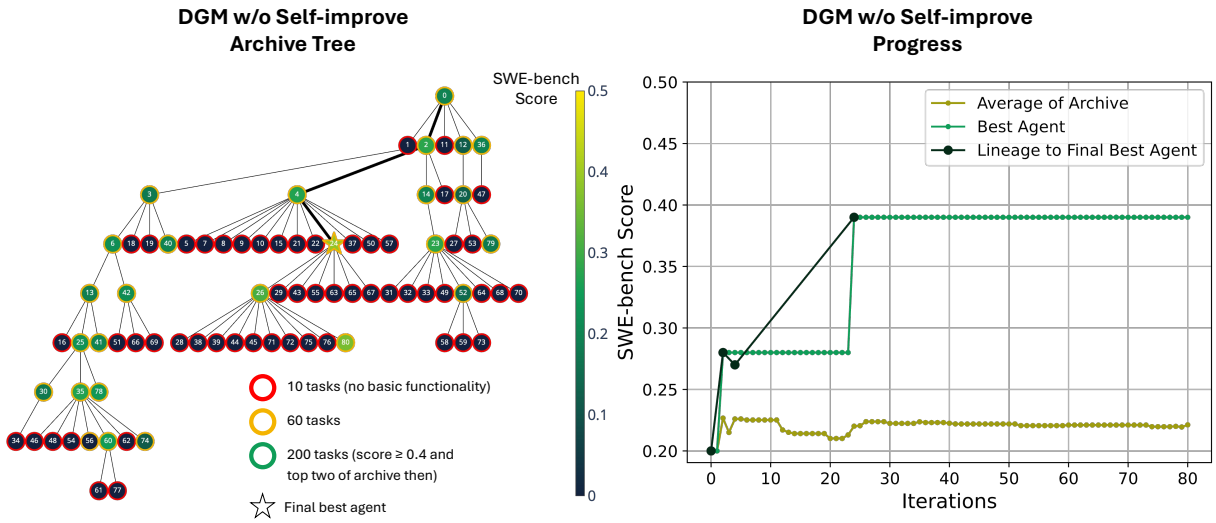


图 1.5: 没有自我改进智能体的 DGM. 修改并生成下一代编码智能体的元智能体始终相同，因此该基线不能随时间持续改进. 左: SWE-bench 运行生成的编码智能体档案. 每个节点代表一个智能体，节点 0 是基础智能体；节点颜色表示 SWE-bench 解题百分比，边框颜色表示评价任务数，边表示哪个智能体自我修改产生子代. 右: SWE-bench 上该基线的进展图. 浅绿线是具有基本代码库编辑功能智能体的平均分，绿线跟踪每次迭代时档案中任一智能体达到的最高分，深色线是最终最佳智能体及其前驱的谱系. 原图：Jenny Zhang、Shengran Hu、Cong Lu、Robert Lange、Jeff Clune，CC BY 4.0；英文标注保留.

运行使用 o3-mini；我们将基础模型替换为 Claude 3.5 Sonnet（New）或 Claude 3.7 Sonnet，并在完整基准上评价（图1.4中）. Claude 3.5 Sonnet（New）下，初始智能体为 32.0%，发现智能体为 33.3%；Claude 3.7 Sonnet 下分别为 35.6% 与 36.8%. 这些结果提示 DGM 产生的改进可跨基础模型泛化，而非紧密绑定于运行时所用的特定模型.

A.3 父代选择的消融

为进一步研究父代选择机制，我们引入 DGM Greedy. 它总是选择表现最好的节点作为分支父代，而非像本实现一样让每个节点都有非零分支概率，其大小大致由表现分数和子代数决定（附录C.2）. 该消融在此设置下复现 Robeyns 等 [126] 的方法. 表1.1中，DGM Greedy 在 SWE-bench 与 Polyglot 分别达到 39.7% 和 30.0%，本实现则为 50.0% 和 38.0%. 这表明让档案中所有解都能成为潜在垫脚石，可以随时间带来更大改进，强调了开放式探索的重要性.

表 1.1: DGM、消融与基线在 SWE-bench 和 Polyglot 上的比较.

方法	SWE-bench	Polyglot
DGM	50.0%	38.0%
无开放式探索	23.0%	14.0%
无自我改进	39.0%	28.0%
DGM Greedy	39.7%	30.0%

A.4 DGM 运行的补充统计

生成智能体中具备基本代码编辑功能的比例. 为更深入理解 DGM 过程，我们分析 SWE-bench 上生成智能体具有基本编辑功能的比例. 表1.2显示 DGM 的比例最高. 结果突出开放式探索与自我改进的有效性：前者让搜索逃

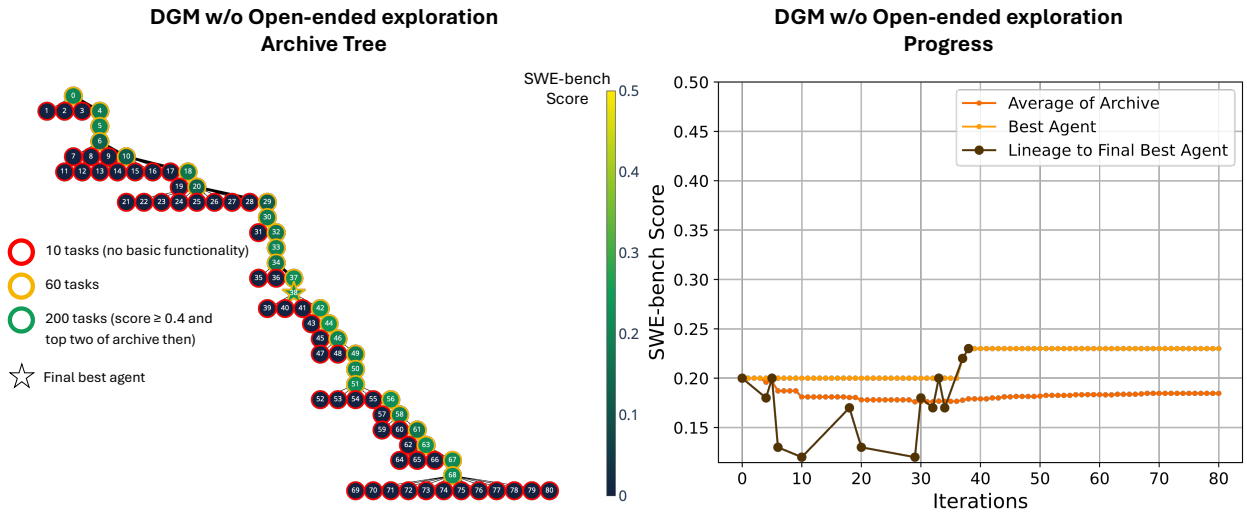


图 1.6: 没有开放式探索的 DGM. 移除档案后，该基线始终使用最近的智能体自我修改，在 SWE-bench 上进展很少. 左：运行中生成的智能体档案. 每个节点代表智能体，节点 0 是基础智能体；节点颜色表示解题百分比，边框颜色表示评价任务数，边表示父代自我修改生成子代. 右：进展图. 橙线是具有基本代码库编辑功能智能体的平均分，浅橙线跟踪每次迭代时档案中任一智能体达到的最高分，深色线是最终最佳智能体及其前驱谱系. 原图：Jenny Zhang、Shengran Hu、Cong Lu、Robert Lange、Jeff Clune，CC BY 4.0；英文标注保留.

离局部最优，后者增强生成更好智能体的能力. **DGM 运行的稳定性.** 为评价稳定性，我们在 Polyglot 上运行算法

表 1.2: SWE-bench 上生成智能体具备基本代码编辑功能的比例.

方法	比例
DGM	51.3%
无开放式探索	32.5%
无自我改进	32.5%

三次，分析性能方差. DGM 的平均准确率为 40.7%，标准差为 2.3%，表明它能在不同运行间取得一致且可复现的结果.

1.17 附录 B 补充相关工作

开放式（第二部分）. 早期方法探索平衡可学习性与趣味性的不同机制. 质量多样性算法试图以多样的高表现行为照亮庞大解空间 [124, 21, 107, 111]. 其他方法强调目标导向探索 [36, 38, 132, 4, 41]、内在动机 [86, 117, 89, 121] 或学习进度框架 [73, 48, 11, 25, 27, 70, 32, 137, 138, 81]. 近期，大规模基础模型 [17, 125] 成为人类趣味概念的强代理 [175, 43, 131]，也成为提出新代码解的有效突变算子 [127, 114, 87, 43, 63]. 基础模型可以引导自主设定目标的智能体 [27, 28, 26]，建模人类对质量和多样性的偏好 [16, 34, 156, 79, 80, 130, 93, 56]，设计奖励函数 [161, 155, 101, 43]，创建模拟环境 [152, 109, 1, 110, 18, 120, 138]，驱动不断演化的多智能体动态 [33, 180]，搜索多样的行走机器人形态 [87]，并为基准或目标优化搜索广阔解空间 [83, 175, 43, 63, 99, 127, 44, 98, 74, 100, 95, 114].

程序合成. 程序合成 [2, 123, 19, 53, 40] 旨在生成符合输入输出样例或逻辑公式等外部规范的代码. 混合方法把

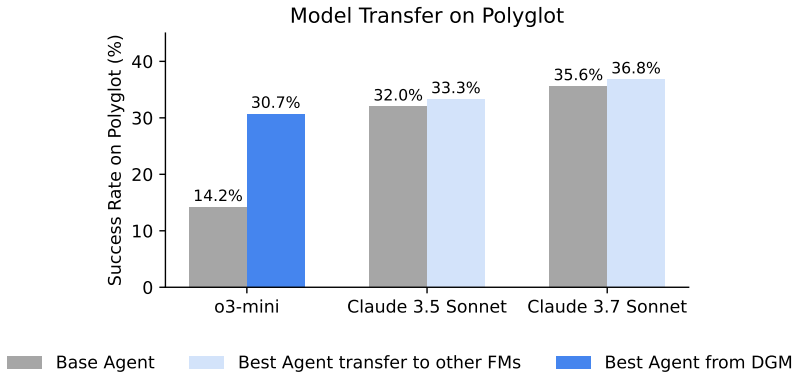


图 1.7: Polyglot 上的模型间迁移. 原图：Jenny Zhang、Shengran Hu、Cong Lu、Robert Lange、Jeff Clune，CC BY 4.0；英文标注保留.

符号方法与神经网络或基础模型引导结合：Li 等 [90] 用大语言模型建议引导 SyGuS 场景中的符号搜索，优于纯枚举；Barke 等 [12] 结合大语言模型补全与学习的代理模型，引导领域专用语言中的合成；Shi 等 [142] 用神经策略在搜索中构建高阶和 lambda 抽象，在列表操作任务上优于纯大语言模型和符号基线. DGM 的区别是，不只为外部任务产生程序，还聚焦智能体自我修改，重写自身实现以增强未来自我改进能力.

来自达尔文演化的启发. 本研究深受达尔文演化 [30] 的变异、选择和谱系保留，即垫脚石机制启发，将其引入自我修改编码智能体. DGM 保留过去版本的档案，从中抽取父代，以代码编辑突变生成子代，再在编码基准上经验评价，成功者加入档案，支持多条演化轨迹并行探索. 这类似生物演化 [39, 162] 保留遗传多样性 [104]、利用变异 [77] 并通过自然选择保留有益突变 [35] 的方式.

1.18 附录 C 算法细节

C.1 初始编码智能体

本节给出初始智能体可用工具及任务提示的细节. 所提供 Bash 工具的信息如下，程序文本保留原文；其全部说明的中文译文紧随其后.

```
def tool_info():
    return {
        "name": "bash",
        "description": """Run commands in a bash shell\n
* When invoking this tool, the contents of the "command" parameter does NOT need to be
↪  XML-escaped.\n
* You don't have access to the internet via this tool.\n
* You do have access to a mirror of common linux and python packages via apt and pip.\n
* State is persistent across command calls and discussions with the user.\n
```

```
* To inspect a particular line range of a file, e.g. lines 10-25, try 'sed -n 10,25p
↪  /path/to/the/file'.\n
* Please avoid commands that may produce a very large amount of output.\n
* Please run long lived commands in the background, e.g. 'sleep 10 &' or start a server in the
↪  background.""",
        "input_schema": {
            "type": "object",
            "properties": {
                "command": {
                    "type": "string",
                    "description": "The bash command to run."
                }
            },
            "required": ["command"]
        }
    }
```

Bash 工具说明的完整中文译文. 在 bash shell 中运行命令. 调用该工具时，command 参数内容无需作 XML 转义. 不能通过此工具访问互联网. 可以通过 apt 和 pip 访问常见 Linux 与 Python 软件包的镜像. 状态在多次命令调用及与用户的讨论间保持持久. 若要查看文件特定行范围，如 10–25 行，可使用 `sed -n 10,25p /path/to/the/file`. 避免可能产生大量输出的命令. 长时间运行的命令应放到后台，例如 `sleep 10 &`，或在后台启动服务器. 输入模式为 object，含字符串属性 command，其说明为“要运行的 bash 命令”；必填属性是 command.

所提供编辑工具的信息如下：

```
def tool_info():
    return {
        "name": "editor",
        "description": """Custom editing tool for viewing, creating, and editing files\n
* State is persistent across command calls and discussions with the user.\n
* If `path` is a file, `view` displays the entire file with line numbers. If `path` is a
↪  directory, `view` lists non-hidden files and directories up to 2 levels deep.\n
* The `create` command cannot be used if the specified `path` already exists as a file.\n
* If a `command` generates a long output, it will be truncated and marked with `<response
↪  clipped>`.\n
* The `edit` command overwrites the entire file with the provided `file_text`.\n
```

```
* No partial/line-range edits or partial viewing are supported.""",
        "input_schema": {
            "type": "object",
            "properties": {
                "command": {
                    "type": "string",
                    "enum": ["view", "create", "edit"],
                    "description": "The command to run: `view`, `create`, or `edit`."
                },
                "path": {
                    "description": "Absolute path to file or directory, e.g. `/repo/file.py` or
                    ↪  `/repo`.",
                    "type": "string"
                },
                "file_text": {
                    "description": "Required parameter of `create` or `edit` command,
                    ↪  containing the content for the entire file.",
                    "type": "string"
                }
            },
            "required": ["command", "path"]
        }
    }
```

编辑工具说明的完整中文译文. 这是用于查看、创建和编辑文件的定制编辑工具. 状态在多次命令调用及与用户讨论间持久保存. 如果 path 是文件，view 显示含行号的整个文件；如果 path 是目录，view 列出最多两层深的非隐藏文件与目录. 若指定 path 已是文件，不能使用 create. command 若生成很长输出，会被截断并标为 `<response clipped>`. edit 用提供的 file_text 覆盖整个文件. 不支持局部编辑、指定行范围编辑或局部查看. 输入模式为 object，command 为字符串，枚举为 view、create、edit，说明为 “要运行的命令：view、create 或 edit”；path 为字符串，说明为 “文件或目录的绝对路径，例如 /repo/file.py 或 /repo”；file_text 为字符串，说明为 “create 或 edit 所需参数，包含整个文件的内容” . 必填属性是 command 和 path.

任务提示的完整中文译文如下，模板占位符原样保留：

我已在目录 `{self.git_tempdir}` 中上传一个 Python 代码仓库。请帮助解决以下问题。

```
<problem_description>
{self.problem_statement}
</problem_description>

<test_description>
{self.test_description}
</test_description>
```

你的任务是修改 `{self.git_tempdir}` 目录中的文件，解决 `<problem_description>`。所需依赖我已处理好。

C.2 父代选择

每次 DGM 迭代从档案选取一组父智能体，自我修改并产生新子代. 下面给出受 Ecoffet 等 [36] 启发的选择细节. 未来可探索其他搜索与探索方法 [29, 144, 59].

在第 t 次迭代，令候选智能体档案为

$$\mathcal{A}^t = \{a_1^t, a_2^t, \ldots, a_N^t\}.$$

首先定义合资格集合

$$\mathcal{E}^t = \{a_i^t \in \mathcal{A}^t : \alpha_i < 1\},$$

即表现分数尚未完美的智能体. 我们从 $\mathcal{E}^t$ 有放回抽取 k 个父代，概率依据表现和具有代码库编辑功能的子代数的组合量. 具体地，对每个合资格智能体，

$$\text{表现：} \quad \alpha_i = \text{performance}(a_i^t), \tag{1.1}$$

$$\text{子代数：} \quad n_i = \text{functioning_children_count}(a_i^t), \tag{1.2}$$

$$\text{sigmoid 缩放表现：} \quad s_i = \frac{1}{1+\exp(-\lambda(\alpha_i - \alpha_0))}, \tag{1.3}$$

$$\text{新颖性奖励：} \quad h_i = \frac{1}{1+n_i}, \tag{1.4}$$

$$\text{未归一化权重：} \quad w_i = s_i h_i, \tag{1.5}$$

$$\text{选择概率：} \quad p_i = \frac{w_i}{\sum_{j=1}^{N} w_j}, \tag{1.6}$$

$$\text{采样：} \quad \{\text{父代}\} \sim \text{Categorical}(\{p_i\}_{a_i^t \in \mathcal{E}^t}) \ \text{（抽取 } k \text{ 次）}. \tag{1.7}$$

$\lambda > 0$ 控制 sigmoid 的陡峭程度，α_0 是准确率中点. 我们取 $\lambda = 10, \alpha_0 = 0.5$. k 决定每次迭代并行进行多少次自我改进：SWE-bench 取 2，Polyglot 取 4.

注 译者注：原文公式中的归一化求和上限写作 N，同时文字规定仅从合资格集合采样；上式照原文保留. 实现核对时应明确归一化是否限于合资格下标，而不能把这一记号问题当作额外的新算法.

C.3 自我改进提示

每次迭代选择一个智能体自我修改。当前实现利用另一个基础模型诊断改进机会，再传给编码智能体实现。为识别下一项潜在改进，把所选智能体的编码基准评价日志输入基础模型。尽管目前另设诊断步骤，但原则上并无根本限制阻止 DGM 自主分析表现、产生改进假说并实现后续修改。我们选择 OpenAI o1[153]，因为发现其推理能力优于实验时可用的其他模型。以下给出诊断提示。从诊断输出提取 `implementation_suggestion` 与 `problem_description`，作为编码智能体的自我修改任务。

```
# 编码智能体概要

- ** 主文件 **: `coding_agent.py`
  - 主要类: `AgenticSystem`
  - `forward()` 函数是核心入口。
  - 提示位于 `forward()` 函数内部或 `prompts/` 目录中。
- ** 工具 **: `tools/`
  - `tools/` 目录包含供大语言模型执行特定任务的各种工具。
  - 每个工具必须具有 `tool_info()`，返回含 'name'、'description'、'input_schema' 的 JSON 对象。
  ↪ 'input_schema' 应是包含 'type'、'properties'、'required' 的 JSON 对象。
  - 每个工具必须具有 `tool_function()`，接收 input_schema 定义的参数，执行工具任务，并返回字符串。
  - 可参考其他工具。
- ** 实用函数 **: `utils/`
  - `utils/` 目录包含代码库各处使用的实用函数。

- ** 补充细节 **:
  - 智能体很擅长在适当时机自动使用合适的可用工具。因此不要让智能体流程显式强制使用某个工具。
  - 文件编辑、bash 命令等常见工具容易被识别并适当使用。但复杂、小众工具可能需要在提示中明确说明。
  - 工具应尽可能通用，确保能在任意 GitHub 仓库中工作。避免硬编码路径等仓库专有细节或行为。
  - 不要在智能体代码里使用 'while True' 循环。这可能使智能体卡住而无法响应。
  - 使用辅助函数前，先验证其实现细节，保证正确集成与预期行为。
  - 不要直接安装额外软件包或依赖。如需新依赖，更新 `requirements.txt`，用 `pip install -r
  ↪ requirements.txt` 安装。

以下是编码智能体的实现。

# 编码智能体实现
----- 编码智能体实现开始 -----
```

{code}
----- 编码智能体实现结束 -----

你的任务是识别一项能够提高智能体编码能力的详细计划。改进不应针对某个特定 GitHub issue 或仓库。

以下是编码智能体尝试解决 GitHub issue 但失败的日志。

智能体运行日志
----- 智能体运行日志开始 -----
{md_log}
----- 智能体运行日志结束 -----

GitHub Issue
智能体尝试解决的 GitHub issue。
----- GitHub Issue 开始 -----
{github_issue}
----- GitHub Issue 结束 -----

预测补丁
智能体为解决问题预测的补丁。
----- 预测补丁开始 -----
{predicted_patch}
----- 预测补丁结束 -----

私有测试补丁
SWE-bench 用于检测问题是否解决的官方私有测试。评价时智能体无法访问。智能体应尝试实现自己的测试。
----- 私有测试补丁开始 -----
{test_patch}
----- 私有测试补丁结束 -----

Issue 测试结果
SWE-bench 使用上述官方私有测试得到的结果。
----- Issue 测试结果开始 -----
{eval_log}

```
----- Issue 测试结果结束 -----

请严格按以下格式回答，包括 JSON 开始和结束标记：

```json
<JSON>
```

在 <JSON> 中，提供包含以下字段的 JSON 回答：
- "log_summarization"：分析以上日志，总结智能体如何尝试解决 GitHub issue。说明使用哪些工具、
↪ 如何使用、问题求解方式，以及遇到的任何问题。
- "potential_improvements"：识别能够增强编码能力的潜在改进。聚焦通用编码能力，
↪ 例如可用于任意仓库的更好或新工具，而非针对 issue 的修复，例如只能用于某框架的工具。
↪ 全部必要依赖和环境设置已处理，因此不要聚焦这些方面。
- "improvement_proposal"：从潜在改进中选择一项高影响改进，详细描述。这应是一份聚焦且全面的计划，
↪ 提高智能体总体编码能力。
- "implementation_suggestion"：参考智能体概要和实现，认真考虑增加或改善何种功能、工具最能实现提议。
↪ 如果修改现有工具就可实现功能，应描述必要修改，而非建议新工具。
- "problem_description"：把改进提案和实现建议表述为 GitHub issue 描述。应清楚说明功能，使查看
↪ issue 与仓库的软件工程师能够实现。

你的回答会被自动解析，因此确保字符串严格遵循正确格式。输出中不要包含 `<JSON>` 标签。
```

虽然诊断提示中包含私有测试补丁，可以设想智能体通过硬编码解答轻易得高分，但我们人工检查日志和修改后，并未观察到记忆或过拟合特定私有测试所表现的问题逻辑或行为. 此外，在一个任务或大语言模型上发现的改进往往能泛化到其他任务和模型（图1.4），说明这些改进不是任务专用的，而是提高总体表现的一般变化.

Polyglot 实验的补充提示：

```
### 文档：工具函数模式

认真考虑是增加或增强当前工具，还是编辑 forward() 中的工作流。

特别注意确保 "required" 和 "type" 始终位于正确的嵌套层级。例如，"required" 应与 "properties" 同级，
↪ 而不是位于其内部。
确保每个属性，无论多短，都有正确嵌套的 type 和 description。
```

不允许使用你已见过参数之外的其他参数。例如，type 为 array 的 "edit_line_ranges" 不允许使用 "minItems" 和 "maxItems"。

...

以下是编码智能体尝试解决一项编程任务的日志。
每项任务使用一种编程语言，但智能体需要处理不同语言，包括 C++、Go、Java、JavaScript、Python 和 Rust。

C.4 伪代码

以下是第3节 DGM 算法的伪代码. 伪代码操作名保留与原文对应关系，注释完整译为中文. **原文算法 1：达尔文哥德尔机**

1. 输入：初始编码智能体 g_0、基准套件 B、最大迭代数 T；输出：智能体档案 $\mathcal{A}$.
2. $s_0 \leftarrow \text{evaluate}(g_0, B)$，评价基础智能体；初始化 $\mathcal{A} \leftarrow \{(g_0, s_0)\}$，从基础智能体开始.
3. 对 $t = 1, \ldots, T$：令 $\mathcal{P} \leftarrow \text{SelectParents}(\mathcal{A})$，选择父智能体. 对每个 $p \in \mathcal{P}$，执行 $c \leftarrow p.\text{modify}(p)$，自我修改；再令 $s \leftarrow \text{evaluate}(c, B)$，在基准上评价. 若 c.is_valid()，则 $\mathcal{A} \leftarrow \mathcal{A} \cup \{(c, s)\}$，保留具备代码编辑能力的子代.
4. 返回 $\mathcal{A}$.

以下是第4.3节无自我改进智能体基线的伪代码. **原文算法 2：无自我改进智能体的达尔文哥德尔机**

1. 输入：g_0, B, T；输出：档案 $\mathcal{A}$.
2. $s_0 \leftarrow \text{evaluate}(g_0, B)$，评价基础智能体；初始化 $\mathcal{A} \leftarrow \{(g_0, s_0)\}$.
3. 对 $t = 1, \ldots, T$：令 $\mathcal{P} \leftarrow \text{SelectParents}(\mathcal{A})$. 对每个 $p \in \mathcal{P}$，执行 $c \leftarrow g_0.\text{modify}(p)$，使用基础智能体修改；令 $s \leftarrow \text{evaluate}(c, B)$. 若 c.is_valid()，则 $\mathcal{A} \leftarrow \mathcal{A} \cup \{(c, s)\}$，保留能编辑代码的子代.
4. 返回 $\mathcal{A}$.

以下是第4.3节无开放式探索基线的伪代码. **原文算法 3：无开放式探索的达尔文哥德尔机**

1. 输入：g_0, B, T；输出：档案 $\mathcal{A}$.
2. $s_0 \leftarrow \text{evaluate}(g_0, B)$，评价基础智能体；初始化 $\mathcal{A} \leftarrow \{(g_0, s_0)\}$.
3. 对 $t = 1, \ldots, T$：令 $\mathcal{P} \leftarrow \text{SelectParents}(\mathcal{A})$. 对每个 $p \in \mathcal{P}$，执行 $c \leftarrow p.\text{modify}(p)$，自我修改；令 $s \leftarrow \text{evaluate}(c, B)$. 若 c.is_valid()，则 $\mathcal{A} \leftarrow \{(c, s)\}$，只保留最新智能体.
4. 返回 $\mathcal{A}$.

1.19 附录 D 实验细节

D.1 基础模型超参数

表1.3列出每种实验设置的基础模型，与第4.1节一致. SWE-bench 更具挑战，因此依据预先测试使用更强编码模型 Claude 3.5 Sonnet（New）. 为加快迭代并避免与 Claude 相同的速率限制，Polyglot 使用 o3-mini. 每种设置下所有基础模型的温度都设为 1.0.

表 1.3: 各实验设置使用的基础模型.

基准	SWE-bench	Polyglot
自我修改	Claude 3.5 Sonnet（New）	Claude 3.5 Sonnet（New）
评价	Claude 3.5 Sonnet（New）	o3-mini

1.20 附录 E 基准细节

E.1 费用估计

完成第4节一次 SWE-bench DGM 运行的估计费用约 22000 美元. 对比之下，两个基线各一次运行约 10000 美元. 尽管 DGM 费用明显更高，但能持续改进的方法，即使代价更高，也优于不能改进或停滞在可能永远达不到 DGM 的水平的方法. 更细的费用如下. 我们承认当前 SWE-bench 实验需要大量计算，因此还纳入费用明显较低的

表 1.4: 费用细分（原文该表无标题、未编号）.

模型	基准	任务数	费用（美元）
Claude 3.5 Sonnet（New）	SWE-bench	60	350
o3-mini	Polyglot	60	5

Polyglot 实验. 这说明费用随任务复杂性有很大变化，SWE-bench 是较复杂、资源需求较高的编码基准之一. 许多有影响力的方法，如早期大语言模型训练，最初也有很高计算需求. 与这些开创性工作类似，我们希望为未来提高方法效率与可扩展性的研究打开大门. 此外，SWE-bench 排行榜的许多领先智能体由工业企业支持，雇用全职专家研究员与工程师，产生可观人工成本. 本方法则通过无需人工干预的完全自主自我改进达到最佳水平；若比较专门人工智能开发人才与 API 使用的成本，可能更有效率. 最后，随着基础模型继续进步、计算费用继续下降，DGM 等方法会越来越高效且易获得.

DGM 发现的高表现智能体确实比初始智能体有更高推理费用，但费用与表现并非严格相关；有些昂贵智能体不如便宜者.

E.2 SWE-bench 任务

验证基础功能的初始 10 项任务：

```
django__django-10973
django__django-11066
django__django-12754
django__django-15930
django__django-13279
django__django-16661
django__django-13346
django__django-10880
django__django-10999
```

`django__django-11087`

估计总体有效性的额外 50 项任务：

`django__django-9296`
`django__django-11790`
`django__django-11815`
`django__django-11848`
`django__django-11880`
`django__django-11885`
`django__django-11951`
`django__django-11964`
`django__django-11999`
`django__django-12039`
`django__django-12050`
`django__django-12143`
`django__django-12155`
`django__django-12193`
`django__django-12209`
`django__django-12262`
`django__django-12273`
`django__django-12276`
`django__django-12304`
`django__django-12308`
`django__django-12325`
`django__django-12406`
`django__django-12708`
`django__django-12713`
`django__django-12774`
`sphinx-doc__sphinx-7454`
`sphinx-doc__sphinx-7590`
`sphinx-doc__sphinx-7748`
`sphinx-doc__sphinx-7757`
`sphinx-doc__sphinx-7985`
`sphinx-doc__sphinx-8035`

sphinx-doc__sphinx-8056

sphinx-doc__sphinx-8265

sphinx-doc__sphinx-8269

sphinx-doc__sphinx-8475

sphinx-doc__sphinx-8548

sphinx-doc__sphinx-8551

sphinx-doc__sphinx-8638

sphinx-doc__sphinx-8721

sphinx-doc__sphinx-9229

sphinx-doc__sphinx-9230

sphinx-doc__sphinx-9281

sphinx-doc__sphinx-9320

sphinx-doc__sphinx-9367

sphinx-doc__sphinx-9461

sphinx-doc__sphinx-9698

sphinx-doc__sphinx-10449

sphinx-doc__sphinx-10466

sphinx-doc__sphinx-10673

sphinx-doc__sphinx-11510

更精确评价表现的额外 140 项任务：

astropy__astropy-12907

astropy__astropy-13033

astropy__astropy-13236

astropy__astropy-13398

astropy__astropy-13453

astropy__astropy-13579

astropy__astropy-13977

astropy__astropy-14096

astropy__astropy-14182

astropy__astropy-14309

astropy__astropy-14365

astropy__astropy-14369

astropy__astropy-14508

astropy__astropy-14539

astropy__astropy-14598

astropy__astropy-14995

astropy__astropy-7166

astropy__astropy-7336

astropy__astropy-7606

astropy__astropy-7671

astropy__astropy-8707

astropy__astropy-8872

django__django-10097

django__django-10554

django__django-10914

django__django-11095

django__django-11099

django__django-11119

django__django-11133

django__django-11138

django__django-11141

django__django-11149

django__django-11163

django__django-11179

django__django-11206

django__django-11211

django__django-11239

django__django-11265

django__django-11276

django__django-11292

django__django-11299

django__django-11333

django__django-11400

django__django-11433

django__django-11451

django__django-11477

django__django-11490

django__django-11532

django__django-11551

django__django-11555

django__django-11603

django__django-11728

django__django-11734

django__django-11740

django__django-11749

django__django-11820

django__django-12125

django__django-12419

django__django-12663

django__django-12741

django__django-12858

django__django-12965

django__django-13012

django__django-13023

django__django-13028

django__django-13033

django__django-13089

django__django-13109

django__django-13112

django__django-13121

django__django-13128

django__django-13158

django__django-13195

django__django-13212

django__django-13297

django__django-13315

django__django-13343

django__django-13344

django__django-13363

django__django-13401

django__django-13406

django__django-13410

django__django-13417

django__django-13449

django__django-13512

django__django-13513

django__django-13516

django__django-13551

django__django-13568

django__django-13569

django__django-13590

django__django-13658

django__django-13670

django__django-13741

django__django-13786

django__django-13794

django__django-13807

django__django-13809

django__django-13810

django__django-13820

django__django-13821

django__django-13837

django__django-13925

django__django-13933

django__django-13964

django__django-14007

django__django-14011

django__django-14017

django__django-14034

django__django-14053

django__django-14089

django__django-14122

django__django-14140

django__django-14155

django__django-14170

```
django__django-14238
django__django-14311
django__django-14315
django__django-14349
django__django-14351
django__django-14373
django__django-14376
django__django-14404
django__django-14434
django__django-14493
django__django-14500
django__django-14534
django__django-14539
django__django-14559
django__django-14580
django__django-14608
django__django-14631
django__django-14672
django__django-14725
django__django-14752
django__django-14765
django__django-14771
django__django-14787
django__django-14792
django__django-14855
```

E.3 Polyglot 任务

验证基本功能的初始 10 项任务：

```
go__dominoes
cpp__all-your-base
python__dominoes
java__sgf-parsing
javascript__robot-name
rust__variable-length-quantity
```

python__beer-song

go__book-store

javascript__bottle-song

rust__bowling

估计总体有效性的额外 50 项任务：

javascript__queen-attack

rust__wordy

python__dot-dsl

java__satellite

cpp__diamond

rust__accumulate

go__error-handling

cpp__queen-attack

rust__poker

python__sgf-parsing

rust__react

java__ledger

go__connect

rust__macros

javascript__triangle

java__zipper

java__bowling

python__tree-building

javascript__say

java__wordy

python__food-chain

javascript__wordy

python__poker

javascript__grade-school

cpp__gigasecond

java__forth

python__dominoes

go__word-search

javascript__simple-linked-list

go__counter

java__react

javascript__ocr-numbers

python__scale-generator

java__go-counting

rust__doubly-linked-list

python__grade-school

javascript__forth

python__wordy

java__mazy-mice

cpp__bank-account

python__zipper

java__custom-set

java__rest-api

go__transpose

rust__gigasecond

rust__say

go__food-chain

rust__pig-latin

go__markdown

go__crypto-square

E.4 SWE-bench 最佳方案

在撰写论文时，即 2025 年 4 月 16 日，SWE-bench Verified 上表现最高、经核验且开源的参赛方案是 OpenHands + CodeAct v2.1（claude-3-5-sonnet-20241022）[158]，达到 53.0%. “经核验”指 SWE-bench 团队获准访问系统，能复现补丁生成. 只考虑 DGM 使用的同一 200 任务子集时，该方案达到 51.0%.

E.5 Polyglot 代表性智能体

社区流行编码智能体 Aider[47] 在 2024 年春发布，GitHub 星标超过 33000；人类开发者持续开发，并针对 Polyglot 测试一年多，主要用于评价其表现. Aider 也成为评价不同模型的标准基线，原文所述当前 Polyglot 最强模型是 o3（high）与 GPT-4.1 的组合. 我们采用与排行榜相似的设置，但关键区别是排行榜报告 pass@2，允许查看一次真实测试反馈；我们使用 pass@1，从不让智能体看到真实测试结果，认为这更符合实际编码应用.

1.21 附录 F 发现的最佳智能体

F.1 SWE-bench 上的 DGM

以下完整差异补丁依次促成 DGM 在 SWE-bench 上发现的最佳智能体. 程序、英文字符串与注释按原文保留，保留所有上下文行与增删标记；并非重新实现或概括.

原文补丁 F.1.1

```
diff --git a/coding_agent.py b/coding_agent.py
index 2cd395a..9a2cc2f 100644
--- a/coding_agent.py
+++ b/coding_agent.py
@@ -4,6 +4,7 @@ import logging
 from logging.handlers import RotatingFileHandler
 import os
 import threading
+import re

 from llm_withtools import CLAUDE_MODEL, OPENAI_MODEL, chat_with_agent
 from utils.eval_utils import get_report_score, msg_history_to_report, score_tie_breaker
@@ -63,6 +64,42 @@ def safe_log(message, level=logging.INFO):
     else:
         print(f"Warning: No logger found for thread {threading.get_ident()}")

+def is_patch_valid(patch_str):
+    """
+    Parse the patch to check if any non-test source files are modified.
+    Returns (bool, str) tuple: (is_valid, reason)
+    """
+    if not patch_str or patch_str.isspace():
+        return False, "Empty patch"
+
+    # Parse the patch to find modified files
+    modified_files = []
+    diff_header_pattern = re.compile(r'^\+\+\+ b/(.+)$', re.MULTILINE)
+    for match in diff_header_pattern.finditer(patch_str):
```

```
+        filepath = match.group(1)
+        if filepath != '/dev/null':  # Skip deleted files
+            modified_files.append(filepath)
+
+    if not modified_files:
+        return False, "No files modified"
+
+    # Check if any non-test files are modified
+    test_patterns = (
+        lambda f: f.startswith('tests/'),
+        lambda f: f.startswith('test_'),
+        lambda f: f.endswith('_test.py')
+    )
+
+    source_files = [
+        f for f in modified_files
+        if not any(pattern(f) for pattern in test_patterns)
+    ]
+
+    if not source_files:
+        return False, "Only test files were modified"
+
+    return True, "Valid patch with source file modifications"
+
 class AgenticSystem:
     def __init__(
             self,
@@ -73,6 +110,7 @@ class AgenticSystem:
             test_description=None,
             self_improve=False,
             instance_id=None,
+            max_retries=3,
         ):
         self.problem_statement = problem_statement
```

```
         self.git_tempdir = git_tempdir
@@ -82,6 +120,7 @@ class AgenticSystem:
         self.self_improve = self_improve
         self.instance_id = instance_id if not self_improve else 'dgm'
         self.code_model = CLAUDE_MODEL
+        self.max_retries = max_retries

         # Initialize logger and store it in thread-local storage
         self.logger = setup_logger(chat_history_file)
@@ -153,7 +192,7 @@ Your task is to run the regression tests in the {self.git_tempdir}
 ↪  directory to
         """
         The forward function for the AgenticSystem.
         """
-        instruction = f"""I have uploaded a Python code repository in the directory
 ↪  {self.git_tempdir}. Help solve the following problem.
+        base_instruction = f"""I have uploaded a Python code repository in the directory
 ↪  {self.git_tempdir}. Help solve the following problem.

 <problem_description>
 {self.problem_statement}
@@ -165,7 +204,39 @@ Your task is to run the regression tests in the {self.git_tempdir}
 ↪  directory to

 Your task is to make changes to the files in the {self.git_tempdir} directory to address the
  ↪  <problem_description>. I have already taken care of the required dependencies.
 """
-        new_msg_history = chat_with_agent(instruction, model=self.code_model, msg_history=[],
 ↪  logging=safe_log)
+
+        retry_count = 0
+        while retry_count < self.max_retries:
+            safe_log(f"\n=== Attempt {retry_count + 1} of {self.max_retries} ===")
+
```

```
+            # Reset to base commit before each attempt
+            if retry_count > 0:
+                reset_to_commit(self.git_tempdir, self.base_commit)
+
+            # Add retry context to instruction if this is a retry attempt
+            instruction = base_instruction
+            if retry_count > 0:
+                instruction += f"""\nNOTE: Previous attempt(s) failed because they either
↪  produced empty patches or only modified test files.
+Please ensure your solution includes changes to the main source code files, not just test
↪  files."""
+
+            # Run the agent
+            new_msg_history = chat_with_agent(instruction, model=self.code_model,
↪  msg_history=[], logging=safe_log)
+
+            # Check the patch
+            patch = self.get_current_edits()
+            is_valid, reason = is_patch_valid(patch)
+
+            if is_valid:
+                safe_log(f"Valid patch generated: {reason}")
+                break
+            else:
+                safe_log(f"Invalid patch: {reason}")
+                if retry_count < self.max_retries - 1:
+                    safe_log("Retrying with a new attempt...")
+                else:
+                    safe_log("Maximum retries reached. Unable to generate a valid patch.")
+
+            retry_count += 1

 def main():
     parser = argparse.ArgumentParser(description='Process repository with an agentic system.')
```

```
@@ -177,6 +248,7 @@ def main():
     parser.add_argument('--test_description', default=None, required=False, help='Description
     ↪  of how to test the repository')
     parser.add_argument('--self_improve', default=False, action='store_true', help='Whether to
     ↪  self-improve the repository or solving swe')
     parser.add_argument('--instance_id', default=None, help='Instance ID for SWE issue')
+    parser.add_argument('--max_retries', type=int, default=3, help='Maximum number of patch
↪  generation attempts')
     args = parser.parse_args()

     # Process the repository
@@ -188,6 +260,7 @@ def main():
         test_description=args.test_description,
         self_improve=args.self_improve,
         instance_id=args.instance_id,
+        max_retries=args.max_retries,
     )

     # Run the agentic system to try to solve the problem
@@ -200,4 +273,4 @@ def main():
         f.write(model_patch)

 if __name__ == "__main__":
-    main()
+    main()
\ No newline at end of file
diff --git a/tests/test_patch_validator.py b/tests/test_patch_validator.py
new file mode 100644
index 0000000..5689f7d
--- /dev/null
+++ b/tests/test_patch_validator.py
@@ -0,0 +1,77 @@
+import pytest
+from coding_agent import is_patch_valid
```

```
+
+def test_empty_patch():
+    # Test empty patch
+    is_valid, reason = is_patch_valid("")
+    assert not is_valid
+    assert reason == "Empty patch"
+
+    # Test whitespace-only patch
+    is_valid, reason = is_patch_valid("   \n   ")
+    assert not is_valid
+    assert reason == "Empty patch"
+
+def test_test_only_patch():
+    patch = """
+diff --git a/tests/test_edit_tool.py b/tests/test_edit_tool.py
+index abc123..def456 100644
+--- a/tests/test_edit_tool.py
++++ b/tests/test_edit_tool.py
+@@ -10,6 +10,8 @@ def test_something():
+     assert True
++    assert 1 == 1
+"""
+    is_valid, reason = is_patch_valid(patch)
+    assert not is_valid
+    assert reason == "Only test files were modified"
+
+def test_source_file_patch():
+    patch = """
+diff --git a/tools/edit.py b/tools/edit.py
+index abc123..def456 100644
+--- a/tools/edit.py
++++ b/tools/edit.py
+@@ -10,6 +10,8 @@ class Editor:
+     def edit(self):
```

```
+         pass
++        return True
+"""
+    is_valid, reason = is_patch_valid(patch)
+    assert is_valid
+    assert reason == "Valid patch with source file modifications"
+
+def test_mixed_files_patch():
+    patch = """
+diff --git a/tools/edit.py b/tools/edit.py
+index abc123..def456 100644
+--- a/tools/edit.py
++++ b/tools/edit.py
+@@ -10,6 +10,8 @@ class Editor:
+     def edit(self):
+         pass
++        return True
+
+diff --git a/tests/test_edit.py b/tests/test_edit.py
+index abc123..def456 100644
+--- a/tests/test_edit.py
++++ b/tests/test_edit.py
+@@ -10,6 +10,8 @@ def test_something():
+     assert True
++    assert 1 == 1
+"""
+    is_valid, reason = is_patch_valid(patch)
+    assert is_valid
+    assert reason == "Valid patch with source file modifications"
+
+def test_no_files_modified():
+    patch = """
+diff --git a/nonexistent.py b/nonexistent.py
+deleted file mode 100644
```

```
+index abc123..0000000
+--- a/nonexistent.py
++++ /dev/null
+"""
+    is_valid, reason = is_patch_valid(patch)
+    assert not is_valid
+    assert reason == "No files modified"
\ No newline at end of file
```

原文补丁 F.1.2

```
diff --git a/tools/edit.py b/tools/edit.py
index 59137ee..16ae521 100644
--- a/tools/edit.py
+++ b/tools/edit.py
@@ -1,16 +1,17 @@
 from pathlib import Path
 import subprocess
+from typing import Optional, List, Tuple, Union

 def tool_info():
     return {
         "name": "editor",
         "description": """Custom editing tool for viewing, creating, and editing files\n
 * State is persistent across command calls and discussions with the user.\n
-* If `path` is a file, `view` displays the entire file with line numbers. If `path` is a
↪  directory, `view` lists non-hidden files and directories up to 2 levels deep.\n
+* If `path` is a file, `view` displays the file with line numbers. With optional `view_range`
↪  [start, end], it displays only specified lines. Use -1 in `end` for all remaining lines.\n
+* If `path` is a directory, `view` lists non-hidden files and directories up to 2 levels
↪  deep.\n
 * The `create` command cannot be used if the specified `path` already exists as a file.\n
 * If a `command` generates a long output, it will be truncated and marked with `<response
↪  clipped>`.\n
-* The `edit` command overwrites the entire file with the provided `file_text`.\n
```

```
-* No partial/line-range edits or partial viewing are supported.""",
+* The `edit` command overwrites the entire file with the provided `file_text`.""",
         "input_schema": {
             "type": "object",
             "properties": {
@@ -26,6 +27,13 @@ def tool_info():
                 "file_text": {
                     "description": "Required parameter of `create` or `edit` command,
                     ↪  containing the content for the entire file.",
                     "type": "string"
+                },
+                "view_range": {
+                    "description": "Optional parameter for `view` command. Array of
↪  [start_line, end_line] (1-based). Use -1 for end_line to read until end of file.",
+                    "type": "array",
+                    "items": {"type": "integer"},
+                    "minItems": 2,
+                    "maxItems": 2
                 }
             },
             "required": ["command", "path"]
@@ -89,6 +97,46 @@ def read_file(path: Path) -> str:
     except Exception as e:
         raise ValueError(f"Failed to read file: {e}")

+def read_file_range(path: Path, line_range: Optional[List[int]] = None) -> Tuple[str, int]:
+    """
+    Read and return file contents within specified line range.
+    Returns tuple of (content, start_line).
+
+    Args:
+        path: Path object for the file
+        line_range: Optional [start, end] line numbers (1-based). Use -1 for end to read until
↪  EOF.
```

```
+    """
+    try:
+        if line_range is None:
+            return read_file(path), 1
+
+        start, end = line_range
+        if start < 1:
+            raise ValueError("Start line must be >= 1")
+        if end != -1 and end < start:
+            raise ValueError("End line must be >= start line or -1")
+
+        with path.open() as f:
+            # Skip lines before start
+            for _ in range(start - 1):
+                next(f, None)
+
+            lines = []
+            current_line = start
+            while True:
+                line = next(f, None)
+                if line is None:  # EOF
+                    break
+                if end != -1 and current_line > end:
+                    break
+                lines.append(line.rstrip('\n'))
+                current_line += 1
+
+        return '\n'.join(lines), start
+
+    except Exception as e:
+        raise ValueError(f"Failed to read file range: {e}")
+
 def write_file(path: Path, content: str):
     """Write (overwrite) entire file contents."""
```

```
     try:
@@ -96,9 +144,18 @@ def write_file(path: Path, content: str):
     except Exception as e:
         raise ValueError(f"Failed to write file: {e}")

-def view_path(path_obj: Path) -> str:
-    """View the entire file contents or directory listing."""
+def view_path(path_obj: Path, view_range: Optional[List[int]] = None) -> str:
+    """
+    View the file contents (optionally within a range) or directory listing.
+
+    Args:
+        path_obj: Path object for the file or directory
+        view_range: Optional [start, end] line numbers for file viewing
+    """
     if path_obj.is_dir():
+        if view_range is not None:
+            raise ValueError("view_range is not supported for directory listings")
+
         # For directories: list non-hidden files up to 2 levels deep
         try:
             result = subprocess.run(
@@ -115,14 +172,14 @@ def view_path(path_obj: Path) -> str:
         except Exception as e:
             raise ValueError(f"Failed to list directory: {e}")

-    # If it's a file, show the entire file with line numbers
-    content = read_file(path_obj)
-    return format_output(content, str(path_obj))
+    # If it's a file, show the file content (with optional line range)
+    content, start_line = read_file_range(path_obj, view_range)
+    return format_output(content, str(path_obj), start_line)

-def tool_function(command: str, path: str, file_text: str = None) -> str:
```

```
+def tool_function(command: str, path: str, file_text: str = None, view_range:
↪  Optional[List[int]] = None) -> str:
     """
     Main tool function that handles:
-      - 'view'  : View the entire file or directory listing
+      - 'view'  : View file or directory listing, optionally within line range for files
       - 'create': Create a new file with the given file_text
       - 'edit'  : Overwrite an existing file with file_text
     """
@@ -130,7 +187,7 @@ def tool_function(command: str, path: str, file_text: str = None) -> str:
         path_obj = validate_path(path, command)

         if command == "view":
-            return view_path(path_obj)
+            return view_path(path_obj, view_range)

         elif command == "create":
             if file_text is None:
@@ -152,4 +209,4 @@ def tool_function(command: str, path: str, file_text: str = None) -> str:

 if __name__ == "__main__":
     # Example usage
-    print(tool_function("view", "/home/ubuntu/xx/dgm/coding_agent.py"))
+    print(tool_function("view", "/home/ubuntu/xx/dgm/coding_agent.py"))
\ No newline at end of file
diff --git a/tests/test_tools/test_edit.py b/tests/test_tools/test_edit.py
new file mode 100644
index 0000000..04f535b
--- /dev/null
+++ b/tests/test_tools/test_edit.py
@@ -0,0 +1,54 @@
+import pytest
+from pathlib import Path
+from tools.edit import tool_function
```

```
+
+def test_view_line_range(tmp_path):
+    # Create a test file
+    test_file = tmp_path / "test.txt"
+    test_content = "line1\nline2\nline3\nline4\nline5\n"
+    test_file.write_text(test_content)
+
+    # Test viewing specific line range
+    result = tool_function("view", str(test_file), view_range=[2, 4])
+    assert "line2" in result
+    assert "line3" in result
+    assert "line4" in result
+    assert "line1" not in result
+    assert "line5" not in result
+    assert "     2\t" in result  # Correct line numbering
+
+    # Test viewing from start to middle
+    result = tool_function("view", str(test_file), view_range=[1, 3])
+    assert "line1" in result
+    assert "line2" in result
+    assert "line3" in result
+    assert "line4" not in result
+    assert "     1\t" in result
+
+    # Test viewing from middle to end with -1
+    result = tool_function("view", str(test_file), view_range=[3, -1])
+    assert "line1" not in result
+    assert "line2" not in result
+    assert "line3" in result
+    assert "line4" in result
+    assert "line5" in result
+    assert "     3\t" in result
+
+def test_view_range_validation(tmp_path):
```

```
+    # Create a test file
+    test_file = tmp_path / "test.txt"
+    test_content = "line1\nline2\nline3\n"
+    test_file.write_text(test_content)
+
+    # Test invalid start line
+    result = tool_function("view", str(test_file), view_range=[0, 2])
+    assert "Failed to read file range: Start line must be >= 1" in result
+
+    # Test invalid range (end < start)
+    result = tool_function("view", str(test_file), view_range=[2, 1])
+    assert "Failed to read file range: End line must be >= start line or -1" in result
+
+def test_view_range_with_directory(tmp_path):
+    # Test that view_range is rejected for directories
+    result = tool_function("view", str(tmp_path), view_range=[1, 10])
+    assert "Error: view_range is not supported for directory listings" in result
\ No newline at end of file
```

原文补丁 F.1.3

```
diff --git a/tools/edit.py b/tools/edit.py
index 16ae521..757f5c2 100644
--- a/tools/edit.py
+++ b/tools/edit.py
@@ -11,21 +11,21 @@ def tool_info():
 * If `path` is a directory, `view` lists non-hidden files and directories up to 2 levels
  ↪  deep.\n
 * The `create` command cannot be used if the specified `path` already exists as a file.\n
 * If a `command` generates a long output, it will be truncated and marked with `<response
  ↪  clipped>`.\n
-* The `edit` command overwrites the entire file with the provided `file_text`.""",
+* The `str_replace` command replaces a unique occurrence of old_str with new_str, failing if
 ↪  old_str is not found or appears multiple times.""",
         "input_schema": {
```

```
            "type": "object",
            "properties": {
                "command": {
                    "type": "string",
-                   "enum": ["view", "create", "edit"],
-                   "description": "The command to run: `view`, `create`, or `edit`."
+                   "enum": ["view", "create", "str_replace"],
+                   "description": "The command to run: `view`, `create`, or `str_replace`."
                },
                "path": {
                    "description": "Absolute path to file or directory, e.g. `/repo/file.py`
                    ↪  or `/repo`.",
                    "type": "string"
                },
                "file_text": {
-                   "description": "Required parameter of `create` or `edit` command,
↪  containing the content for the entire file.",
+                   "description": "Required parameter of `create` command, containing the
↪  content for the entire file.",
                    "type": "string"
                },
                "view_range": {
@@ -34,6 +34,14 @@ def tool_info():
                    "items": {"type": "integer"},
                    "minItems": 2,
                    "maxItems": 2
+               },
+               "old_str": {
+                   "description": "Required parameter of `str_replace` command, containing
↪  the exact text to find and replace.",
+                   "type": "string"
+               },
+               "new_str": {
```

```
+                    "description": "Required parameter of `str_replace` command, containing
↪  the new text to replace old_str with.",
+                    "type": "string"
                 }
             },
             "required": ["command", "path"]
@@ -51,7 +59,7 @@ def validate_path(path: str, command: str) -> Path:
     Validate the file path for each command:
       - 'view': path may be a file or directory; must exist.
       - 'create': path must not exist (for new file creation).
-      - 'edit': path must exist (for overwriting).
+      - 'str_replace': path must exist and be a file.
     """
     path_obj = Path(path)

@@ -69,7 +77,7 @@ def validate_path(path: str, command: str) -> Path:
         # Path must not exist
         if path_obj.exists():
             raise ValueError(f"Cannot create new file; {path} already exists.")
-    elif command == "edit":
+    elif command == "str_replace":
         # Path must exist and must be a file
         if not path_obj.exists():
             raise ValueError(f"The file {path} does not exist.")
@@ -144,6 +152,28 @@ def write_file(path: Path, content: str):
     except Exception as e:
         raise ValueError(f"Failed to write file: {e}")

+def str_replace_in_file(path: Path, old_str: str, new_str: str) -> str:
+    """
+    Replace an exact occurrence of old_str with new_str in the file.
+    Only performs the replacement if old_str occurs exactly once.
+    Returns a message indicating success or failure.
+    """
```

```
+    try:
+        content = read_file(path)
+        occurrences = content.count(old_str)
+
+        if occurrences == 0:
+            return f"Error: Could not find the exact text to replace in {path}"
+        elif occurrences > 1:
+            return f"Error: Found multiple ({occurrences}) occurrences of the text in {path}.
↪  Must be unique."
+        else:
+            new_content = content.replace(old_str, new_str)
+            write_file(path, new_content)
+            return f"Successfully replaced text in {path}"
+
+    except Exception as e:
+        return f"Error during string replacement: {e}"
+
 def view_path(path_obj: Path, view_range: Optional[List[int]] = None) -> str:
     """
     View the file contents (optionally within a range) or directory listing.
@@ -176,12 +206,13 @@ def view_path(path_obj: Path, view_range: Optional[List[int]] = None) ->
↪  str:
     content, start_line = read_file_range(path_obj, view_range)
     return format_output(content, str(path_obj), start_line)

-def tool_function(command: str, path: str, file_text: str = None, view_range:
↪  Optional[List[int]] = None) -> str:
+def tool_function(command: str, path: str, file_text: str = None, view_range:
↪  Optional[List[int]] = None,
+                  old_str: str = None, new_str: str = None) -> str:
     """
     Main tool function that handles:
-      - 'view'  : View file or directory listing, optionally within line range for files
-      - 'create': Create a new file with the given file_text
```

```
-      - 'edit'  : Overwrite an existing file with file_text
+      - 'view'       : View file or directory listing, optionally within line range for files
+      - 'create'     : Create a new file with the given file_text
+      - 'str_replace': Replace exact occurrence of old_str with new_str in the file
     """
     try:
         path_obj = validate_path(path, command)
@@ -195,11 +226,10 @@ def tool_function(command: str, path: str, file_text: str = None,
↪  view_range: Op
             write_file(path_obj, file_text)
             return f"File created successfully at: {path}"

-        elif command == "edit":
-            if file_text is None:
-                raise ValueError("Missing required `file_text` for 'edit' command.")
-            write_file(path_obj, file_text)
-            return f"File at {path} has been overwritten with new content."
+        elif command == "str_replace":
+            if old_str is None or new_str is None:
+                raise ValueError("Missing required `old_str` and/or `new_str` for
↪  'str_replace' command.")
+            return str_replace_in_file(path_obj, old_str, new_str)

         else:
             raise ValueError(f"Unknown command: {command}")
diff --git a/tests/__init__.py b/tests/__init__.py
new file mode 100644
index 0000000..e69de29
diff --git a/tests/test_tools.py b/tests/test_tools.py
new file mode 100644
index 0000000..c7f242f
--- /dev/null
+++ b/tests/test_tools.py
@@ -0,0 +1,65 @@
```

```
+import pytest
+from pathlib import Path
+from tools.edit import tool_function
+
+# Test fixtures
+@pytest.fixture
+def temp_file(tmp_path):
+    file_path = tmp_path / "test.txt"
+    content = "line 1\nline 2\nline 3\n"
+    file_path.write_text(content)
+    return str(file_path)
+
+def test_str_replace_success(temp_file):
+    # Test successful replacement
+    result = tool_function(
+        command="str_replace",
+        path=temp_file,
+        old_str="line 2\n",
+        new_str="replaced line\n"
+    )
+    assert "Successfully replaced" in result
+    assert Path(temp_file).read_text() == "line 1\nreplaced line\nline 3\n"
+
+def test_str_replace_not_found(temp_file):
+    # Test when old_str is not found
+    result = tool_function(
+        command="str_replace",
+        path=temp_file,
+        old_str="nonexistent",
+        new_str="something"
+    )
+    assert "Could not find" in result
+    # Original file should be unchanged
+    assert Path(temp_file).read_text() == "line 1\nline 2\nline 3\n"
```

```
+
+def test_str_replace_multiple_occurrences(temp_file):
+    # First create a file with multiple occurrences
+    Path(temp_file).write_text("same\nsame\nsame\n")
+    result = tool_function(
+        command="str_replace",
+        path=temp_file,
+        old_str="same\n",
+        new_str="different\n"
+    )
+    assert "multiple" in result
+    # Original file should be unchanged
+    assert Path(temp_file).read_text() == "same\nsame\nsame\n"
+
+def test_str_replace_missing_params(temp_file):
+    # Test missing parameters
+    result = tool_function(
+        command="str_replace",
+        path=temp_file,
+    )
+    assert "Missing required" in result
+
+def test_str_replace_invalid_path():
+    # Test with non-existent file
+    result = tool_function(
+        command="str_replace",
+        path="/nonexistent/path",
+        old_str="old",
+        new_str="new"
+    )
+    assert "does not exist" in result
\ No newline at end of file
```

原文补丁 F.1.4

```
diff --git a/llm_withtools.py b/llm_withtools.py
index d1394bb..6cc3604 100644
--- a/llm_withtools.py
+++ b/llm_withtools.py
@@ -29,7 +29,7 @@ def process_tool_call(tools_dict, tool_name, tool_input):
 )
 def get_response_withtools(
     client, model, messages, tools, tool_choice,
-    logging=None,
+    logging=None, system_message=None,
 ):
     try:
         if 'claude' in model:
@@ -52,13 +52,32 @@ def get_response_withtools(
             raise ValueError(f"Unsupported model: {model}")
         return response
     except Exception as e:
-        logging(f"Error in get_response_withtools: {str(e)}")
+        error_msg = str(e)
+        logging(f"Error in get_response_withtools: {error_msg}")

         # Hitting the context window limit
-        if 'Input is too long for requested model' in str(e):
-            pass
+        if 'Input is too long for requested model' in error_msg or 'maximum context length' in
↪  error_msg:
+            if not system_message:
+                # Extract system message from the first message if available
+                system_message = messages[0].get('content', '') if messages else ''
+                if isinstance(system_message, list):
+                    system_message = ' '.join(block['text'] for block in system_message if
↪  block['type'] == 'text')
+
```

```
+            # Summarize the conversation history
+            summarized_messages = summarize_messages(client, model, messages, system_message)
+
+            # Retry with summarized messages
+            return get_response_withtools(
+                client=client,
+                model=model,
+                messages=summarized_messages,
+                tools=tools,
+                tool_choice=tool_choice,
+                logging=logging,
+                system_message=system_message
+            )

-        raise  # Re-raise the exception after logging
+        raise  # Re-raise other exceptions

 def check_for_tool_use(response, model=''):
     """
@@ -247,6 +266,57 @@ def convert_msg_history_openai(msg_history):

     return new_msg_history

+def summarize_messages(client, model, messages, system_message):
+    """
+    Creates a condensed summary of older messages while preserving recent context.
+    Only summarizes assistant and user messages, keeps tool results as is for accuracy.
+    """
+    # Keep the most recent messages intact
+    recent_msgs = messages[-2:] if len(messages) > 2 else messages
+    if len(messages) <= 2:
+        return messages
+
+    # Prepare messages to be summarized
```

```
+    msgs_to_summarize = messages[:-2]
+
+    # Create a prompt to summarize the conversation
+    summary_request = "Please create a concise summary of this conversation that preserves the
↪  key context and important details:"
+    for msg in msgs_to_summarize:
+        if isinstance(msg.get('content', ''), list):
+            content = ' '.join(block['text'] for block in msg['content'] if block['type'] ==
↪  'text')
+        else:
+            content = str(msg.get('content', ''))
+        if msg.get('role') in ['assistant', 'user']:
+            summary_request += f"\n{msg['role']}: {content}"
+
+    try:
+        # Get summary from the model
+        summary_response, _ = get_response_from_llm(
+            msg=summary_request,
+            client=client,
+            model=model,
+            system_message="You are a summarizer. Create a concise but informative summary.",
+            print_debug=False,
+            msg_history=[]
+        )
+
+        # Create new message history with the summary
+        summarized_history = [{
+            "role": "system",
+            "content": [{"type": "text", "text": system_message}]
+        }, {
+            "role": "assistant",
+            "content": [{"type": "text", "text": f"Previous conversation summary:
↪  {summary_response}"}]
+        }]
```

```
+
+        # Add back the recent messages
+        summarized_history.extend(recent_msgs)
+
+        return summarized_history
+    except Exception:
+        # If summarization fails, return original messages with the most recent ones
+        return [messages[0]] + recent_msgs
+
 def convert_msg_history(msg_history, model=None):
     """
     Convert message history from the model-specific format to a generic format.
@@ -263,7 +333,14 @@ def chat_with_agent_manualtools(msg, model, msg_history=None,
↪  logging=print):
     if msg_history is None:
         msg_history = []
     system_message = f'You are a coding agent.\n\n{get_tooluse_prompt()}'
-    new_msg_history = msg_history
+    new_msg_history = msg_history.copy() if msg_history else []
+
+    # Ensure system message is the first message in history
+    if not new_msg_history or new_msg_history[0].get('role') != 'system':
+        new_msg_history.insert(0, {
+            "role": "system",
+            "content": [{"type": "text", "text": system_message}]
+        })

     try:
         # Load all tools
```

原文补丁 F.1.5

```
diff --git a/coding_agent.py b/coding_agent.py
index 9a2cc2f..3f1bc1d 100644
--- a/coding_agent.py
```

```
+++ b/coding_agent.py
@@ -111,6 +111,7 @@ class AgenticSystem:
             self_improve=False,
             instance_id=None,
             max_retries=3,
+            num_candidates=3,
         ):
         self.problem_statement = problem_statement
         self.git_tempdir = git_tempdir
@@ -121,6 +122,7 @@ class AgenticSystem:
         self.instance_id = instance_id if not self_improve else 'dgm'
         self.code_model = CLAUDE_MODEL
         self.max_retries = max_retries
+        self.num_candidates = num_candidates

         # Initialize logger and store it in thread-local storage
         self.logger = setup_logger(chat_history_file)
@@ -190,7 +192,7 @@ Your task is to run the regression tests in the {self.git_tempdir}
 ↪  directory to

     def forward(self):
         """
-        The forward function for the AgenticSystem.
+        The forward function for the AgenticSystem that generates and evaluates multiple
 ↪  candidate patches.
         """
         base_instruction = f"""I have uploaded a Python code repository in the directory
         ↪  {self.git_tempdir}. Help solve the following problem.

@@ -205,10 +207,18 @@ Your task is to run the regression tests in the {self.git_tempdir}
 ↪  directory to
 Your task is to make changes to the files in the {self.git_tempdir} directory to address the
 ↪  <problem_description>. I have already taken care of the required dependencies.
 """
```

```
+        # Get regression tests summary once at the start
+        regression_tests_summary = self.get_regression_tests()
+
+        # Lists to store candidates
+        valid_patches = []
+        valid_reports = []
+
         retry_count = 0
-        while retry_count < self.max_retries:
+        while retry_count < self.max_retries and len(valid_patches) < self.num_candidates:
             safe_log(f"\n=== Attempt {retry_count + 1} of {self.max_retries} ===")
-
+            safe_log(f"Valid solutions so far: {len(valid_patches)} of {self.num_candidates}
↪  desired")
+
             # Reset to base commit before each attempt
             if retry_count > 0:
                 reset_to_commit(self.git_tempdir, self.base_commit)
@@ -216,8 +226,8 @@ Your task is to make changes to the files in the {self.git_tempdir}
↪  directory to
             # Add retry context to instruction if this is a retry attempt
             instruction = base_instruction
             if retry_count > 0:
-                instruction += f"""\nNOTE: Previous attempt(s) failed because they either
↪  produced empty patches or only modified test files.
-Please ensure your solution includes changes to the main source code files, not just test
↪  files."""
+                instruction += f"""\nNOTE: Previous attempt(s) did not produce enough valid
↪  solutions.
+Please provide a different approach to solve the problem. Your solution must include changes
↪  to the main source code files, not just test files."""

             # Run the agent
```

```
            new_msg_history = chat_with_agent(instruction, model=self.code_model,
            ↪  msg_history=[], logging=safe_log)
@@ -228,16 +238,45 @@ Please ensure your solution includes changes to the main source code
↪  files, not

            if is_valid:
                safe_log(f"Valid patch generated: {reason}")
-                break
+                # Run regression tests for this candidate
+                test_report = self.run_regression_tests(regression_tests_summary)
+                test_score = get_report_score(test_report)
+                safe_log(f"Test score: {test_score}")
+
+                valid_patches.append(patch)
+                valid_reports.append(test_report)
+
+                if len(valid_patches) >= self.num_candidates:
+                    break
            else:
                safe_log(f"Invalid patch: {reason}")
-                if retry_count < self.max_retries - 1:
-                    safe_log("Retrying with a new attempt...")
-                else:
-                    safe_log("Maximum retries reached. Unable to generate a valid patch.")

            retry_count += 1

+        if not valid_patches:
+            safe_log("Failed to generate any valid patches.")
+            return
+
+        # Use score_tie_breaker to select the best patch
+        safe_log(f"\n=== Selecting Best Solution from {len(valid_patches)} Candidates ===")
+        best_index = score_tie_breaker(
```

```
+            self.problem_statement,
+            valid_patches,
+            valid_reports,
+            logging=safe_log
+        )
+
+        # Reset to base and apply the best patch
+        reset_to_commit(self.git_tempdir, self.base_commit)
+        best_patch = valid_patches[best_index]
+        safe_log(f"\n=== Applying Best Solution (Candidate {best_index + 1}) ===")
+        apply_patch(self.git_tempdir, best_patch)
+
+        # Final validation of the selected patch
+        final_test_report = self.run_regression_tests(regression_tests_summary)
+        final_score = get_report_score(final_test_report)
+        safe_log(f"Final solution test score: {final_score}")
+
 def main():
     parser = argparse.ArgumentParser(description='Process repository with an agentic system.')
     parser.add_argument('--problem_statement', required=True, help='The problem statement to
     ↪  process')
@@ -249,6 +288,7 @@ def main():
     parser.add_argument('--self_improve', default=False, action='store_true', help='Whether to
     ↪  self-improve the repository or solving swe')
     parser.add_argument('--instance_id', default=None, help='Instance ID for SWE issue')
     parser.add_argument('--max_retries', type=int, default=3, help='Maximum number of patch
     ↪  generation attempts')
+    parser.add_argument('--num_candidates', type=int, default=3, help='Number of candidate
↪  solutions to generate')
     args = parser.parse_args()

     # Process the repository
@@ -261,6 +301,7 @@ def main():
         self_improve=args.self_improve,
```

```
         instance_id=args.instance_id,
         max_retries=args.max_retries,
+        num_candidates=args.num_candidates,
     )

     # Run the agentic system to try to solve the problem
```

原文补丁 F.1.6

```
diff --git a/coding_agent.py b/coding_agent.py
index 3f1bc1d..588938d 100644
--- a/coding_agent.py
+++ b/coding_agent.py
@@ -193,42 +193,59 @@ Your task is to run the regression tests in the {self.git_tempdir}
↪  directory to
     def forward(self):
         """
         The forward function for the AgenticSystem that generates and evaluates multiple
         ↪  candidate patches.
+        This version maintains history of prior valid patches and test results, only using the
↪  tie-breaker
+        when necessary.
         """
-        base_instruction = f"""I have uploaded a Python code repository in the directory
↪  {self.git_tempdir}. Help solve the following problem.
-
-<problem_description>
-{self.problem_statement}
-</problem_description>
-
-<test_description>
-{self.test_description}
-</test_description>
-
```

```
-Your task is to make changes to the files in the {self.git_tempdir} directory to address the
↪  <problem_description>. I have already taken care of the required dependencies.
-"""
-
-        # Get regression tests summary once at the start
         regression_tests_summary = self.get_regression_tests()

-        # Lists to store candidates
+        # Lists to store all valid patches and their information
         valid_patches = []
         valid_reports = []
+        valid_scores = []
+        best_score = 0
+        best_patches_indices = []  # Indices of patches that share the best score

         retry_count = 0
         while retry_count < self.max_retries and len(valid_patches) < self.num_candidates:
             safe_log(f"\n=== Attempt {retry_count + 1} of {self.max_retries} ===")
             safe_log(f"Valid solutions so far: {len(valid_patches)} of {self.num_candidates}
             ↪  desired")
+            safe_log(f"Current best test score: {best_score}")

             # Reset to base commit before each attempt
             if retry_count > 0:
                 reset_to_commit(self.git_tempdir, self.base_commit)

-            # Add retry context to instruction if this is a retry attempt
-            instruction = base_instruction
-            if retry_count > 0:
-                instruction += f"""\nNOTE: Previous attempt(s) did not produce enough valid
↪  solutions.
+            # Construct instruction with previous best solutions if available
+            instruction = f"""I have uploaded a Python code repository in the directory
↪  {self.git_tempdir}. Help solve the following problem.
```

```
+
+<problem_description>
+{self.problem_statement}
+</problem_description>
+
+<test_description>
+{self.test_description}
+</test_description>"""
+
+            # Add previous solutions context if available
+            if valid_patches and retry_count > 0:
+                previous_solutions = []
+                for i, (patch, report, score) in enumerate(zip(valid_patches, valid_reports,
↪  valid_scores)):
+                    previous_solutions.append(f"""
+Previous Solution {i+1}:
+<code_changes>
+{patch}
+</code_changes>
+Test Score: {score}
+Test Report: {report}
+""")
+                instruction += "\n\nPrevious solution attempts:\n" +
↪  "\n".join(previous_solutions)
+                instruction += "\nPlease provide a new solution that addresses any limitations
↪  in the previous attempts or explores a different approach."
+            elif retry_count > 0:
+                instruction += """\nNOTE: Previous attempt(s) did not produce enough valid
↪  solutions.
 Please provide a different approach to solve the problem. Your solution must include changes
 ↪  to the main source code files, not just test files."""
```

```
+            instruction += f"\n\nYour task is to make changes to the files in the
↪  {self.git_tempdir} directory to address the <problem_description>. I have already taken
↪  care of the required dependencies."
+
             # Run the agent
             new_msg_history = chat_with_agent(instruction, model=self.code_model,
             ↪  msg_history=[], logging=safe_log)

@@ -245,6 +262,14 @@ Please provide a different approach to solve the problem. Your solution
↪  must inc

                 valid_patches.append(patch)
                 valid_reports.append(test_report)
+                valid_scores.append(test_score)
+
+                # Update best score and indices
+                if test_score > best_score:
+                    best_score = test_score
+                    best_patches_indices = [len(valid_patches) - 1]
+                elif test_score == best_score:
+                    best_patches_indices.append(len(valid_patches) - 1)

                 if len(valid_patches) >= self.num_candidates:
                     break
@@ -257,25 +282,30 @@ Please provide a different approach to solve the problem. Your solution
↪  must inc
             safe_log("Failed to generate any valid patches.")
             return

-        # Use score_tie_breaker to select the best patch
+        # Only use tie-breaker if we have multiple patches with the best score
         safe_log(f"\n=== Selecting Best Solution from {len(valid_patches)} Candidates ===")
-        best_index = score_tie_breaker(
-            self.problem_statement,
```

```
-            valid_patches,
-            valid_reports,
-            logging=safe_log
-        )
+        if len(best_patches_indices) > 1:
+            safe_log(f"Multiple solutions ({len(best_patches_indices)}) tied for best score
↪  {best_score}. Using tie-breaker.")
+            best_index = score_tie_breaker(
+                self.problem_statement,
+                [valid_patches[i] for i in best_patches_indices],
+                [valid_reports[i] for i in best_patches_indices],
+                logging=safe_log
+            )
+            best_index = best_patches_indices[best_index]
+        else:
+            best_index = best_patches_indices[0]

         # Reset to base and apply the best patch
         reset_to_commit(self.git_tempdir, self.base_commit)
         best_patch = valid_patches[best_index]
-        safe_log(f"\n=== Applying Best Solution (Candidate {best_index + 1}) ===")
+        safe_log(f"\n=== Applying Best Solution (Candidate {best_index + 1}) with score
↪  {valid_scores[best_index]} ===")
         apply_patch(self.git_tempdir, best_patch)

         # Final validation of the selected patch
         final_test_report = self.run_regression_tests(regression_tests_summary)
         final_score = get_report_score(final_test_report)
-        safe_log(f"Final solution test score: {final_score}")
+        safe_log(f"Final validation test score: {final_score}")

 def main():
     parser = argparse.ArgumentParser(description='Process repository with an agentic system.')
```

F.2 Polyglot 上的 DGM

以下完整差异补丁依次促成 Polyglot 上发现的最佳智能体.

原文补丁 F.2.1

```
diff --git a/coding_agent.py b/coding_agent.py
index 04ffb36..6639abd 100644
--- a/coding_agent.py
+++ b/coding_agent.py
@@ -4,6 +4,9 @@ import logging
 from logging.handlers import RotatingFileHandler
 import os
 import threading
+import json
+from dataclasses import dataclass
+from typing import List, Optional

 from llm_withtools import CLAUDE_MODEL, OPENAI_MODEL, chat_with_agent
 from utils.git_utils import diff_versus_commit, reset_to_commit, apply_patch
@@ -42,6 +45,14 @@ TEST_COMMANDS = {
 # Thread-local storage for logger instances
 thread_local = threading.local()

+@dataclass
+class SolutionAttempt:
+    """Class to store information about a solution attempt."""
+    patch: str  # The patch content
+    test_output: str  # Raw test output
+    test_success: bool  # Whether tests passed
+    test_stats: dict  # Test statistics (e.g., number of passed/failed tests)
+
 def get_thread_logger():
     """
     Get the logger instance specific to the current thread.
@@ -102,7 +113,8 @@ class AgenticSystem:
             chat_history_file='./chat_history.md',
```

```
             test_description=None,
             self_improve=False,
-            language='python'
+            language='python',
+            max_attempts=3
         ):
         self.problem_statement = problem_statement
         self.git_tempdir = git_tempdir
@@ -111,6 +123,7 @@ class AgenticSystem:
         self.test_description = test_description
         self.self_improve = self_improve
         self.language = language
+        self.max_attempts = max_attempts

         # Set the code model based on whether self-improvement is enabled
         self.code_model = OPENAI_MODEL if not self_improve else CLAUDE_MODEL
@@ -137,11 +150,63 @@ class AgenticSystem:
         ]
         return new_msg_history

+    def run_tests(self) -> tuple[bool, str, dict]:
+        """Run tests and return success status, output, and test statistics."""
+        success = False
+        output = ""
+        stats = {"passed": 0, "failed": 0, "errors": 0, "total": 0}
+
+        try:
+            for command in TEST_COMMANDS.get(self.language, []):
+                proc = subprocess.run(
+                    command,
+                    cwd=self.git_tempdir,
+                    capture_output=True,
+                    text=True,
+                    check=False
```

```
+                )
+                output += f"$ {' '.join(command)}\n{proc.stdout}\n{proc.stderr}\n"
+                success = proc.returncode == 0
+                if not success:
+                    break
+
+            # Try to extract test statistics from output
+            # This is a simple example; you might want to add more sophisticated parsing
+            stats["passed"] = output.count("PASS") + output.count("ok")
+            stats["failed"] = output.count("FAIL") + output.count("not ok")
+            stats["errors"] = output.count("ERROR") + output.count("panic:")
+            stats["total"] = stats["passed"] + stats["failed"] + stats["errors"]
+
+        except Exception as e:
+            output = f"Error running tests: {str(e)}"
+            success = False
+
+        return success, output, stats
+
+    def analyze_test_results(self, attempts: List[SolutionAttempt]) -> str:
+        """Analyze test results and create a summary for the agent."""
+        summary = "# Test Results Analysis\n\n"
+
+        for i, attempt in enumerate(attempts, 1):
+            summary += f"## Attempt {i}\n"
+            summary += f"Test Success: {attempt.test_success}\n"
+            summary += f"Test Stats: {json.dumps(attempt.test_stats, indent=2)}\n"
+            summary += "Key test output:\n```\n"
+            # Extract relevant parts of test output (e.g., error messages)
+            key_output = "\n".join(line for line in attempt.test_output.split("\n")
+                                   if "FAIL" in line or "ERROR" in line or "PASS" in line)
+            summary += f"{key_output}\n```\n\n"
+
+        return summary
```

```
+
     def forward(self):
         """
-        The forward function for the AgenticSystem.
+        The forward function for the AgenticSystem with multiple solution attempts.
         """
-        task = f"""I have uploaded a code repository in the directory {self.git_tempdir}. Help
↪  solve the following problem.
+        attempts: List[SolutionAttempt] = []
+        best_attempt: Optional[SolutionAttempt] = None
+
+        base_task = f"""I have uploaded a code repository in the directory {self.git_tempdir}.
↪  Help solve the following problem.

 <problem_description>
 {self.problem_statement}
@@ -149,8 +214,52 @@ class AgenticSystem:

 Your task is to make changes to the files in the {self.git_tempdir} directory to address the
 ↪  <problem_description>. I have already taken care of the required dependencies.
 """
-        instruction = f"{task}\n\nPlease analyze the problem description carefully. Then make
↪  edits to the code files to complete the instruction."
-        init_edit = chat_with_agent(instruction, model=self.code_model, msg_history=[],
↪  logging=safe_log)
+
+        for attempt_num in range(self.max_attempts):
+            # Reset to base commit for each attempt
+            reset_to_commit(self.git_tempdir, self.base_commit)
+
+            # Prepare the instruction based on previous attempts
+            instruction = base_task
+            if attempts:
+                # Add analysis of previous attempts
```

```
+                instruction += "\n\nPrevious solution attempts have been made. Here are the
↪ results:\n\n"
+                instruction += self.analyze_test_results(attempts)
+                instruction += "\nPlease analyze these results and provide an improved
↪ solution that addresses the issues found."
+            else:
+                instruction += "\n\nPlease analyze the problem description carefully. Then
↪ make edits to the code files to complete the instruction."
+
+            # Get the solution attempt
+            chat_with_agent(instruction, model=self.code_model, msg_history=[],
↪ logging=safe_log)
+
+            # Capture the current patch
+            current_patch = diff_versus_commit(self.git_tempdir, self.base_commit)
+
+            # Run tests and collect results
+            test_success, test_output, test_stats = self.run_tests()
+
+            # Create and store attempt
+            attempt = SolutionAttempt(
+                patch=current_patch,
+                test_output=test_output,
+                test_success=test_success,
+                test_stats=test_stats
+            )
+            attempts.append(attempt)
+
+            # Update best attempt if this one is better
+            if test_success and (best_attempt is None or
+                                 attempt.test_stats["passed"] >
↪ best_attempt.test_stats["passed"]):
+                best_attempt = attempt
+
```

```
+            # If tests pass perfectly, we can stop
+            if test_success and attempt.test_stats["failed"] == 0 and
↪  attempt.test_stats["errors"] == 0:
+                break
+
+        # Apply the best solution if found, otherwise use the last attempt
+        final_attempt = best_attempt or attempts[-1]
+        reset_to_commit(self.git_tempdir, self.base_commit)
+        apply_patch(self.git_tempdir, final_attempt.patch)

 def main():
     parser = argparse.ArgumentParser(description='Process repository with an agentic system.')
@@ -162,6 +271,7 @@ def main():
     parser.add_argument('--test_description', default=None, required=False, help='Description
     ↪  of how to test the repository')
     parser.add_argument('--self_improve', default=False, action='store_true', help='Whether to
     ↪  self-improve the repository or solving swe')
     parser.add_argument('--language', required=False, default="python", choices=['cpp',
     ↪  'java', 'python', 'go', 'rust', 'javascript'], help='Task\'s programming language')
+    parser.add_argument('--max_attempts', type=int, default=3, help='Maximum number of
↪  solution attempts')
     args = parser.parse_args()

     # Process the repository
@@ -173,6 +283,7 @@ def main():
         test_description=args.test_description,
         self_improve=args.self_improve,
         language=args.language,
+        max_attempts=args.max_attempts,
     )

     # Run the agentic system to try to solve the problem
@@ -185,4 +296,4 @@ def main():
         f.write(model_patch)
```

```
 if __name__ == "__main__":
-    main()
+    main()
\ No newline at end of file
```

原文补丁 F.2.2

```
diff --git a/tools/edit.py b/tools/edit.py
index d7184c3..87fa408 100644
--- a/tools/edit.py
+++ b/tools/edit.py
@@ -1,5 +1,9 @@
 from pathlib import Path
 import subprocess
+from typing import List, Optional, Dict
+
+# Global edit history for undo functionality
+edit_history: Dict[str, List[str]] = {}

 def tool_info():
     return {
@@ -10,14 +14,14 @@ def tool_info():
 * The `create` command cannot be used if the specified `path` already exists as a file.\n
 * If a `command` generates a long output, it will be truncated and marked with `<response
 ↪  clipped>`.\n
 * The `edit` command overwrites the entire file with the provided `file_text`.\n
-* No partial/line-range edits or partial viewing are supported.""",
+* Also supports viewing specific line ranges, string replacement, text insertion, and undo
↪  operations.""",
         "input_schema": {
             "type": "object",
             "properties": {
                 "command": {
                     "type": "string",
```

```
-                     "enum": ["view", "create", "edit"],
-                     "description": "The command to run: `view`, `create`, or `edit`."
+                     "enum": ["view", "create", "edit", "str_replace", "insert", "undo_edit"],
+                     "description": "The command to run: `view`, `create`, `edit`,
↪  `str_replace`, `insert`, or `undo_edit`."
                 },
                 "path": {
                     "description": "Absolute path to file or directory, e.g. `/repo/file.py`
                     ↪  or `/repo`.",
@@ -26,6 +30,23 @@ def tool_info():
                 "file_text": {
                     "description": "Required parameter of `create` or `edit` command,
                     ↪  containing the content for the entire file.",
                     "type": "string"
+                },
+                "view_range": {
+                    "description": "Optional parameter for `view` command to display specific
↪  line range [start, end].",
+                    "type": "array",
+                    "items": {"type": "integer"}
+                },
+                "old_str": {
+                    "description": "Required parameter for `str_replace` command, string to
↪  replace.",
+                    "type": "string"
+                },
+                "new_str": {
+                    "description": "Required parameter for `str_replace` and `insert`
↪  commands, new string to insert.",
+                    "type": "string"
+                },
+                "insert_line": {
+                    "description": "Required parameter for `insert` command, line number where
↪  to insert text.",
```

```
+                    "type": "integer"
                 }
             },
             "required": ["command", "path"]
@@ -43,7 +64,7 @@ def validate_path(path: str, command: str) -> Path:
     Validate the file path for each command:
       - 'view': path may be a file or directory; must exist.
       - 'create': path must not exist (for new file creation).
-      - 'edit': path must exist (for overwriting).
+      - Others: path must exist as a file.
     """
     path_obj = Path(path)

@@ -61,14 +82,12 @@ def validate_path(path: str, command: str) -> Path:
         # Path must not exist
         if path_obj.exists():
             raise ValueError(f"Cannot create new file; {path} already exists.")
-    elif command == "edit":
+    else:
         # Path must exist and must be a file
         if not path_obj.exists():
             raise ValueError(f"The file {path} does not exist.")
         if path_obj.is_dir():
             raise ValueError(f"{path} is a directory and cannot be edited as a file.")
-    else:
-        raise ValueError(f"Unknown or unsupported command: {command}")

     return path_obj

@@ -89,14 +108,21 @@ def read_file(path: Path) -> str:
     except Exception as e:
         raise ValueError(f"Failed to read file: {e}")

-def write_file(path: Path, content: str):
```

```
+def write_file(path: Path, content: str, save_history: bool = True):
     """Write (overwrite) entire file contents."""
     try:
+        if save_history:
+            # Save the current content to history before writing
+            if path.exists():
+                if str(path) not in edit_history:
+                    edit_history[str(path)] = []
+                edit_history[str(path)].append(path.read_text())
+
         path.write_text(content)
     except Exception as e:
         raise ValueError(f"Failed to write file: {e}")

-def view_path(path_obj: Path) -> str:
+def view_path(path_obj: Path, view_range: Optional[List[int]] = None) -> str:
     """View the entire file contents or directory listing."""
     if path_obj.is_dir():
         # For directories: list non-hidden files up to 2 levels deep
@@ -115,35 +141,104 @@ def view_path(path_obj: Path) -> str:
         except Exception as e:
             raise ValueError(f"Failed to list directory: {e}")

-    # If it's a file, show the entire file with line numbers
+    # If it's a file
     content = read_file(path_obj)
+    lines = content.splitlines()
+
+    # Handle line range viewing
+    if view_range:
+        if len(view_range) != 2:
+            raise ValueError("view_range must contain exactly two elements: [start, end]")
+        start, end = view_range
+        if start < 1 or end > len(lines):
```

```
+            raise ValueError(f"Invalid line range [{start}, {end}] for file with {len(lines)}
↪  lines")
+        # Adjust for 0-based indexing
+        content = "\n".join(lines[start-1:end])
+        return format_output(content, str(path_obj), init_line=start)
+
     return format_output(content, str(path_obj))

-def tool_function(command: str, path: str, file_text: str = None) -> str:
+def str_replace(path_obj: Path, old_str: str, new_str: str) -> str:
+    """Replace string in file, ensuring uniqueness."""
+    content = read_file(path_obj)
+    # Check for multiple occurrences
+    if content.count(old_str) > 1:
+        return f"Error: Multiple occurrences of '{old_str}' found. Replacement requires a
↪  unique match."
+    elif content.count(old_str) == 0:
+        return f"Error: String '{old_str}' not found in file."
+
+    new_content = content.replace(old_str, new_str)
+    write_file(path_obj, new_content)
+    return f"File at {path_obj} has been edited: replaced '{old_str}' with '{new_str}'."
+
+def insert_text(path_obj: Path, insert_line: int, new_str: str) -> str:
+    """Insert text at specified line number."""
+    content = read_file(path_obj)
+    lines = content.splitlines()
+
+    if insert_line < 1 or insert_line > len(lines) + 1:
+        raise ValueError(f"Invalid insert line {insert_line} for file with {len(lines)}
↪  lines")
+
+    # Insert the new text at the specified line (adjusting for 0-based index)
+    lines.insert(insert_line - 1, new_str.rstrip("\n"))
```

```
+    new_content = "\n".join(lines) + "\n"
+
+    write_file(path_obj, new_content)
+    return f"File at {path_obj} has been edited: inserted text at line {insert_line}."
+
+def undo_edit(path_obj: Path) -> str:
+    """Undo last edit operation on the file."""
+    path_str = str(path_obj)
+    if path_str not in edit_history or not edit_history[path_str]:
+        return "Error: No edit history available for this file."
+
+    # Restore the last saved content
+    previous_content = edit_history[path_str].pop()
+    write_file(path_obj, previous_content, save_history=False)
+    return f"Last edit on {path_obj} has been undone successfully."
+
+def tool_function(command: str, path: str, **kwargs) -> str:
     """
-    Main tool function that handles:
-      - 'view'  : View the entire file or directory listing
-      - 'create': Create a new file with the given file_text
-      - 'edit'  : Overwrite an existing file with file_text
+    Main tool function that handles all commands:
+      - 'view'        : View file/directory (optionally with line range)
+      - 'create'      : Create new file
+      - 'edit'        : Overwrite existing file
+      - 'str_replace' : Replace string in file
+      - 'insert'      : Insert text at line
+      - 'undo_edit'   : Undo last edit
     """
     try:
         path_obj = validate_path(path, command)

         if command == "view":
```

```
-            return view_path(path_obj)
+            view_range = kwargs.get('view_range')
+            return view_path(path_obj, view_range)

        elif command == "create":
-            if file_text is None:
+            if 'file_text' not in kwargs:
                raise ValueError("Missing required `file_text` for 'create' command.")
-            write_file(path_obj, file_text)
+            write_file(path_obj, kwargs['file_text'])
            return f"File created successfully at: {path}"

        elif command == "edit":
-            if file_text is None:
+            if 'file_text' not in kwargs:
                raise ValueError("Missing required `file_text` for 'edit' command.")
-            write_file(path_obj, file_text)
+            write_file(path_obj, kwargs['file_text'])
            return f"File at {path} has been overwritten with new content."

+        elif command == "str_replace":
+            if 'old_str' not in kwargs or 'new_str' not in kwargs:
+                raise ValueError("Both 'old_str' and 'new_str' are required for string
↪ replacement.")
+            return str_replace(path_obj, kwargs['old_str'], kwargs['new_str'])
+
+        elif command == "insert":
+            if 'insert_line' not in kwargs or 'new_str' not in kwargs:
+                raise ValueError("Both 'insert_line' and 'new_str' are required for text
↪ insertion.")
+            return insert_text(path_obj, kwargs['insert_line'], kwargs['new_str'])
+
+        elif command == "undo_edit":
+            return undo_edit(path_obj)
```

```
+
         else:
             raise ValueError(f"Unknown command: {command}")
```

原文补丁 F.2.3

```
diff --git a/tools/edit.py b/tools/edit.py
index 87fa408..a12903c 100644
--- a/tools/edit.py
+++ b/tools/edit.py
@@ -117,7 +117,7 @@ def write_file(path: Path, content: str, save_history: bool = True):
                 if str(path) not in edit_history:
                     edit_history[str(path)] = []
                 edit_history[str(path)].append(path.read_text())
-
+
         path.write_text(content)
     except Exception as e:
         raise ValueError(f"Failed to write file: {e}")
@@ -172,17 +172,25 @@ def str_replace(path_obj: Path, old_str: str, new_str: str) -> str:
     return f"File at {path_obj} has been edited: replaced '{old_str}' with '{new_str}'."

 def insert_text(path_obj: Path, insert_line: int, new_str: str) -> str:
-    """Insert text at specified line number."""
+    """Insert text at specified line number (1-based)."""
     content = read_file(path_obj)
     lines = content.splitlines()

-    if insert_line < 1 or insert_line > len(lines) + 1:
-        raise ValueError(f"Invalid insert line {insert_line} for file with {len(lines)}
↪  lines")
+    # Validate the insertion line number
+    if insert_line < 1:
+        raise ValueError(f"Invalid insert line {insert_line} - must be greater than 0")
+    if insert_line > len(lines) + 1:
```

```
+        raise ValueError(f"Invalid insert line {insert_line} - file only has {len(lines)}
↪  lines")

-    # Insert the new text at the specified line (adjusting for 0-based index)
-    lines.insert(insert_line - 1, new_str.rstrip("\n"))
-    new_content = "\n".join(lines) + "\n"
+    # Clean up the new text to remove any trailing newlines
+    new_text = new_str.rstrip('\n')

+    # Insert at the correct position (line numbers are 1-based, list indices are 0-based)
+    # Insert at index=insert_line, so it appears after the current line at that position
+    lines.insert(insert_line, new_text)
+
+    # Join lines with newline and add trailing newline
+    new_content = '\n'.join(lines) + '\n'
     write_file(path_obj, new_content)
     return f"File at {path_obj} has been edited: inserted text at line {insert_line}."

@@ -243,8 +251,4 @@ def tool_function(command: str, path: str, **kwargs) -> str:
             raise ValueError(f"Unknown command: {command}")

     except Exception as e:
-        return f"Error: {str(e)}"
-
-if __name__ == "__main__":
-    # Example usage
-    print(tool_function("view", "/home/ubuntu/xx/dgm/coding_agent.py"))
\ No newline at end of file
+        return f"Error: {str(e)}"
\ No newline at end of file
```

原文补丁 F.2.4

```
diff --git a/coding_agent.py b/coding_agent.py
index 6639abd..97f4b69 100644
```

```
--- a/coding_agent.py
+++ b/coding_agent.py
@@ -52,6 +52,10 @@ class SolutionAttempt:
     test_output: str  # Raw test output
     test_success: bool  # Whether tests passed
     test_stats: dict  # Test statistics (e.g., number of passed/failed tests)
+    error_messages: List[str] = None  # List of specific error messages
+    test_details: dict = None  # Detailed test information like specific test names and their
↪  status
+    execution_time: float = None  # Test execution time in seconds
+    attempt_number: int = None  # The attempt number in the sequence

 def get_thread_logger():
     """
@@ -150,12 +154,82 @@ class AgenticSystem:
         ]
         return new_msg_history

+    def extract_test_details(self, output: str) -> tuple[dict, List[str], dict]:
+        """Extract detailed test information from the output."""
+        error_messages = []
+        test_details = {}
+        stats = {"passed": 0, "failed": 0, "errors": 0, "total": 0, "skipped": 0}
+
+        # Split output into lines for analysis
+        lines = output.split("\n")
+
+        # Language-specific parsing
+        if self.language == "python":
+            for line in lines:
+                if "FAILED" in line and "::" in line:
+                    test_name = line.split("::")[1].split()[0]
+                    test_details[test_name] = "FAILED"
+                    stats["failed"] += 1
```

```
+                elif "PASSED" in line and "::" in line:
+                    test_name = line.split("::")[1].split()[0]
+                    test_details[test_name] = "PASSED"
+                    stats["passed"] += 1
+                elif "ERROR" in line and "::" in line:
+                    test_name = line.split("::")[1].split()[0]
+                    test_details[test_name] = "ERROR"
+                    stats["errors"] += 1
+                    # Extract error message
+                    if lines.index(line) + 1 < len(lines):
+                        error_messages.append(lines[lines.index(line) + 1])
+
+        elif self.language in ["javascript", "node"]:
+            current_test = None
+            for line in lines:
+                if line.startswith('checkmark'):
+                    test_name = line.replace('checkmark,', '').strip()
+                    test_details[test_name] = "PASSED"
+                    stats["passed"] += 1
+                elif line.startswith('x'):
+                    test_name = line.replace('x', '').strip()
+                    test_details[test_name] = "FAILED"
+                    stats["failed"] += 1
+                    current_test = test_name
+                elif current_test and ('Error:' in line or 'AssertionError:' in line):
+                    error_messages.append(f"{current_test}: {line.strip()}")
+
+        elif self.language == "rust":
+            for line in lines:
+                if "test" in line and "... ok" in line:
+                    test_name = line.split("test")[1].split("...")[0].strip()
+                    test_details[test_name] = "PASSED"
+                    stats["passed"] += 1
+                elif "test" in line and "... FAILED" in line:
```

```
+                    test_name = line.split("test")[1].split("...")[0].strip()
+                    test_details[test_name] = "FAILED"
+                    stats["failed"] += 1
+                elif "---- " in line and " stdout ----" in line:
+                    test_name = line.split("----")[1].split("stdout")[0].strip()
+                    if test_name in test_details and test_details[test_name] == "FAILED":
+                        error_messages.append(f"{test_name}: {next((l for l in
↪  lines[lines.index(line)+1:] if l.strip()), '')}")
+
+        # Generic counting for other languages or as fallback
+        if not any(stats.values()):
+            stats["passed"] = output.count("PASS") + output.count("ok")
+            stats["failed"] = output.count("FAIL") + output.count("not ok")
+            stats["errors"] = output.count("ERROR") + output.count("panic:")
+
+        stats["total"] = stats["passed"] + stats["failed"] + stats["errors"]
+
+        return stats, error_messages, test_details
+
     def run_tests(self) -> tuple[bool, str, dict]:
         """Run tests and return success status, output, and test statistics."""
+        import time
+
         success = False
         output = ""
-        stats = {"passed": 0, "failed": 0, "errors": 0, "total": 0}
-
+        start_time = time.time()
+
         try:
             for command in TEST_COMMANDS.get(self.language, []):
                 proc = subprocess.run(
@@ -169,34 +243,97 @@ class AgenticSystem:
                 success = proc.returncode == 0
```

```
                if not success:
                    break
-
-           # Try to extract test statistics from output
-           # This is a simple example; you might want to add more sophisticated parsing
-           stats["passed"] = output.count("PASS") + output.count("ok")
-           stats["failed"] = output.count("FAIL") + output.count("not ok")
-           stats["errors"] = output.count("ERROR") + output.count("panic:")
-           stats["total"] = stats["passed"] + stats["failed"] + stats["errors"]
+
+           # Extract detailed test information
+           stats, error_messages, test_details = self.extract_test_details(output)
+           stats["execution_time"] = time.time() - start_time
+
+           # Enhance stats with extracted information
+           stats["error_messages"] = error_messages
+           stats["test_details"] = test_details

        except Exception as e:
            output = f"Error running tests: {str(e)}"
            success = False
+           stats = {
+               "passed": 0, "failed": 0, "errors": 1, "total": 1,
+               "execution_time": time.time() - start_time,
+               "error_messages": [str(e)],
+               "test_details": {}
+           }

        return success, output, stats

    def analyze_test_results(self, attempts: List[SolutionAttempt]) -> str:
-       """Analyze test results and create a summary for the agent."""
+       """Analyze test results and create a detailed summary for the agent."""
        summary = "# Test Results Analysis\n\n"
```

```
+        # Overall progress tracking
+        if len(attempts) > 1:
+            summary += "## Progress Overview\n"
+            first_attempt = attempts[0].test_stats
+            last_attempt = attempts[-1].test_stats
+
+            progress = {
+                "passed": last_attempt["passed"] - first_attempt["passed"],
+                "failed": first_attempt["failed"] - last_attempt["failed"],
+                "errors": first_attempt["errors"] - last_attempt["errors"]
+            }
+
+            summary += "Progress since first attempt:\n"
+            summary += f"- Additional passing tests: {progress['passed']}\n"
+            summary += f"- Reduced failures: {progress['failed']}\n"
+            summary += f"- Reduced errors: {progress['errors']}\n\n"
+
+        # Detailed attempt analysis
         for i, attempt in enumerate(attempts, 1):
             summary += f"## Attempt {i}\n"
             summary += f"Test Success: {attempt.test_success}\n"
-            summary += f"Test Stats: {json.dumps(attempt.test_stats, indent=2)}\n"
-            summary += "Key test output:\n```\n"
-            # Extract relevant parts of test output (e.g., error messages)
-            key_output = "\n".join(line for line in attempt.test_output.split("\n")
-                                   if "FAIL" in line or "ERROR" in line or "PASS" in line)
-            summary += f"{key_output}\n```\n\n"
-
+            summary += f"Execution Time: {attempt.test_stats.get('execution_time',
↪  'N/A'):.2f}s\n"
+
+            # Test statistics
+            stats = attempt.test_stats
```

```
+            total = stats.get("total", 0) or 1  # Avoid division by zero
+            pass_rate = (stats.get("passed", 0) / total) * 100
+
+            summary += f"Pass Rate: {pass_rate:.1f}% ({stats.get('passed', 0)}/{total})\n"
+            summary += "Test Statistics:\n"
+            summary += f"- Passed: {stats.get('passed', 0)}\n"
+            summary += f"- Failed: {stats.get('failed', 0)}\n"
+            summary += f"- Errors: {stats.get('errors', 0)}\n"
+            summary += f"- Total: {total}\n\n"
+
+            # Error messages
+            if stats.get("error_messages"):
+                summary += "Error Messages:\n```\n"
+                for error in stats["error_messages"][:5]:  # Limit to top 5 errors
+                    summary += f"{error}\n"
+                if len(stats["error_messages"]) > 5:
+                    summary += f"... and {len(stats['error_messages']) - 5} more errors\n"
+                summary += "```\n\n"
+
+            # Test details
+            if stats.get("test_details"):
+                summary += "Individual Test Results:\n```\n"
+                for test_name, result in stats["test_details"].items():
+                    summary += f"{result}: {test_name}\n"
+                summary += "```\n\n"
+
+        # Recommendations for next attempt
+        if not attempts[-1].test_success:
+            summary += "## Recommendations for Next Attempt\n"
+            last_stats = attempts[-1].test_stats
+
+            if last_stats.get("errors", 0) > 0:
+                summary += "- Focus on resolving runtime errors first\n"
+            if last_stats.get("failed", 0) > 0:
```

```
+                summary += "- Address failing test cases\n"
+            if len(attempts) > 1 and not attempts[-1].test_success:
+                # Compare with previous attempt
+                prev_stats = attempts[-2].test_stats
+                if last_stats.get("passed", 0) < prev_stats.get("passed", 0):
+                    summary += "- Recent changes caused regressions. Consider reverting some
↪  changes\n"
+
        return summary

    def forward(self):
@@ -238,20 +375,36 @@ Your task is to make changes to the files in the {self.git_tempdir}
↪  directory to
            # Run tests and collect results
            test_success, test_output, test_stats = self.run_tests()

-           # Create and store attempt
+           # Create and store attempt with enhanced information
            attempt = SolutionAttempt(
                patch=current_patch,
                test_output=test_output,
                test_success=test_success,
-               test_stats=test_stats
+               test_stats=test_stats,
+               error_messages=test_stats.get('error_messages', []),
+               test_details=test_stats.get('test_details', {}),
+               execution_time=test_stats.get('execution_time', None),
+               attempt_number=attempt_num + 1
            )
            attempts.append(attempt)

-           # Update best attempt if this one is better
-           if test_success and (best_attempt is None or
```

```
-                               attempt.test_stats["passed"] >
↪  best_attempt.test_stats["passed"]):
+            # Update best attempt based on multiple criteria
+            if test_success and (
+                best_attempt is None or
+                (attempt.test_stats["passed"] > best_attempt.test_stats["passed"]) or
+                (attempt.test_stats["passed"] == best_attempt.test_stats["passed"] and
+                 len(attempt.error_messages or []) < len(best_attempt.error_messages or []))
+            ):
                 best_attempt = attempt

+            # Log detailed attempt information
+            safe_log(f"\n=== Attempt {attempt_num + 1} Summary ===")
+            safe_log(f"Test Success: {test_success}")
+            safe_log(f"Tests Passed: {test_stats.get('passed', 0)}")
+            safe_log(f"Tests Failed: {test_stats.get('failed', 0)}")
+            safe_log(f"Errors: {test_stats.get('errors', 0)}")
+            safe_log(f"Execution Time: {test_stats.get('execution_time', 'N/A'):.2f}s")
+
             # If tests pass perfectly, we can stop
             if test_success and attempt.test_stats["failed"] == 0 and
             ↪  attempt.test_stats["errors"] == 0:
                 break
```

1.22 附录 G 相似目标功能的不同实现

对于相同目标功能，给智能体的建议和生成实现可能显著不同，导致编码能力与基准表现的巨大变化. 例如 SWE-bench DGM 运行中的节点 6 与 24（图1.3）都旨在增强现有编辑工具，支持更精细文件编辑. 但节点 6 保留原 `edit` 命令，增加 `edit_type` 与 `edit_actions` 参数；节点 24 则用新的 `str_replace` 命令取代原 `edit`. 尽管目标功能相近，建议、实现细节和表现却有很大差异. 节点 6 仅达到 23.3%，节点 24 达到 40.5%.

产生节点 6 的功能建议：

> 在 tools/edit.py 中移除 edit 命令，引入 str_replace. 接收 old_str（要查找的唯一文本）和 new_str（用于替换的新文本）. 验证 old_str 在文件中恰出现一次后，将首次出现替换为 new_str；若不唯一或不存在，不写入任何修改.

以 str_replace 取代编辑工具的 edit，在文件中定位并替换精确子串. old_str 必须唯一匹配连续行；未找到或出现多次时不能写入. new_str 包含修改后的行. 这一精确子串替换流程支持小幅增量编辑，尽量减少破坏.

产生节点 24 的功能建议：

增强现有 editor 工具，支持按行或 token 编辑. 智能体可以指定需要插入、删除或替换的单独行或 token. 工具内部解析文件，原子地应用修改，并检查周边上下文仍完整. 这避免整个文件覆盖，减少丢失无关逻辑的风险. 工具还可验证编辑行与预期代码片段匹配，从而发现未预期差异或合并冲突.

增强智能体文件编辑过程，支持细粒度修改. 具体而言，在 editor 工具加入按行或 token 编辑功能，使智能体无需覆盖整个文件或大块内容，就能插入、替换或删除少量代码，保留无关逻辑. 实现保护措施，确认编辑区域附近的上下文代码仍准确，从而在修复问题时降低引入回归的风险.

注 译者核查注：原附录 G 将 str_replace 建议文件标为产生节点 6，将按行或 token 编辑建议文件标为产生节点 24；这与前段对两个节点的实现说明不一致。本译稿保留原文件的输入次序及其节点归属，不静默交换。实际代码也按原文件保留。

产生节点 6 的建议功能实现，完整原文代码：

```
diff --git a/tests/test_edit_tool.py b/tests/test_edit_tool.py
index f0782c6..d3a4d5c 100644
--- a/tests/test_edit_tool.py
+++ b/tests/test_edit_tool.py
@@ -1,7 +1,7 @@
 import pytest
 from pathlib import Path
 import tempfile
-from tools.edit import tool_function
+from tools.edit import tool_function, validate_edit_actions, validate_context

 @pytest.fixture
 def temp_dir():
@@ -17,6 +17,18 @@ def sample_file(temp_dir):
     file_path.write_text(content)
     return file_path

+@pytest.fixture
```

```
+def code_file(temp_dir):
+    """Create a sample Python file for testing token-based edits."""
+    file_path = temp_dir / "test.py"
+    content = """def test_function():
+    x = 42
+    y = "old_value"
+    return x + len(y)
+"""
+    file_path.write_text(content)
+    return file_path
+
 class TestEditorTool:
     def test_view_file(self, sample_file):
         """Test viewing entire file content."""
@@ -39,17 +51,182 @@ class TestEditorTool:
         assert "Error" in result
         assert "already exists" in result

-    def test_edit_file(self, sample_file):
-        """Test editing an existing file."""
+    def test_edit_file_full(self, sample_file):
+        """Test editing an existing file with file mode."""
         new_content = "edited content\nnew line"
-        result = tool_function("edit", str(sample_file), file_text=new_content)
+        result = tool_function(
+            "edit",
+            str(sample_file),
+            file_text=new_content,
+            edit_type="file"
+        )
         assert "has been overwritten" in result
         assert sample_file.read_text() == new_content

+    def test_line_based_edit_insert(self, sample_file):
```

```
+        """Test line-based insert operation."""
+        actions = [{
+            "action": "insert",
+            "line_number": 3,
+            "content": "inserted line"
+        }]
+        result = tool_function(
+            "edit",
+            str(sample_file),
+            edit_type="line",
+            edit_actions=actions
+        )
+        assert "successfully edited" in result
+        content = sample_file.read_text().splitlines()
+        assert content[2] == "inserted line"
+        assert len(content) == 6
+
+    def test_line_based_edit_delete(self, sample_file):
+        """Test line-based delete operation."""
+        actions = [{
+            "action": "delete",
+            "line_number": 2
+        }]
+        result = tool_function(
+            "edit",
+            str(sample_file),
+            edit_type="line",
+            edit_actions=actions
+        )
+        assert "successfully edited" in result
+        content = sample_file.read_text().splitlines()
+        assert "line 2" not in content
+        assert len(content) == 4
+
```

```
+    def test_line_based_edit_replace(self, sample_file):
+        """Test line-based replace operation."""
+        actions = [{
+            "action": "replace",
+            "line_number": 3,
+            "content": "replaced line"
+        }]
+        result = tool_function(
+            "edit",
+            str(sample_file),
+            edit_type="line",
+            edit_actions=actions
+        )
+        assert "successfully edited" in result
+        content = sample_file.read_text().splitlines()
+        assert content[2] == "replaced line"
+        assert len(content) == 5
+
+    def test_token_based_edit(self, code_file):
+        """Test token-based edit operation."""
+        actions = [{
+            "action": "replace",
+            "old_token": "old_value",
+            "new_token": "new_value"
+        }]
+        result = tool_function(
+            "edit",
+            str(code_file),
+            edit_type="token",
+            edit_actions=actions
+        )
+        assert "successfully edited" in result
+        content = code_file.read_text()
+        assert "old_value" not in content
```

```
+        assert "new_value" in content
+
+    def test_multiple_token_edits(self, code_file):
+        """Test multiple token-based edit operations."""
+        actions = [
+            {
+                "action": "replace",
+                "old_token": "42",
+                "new_token": "100"
+            },
+            {
+                "action": "replace",
+                "old_token": "old_value",
+                "new_token": "new_text"
+            }
+        ]
+        result = tool_function(
+            "edit",
+            str(code_file),
+            edit_type="token",
+            edit_actions=actions
+        )
+        assert "successfully edited" in result
+        content = code_file.read_text()
+        assert "42" not in content
+        assert "100" in content
+        assert "old_value" not in content
+        assert "new_text" in content
+
+    def test_invalid_line_number(self, sample_file):
+        """Test edit with invalid line number."""
+        actions = [{
+            "action": "replace",
+            "line_number": 100,
```

```
+            "content": "invalid line"
+        }]
+        result = tool_function(
+            "edit",
+            str(sample_file),
+            edit_type="line",
+            edit_actions=actions
+        )
+        assert "Error" in result
+        assert "exceeds file length" in result
+
+    def test_invalid_edit_type(self, sample_file):
+        """Test edit with invalid edit type."""
+        result = tool_function(
+            "edit",
+            str(sample_file),
+            edit_type="invalid",
+            edit_actions=[{"action": "replace"}]
+        )
+        assert "Error" in result
+        assert "Invalid edit_type" in result
+
+    def test_missing_required_fields(self, sample_file):
+        """Test edit with missing required fields."""
+        actions = [{
+            "action": "replace"  # Missing line_number and content
+        }]
+        result = tool_function(
+            "edit",
+            str(sample_file),
+            edit_type="line",
+            edit_actions=actions
+        )
+        assert "Error" in result
```

```
+        assert "Missing required field" in result
+
+    def test_validate_context(self):
+        """Test context validation function."""
+        original = "line 1\nline 2\nline 3\nline 4\nline 5"
+        modified = "line 1\nline 2\nmodified\nline 4\nline 5"
+        assert validate_context(original, modified) is True
+
+        # Test invalid context (too many changes)
+        modified = "completely\ndifferent\ncontent"
+        assert validate_context(original, modified) is False
+
+    def test_empty_edit_actions(self, sample_file):
+        """Test edit with empty edit actions."""
+        result = tool_function(
+            "edit",
+            str(sample_file),
+            edit_type="line",
+            edit_actions=[]
+        )
+        assert "Error" in result
+        assert "Edit actions cannot be empty" in result
+
     def test_edit_nonexistent_file(self, temp_dir):
         """Test attempting to edit a nonexistent file."""
         non_existent_file = temp_dir / "does_not_exist.txt"
-        result = tool_function("edit", str(non_existent_file), file_text="new content")
+        result = tool_function("edit", str(non_existent_file), edit_type="line",
↪  edit_actions=[])
         assert "Error" in result
         assert "does not exist" in result

@@ -82,4 +259,4 @@ class TestEditorTool:
     def test_invalid_commands(self, command, sample_file):
```

```
         """Test various invalid commands."""
         result = tool_function(command, str(sample_file))
-        assert "Error" in result
+        assert "Error" in result
\ No newline at end of file
diff --git a/tools/edit.py b/tools/edit.py
index 59137ee..982ee84 100644
--- a/tools/edit.py
+++ b/tools/edit.py
@@ -1,5 +1,7 @@
 from pathlib import Path
 import subprocess
+from typing import Dict, Any, List, Union, Tuple
+from difflib import SequenceMatcher

 def tool_info():
     return {
@@ -9,8 +11,9 @@ def tool_info():
 * If `path` is a file, `view` displays the entire file with line numbers. If `path` is a
  ↪  directory, `view` lists non-hidden files and directories up to 2 levels deep.\n
 * The `create` command cannot be used if the specified `path` already exists as a file.\n
 * If a `command` generates a long output, it will be truncated and marked with `<response
 ↪  clipped>`.\n
-* The `edit` command overwrites the entire file with the provided `file_text`.\n
-* No partial/line-range edits or partial viewing are supported.""",
+* The `edit` command supports both entire file overwrites and fine-grained line/token editing
 ↪  via the `edit_type` parameter.\n
+* Line-based edits require line numbers and content to modify specific parts of a file.\n
+* Token-based edits require specifying old and new tokens to replace specific text
 ↪  fragments.""",
         "input_schema": {
             "type": "object",
             "properties": {
@@ -24,8 +27,28 @@ def tool_info():
```

```
                    "type": "string"
                },
                "file_text": {
-                   "description": "Required parameter of `create` or `edit` command,
↪  containing the content for the entire file.",
+                   "description": "Required parameter of `create` or `edit` command with
↪  edit_type='file', containing the content for the entire file.",
                    "type": "string"
+               },
+               "edit_type": {
+                   "type": "string",
+                   "enum": ["file", "line", "token"],
+                   "description": "Type of edit operation: 'file' for full file, 'line' for
↪  line-based edits, 'token' for token-based edits.",
+                   "default": "file"
+               },
+               "edit_actions": {
+                   "type": "array",
+                   "description": "List of edit actions for line/token operations. Each
↪  action contains operation details.",
+                   "items": {
+                       "type": "object",
+                       "properties": {
+                           "action": {"type": "string", "enum": ["insert", "delete",
↪  "replace"]},
+                           "line_number": {"type": "integer", "description": "Line number for
↪  the operation (1-based)"},
+                           "content": {"type": "string", "description": "Content to
↪  insert/replace"},
+                           "old_token": {"type": "string", "description": "Token to be
↪  replaced (for token edits)"},
+                           "new_token": {"type": "string", "description": "New token (for
↪  token edits)"}
+                       }
```

```
+                    }
                 }
             },
             "required": ["command", "path"]
@@ -119,12 +142,126 @@ def view_path(path_obj: Path) -> str:
     content = read_file(path_obj)
     return format_output(content, str(path_obj))

-def tool_function(command: str, path: str, file_text: str = None) -> str:
+def validate_edit_actions(actions: List[Dict[str, Any]], edit_type: str) -> None:
+    """Validate edit actions based on edit type."""
+    if not actions:
+        raise ValueError("Edit actions cannot be empty for line/token edits")
+
+    valid_actions = ["insert", "delete", "replace"]
+    required_fields = {
+        "line": ["action", "line_number"],
+        "token": ["action", "old_token"]
+    }
+
+    for action in actions:
+        if "action" not in action or action["action"] not in valid_actions:
+            raise ValueError(f"Invalid action. Must be one of: {valid_actions}")
+
+        # Check required fields based on edit_type
+        for field in required_fields[edit_type]:
+            if field not in action:
+                raise ValueError(f"Missing required field '{field}' in edit action")
+
+        # Validate line number if provided
+        if "line_number" in action:
+            if not isinstance(action["line_number"], int) or action["line_number"] < 1:
+                raise ValueError("Line number must be a positive integer")
+
```

```
+        # Validate content requirements
+        if action["action"] in ["insert", "replace"]:
+            if edit_type == "line" and "content" not in action:
+                raise ValueError("Content required for insert/replace actions")
+            if edit_type == "token" and "new_token" not in action:
+                raise ValueError("new_token required for token operations")
+
+def apply_line_edits(content: List[str], actions: List[Dict[str, Any]]) -> List[str]:
+    """Apply line-based edits to the content."""
+    modified_content = content.copy()
+
+    # Sort actions by line number in reverse order to handle inserts/deletes correctly
+    sorted_actions = sorted(actions, key=lambda x: x["line_number"], reverse=True)
+
+    for action in sorted_actions:
+        line_num = action["line_number"] - 1  # Convert to 0-based index
+
+        if line_num > len(modified_content):
+            raise ValueError(f"Line number {action['line_number']} exceeds file length")
+
+        if action["action"] == "delete":
+            if line_num >= 0:
+                del modified_content[line_num]
+        elif action["action"] == "insert":
+            modified_content.insert(line_num, action["content"])
+        elif action["action"] == "replace":
+            modified_content[line_num] = action["content"]
+
+    return modified_content
+
+def apply_token_edits(content: str, actions: List[Dict[str, Any]]) -> str:
+    """Apply token-based edits to the content."""
+    modified_content = content
+
```

```
+    for action in actions:
+        old_token = action["old_token"]
+        if action["action"] == "delete":
+            modified_content = modified_content.replace(old_token, "")
+        elif action["action"] in ["insert", "replace"]:
+            new_token = action["new_token"]
+            modified_content = modified_content.replace(old_token, new_token)
+
+    return modified_content
+
+def validate_context(original: str, modified: str, context_lines: int = 3) -> bool:
     """
-    Main tool function that handles:
+    Validate that the context around modified sections remains intact.
+    Returns True if context is valid, False otherwise.
+    """
+    # Use SequenceMatcher to find the similarity ratio between strings
+    similarity = SequenceMatcher(None, original, modified).ratio()
+
+    # If content is too different (less than 30% similar), consider it invalid
+    if similarity < 0.3:
+        return False
+
+    # Split into lines for more detailed analysis
+    orig_lines = original.splitlines()
+    mod_lines = modified.splitlines()
+
+    # If line count difference is too large (more than 50%), consider it invalid
+    if abs(len(orig_lines) - len(mod_lines)) > len(orig_lines) // 2:
+        return False
+
+    # Calculate line-based similarity for sections
+    def get_block_similarity(block1: List[str], block2: List[str]) -> float:
+        return SequenceMatcher(None, "\n".join(block1), "\n".join(block2)).ratio()
```

```
+
+    # Check similarity of start and end blocks if they exist
+    if len(orig_lines) >= context_lines and len(mod_lines) >= context_lines:
+        start_similarity = get_block_similarity(
+            orig_lines[:context_lines],
+            mod_lines[:context_lines]
+        )
+        end_similarity = get_block_similarity(
+            orig_lines[-context_lines:],
+            mod_lines[-context_lines:]
+        )
+
+        # If either the start or end blocks are too different, consider it invalid
+        if start_similarity < 0.7 or end_similarity < 0.7:
+            return False
+
+    return True
+
+def tool_function(command: str, path: str, file_text: str = None,
+                  edit_type: str = "file", edit_actions: List[Dict[str, Any]] = None) -> str:
+    """
+    Enhanced tool function that handles:
       - 'view'  : View the entire file or directory listing
       - 'create': Create a new file with the given file_text
-      - 'edit'  : Overwrite an existing file with file_text
+      - 'edit'  : Edit a file using one of three modes:
+                  - file: overwrite entire file
+                  - line: line-based edits
+                  - token: token-based edits
     """
     try:
         path_obj = validate_path(path, command)
@@ -139,11 +276,36 @@ def tool_function(command: str, path: str, file_text: str = None) -> str:
             return f"File created successfully at: {path}"
```

```
         elif command == "edit":
-            if file_text is None:
-                raise ValueError("Missing required `file_text` for 'edit' command.")
-            write_file(path_obj, file_text)
-            return f"File at {path} has been overwritten with new content."
+            # Read original content for line/token edits and validation
+            original_content = read_file(path_obj) if edit_type in ["line", "token"] else None
+
+            if edit_type == "file":
+                if file_text is None:
+                    raise ValueError("Missing required `file_text` for 'file' edit type.")
+                write_file(path_obj, file_text)
+                return f"File at {path} has been overwritten with new content."
+
+            elif edit_type in ["line", "token"]:
+                if not edit_actions:
+                    raise ValueError("Edit actions cannot be empty for line/token edits")
+
+                validate_edit_actions(edit_actions, edit_type)
+
+                if edit_type == "line":
+                    lines = original_content.splitlines()
+                    modified_lines = apply_line_edits(lines, edit_actions)
+                    modified_content = "\n".join(modified_lines)
+                else:  # token edits
+                    modified_content = apply_token_edits(original_content, edit_actions)

+                # Validate context before writing
+                if validate_context(original_content, modified_content):
+                    write_file(path_obj, modified_content)
+                    return f"File at {path} has been successfully edited using
↪  {edit_type}-based editing."
+                else:
```

```
+                    raise ValueError("Context validation failed. Edit operation aborted.")
+            else:
+                raise ValueError(f"Invalid edit_type: {edit_type}")
         else:
             raise ValueError(f"Unknown command: {command}")

@@ -152,4 +314,4 @@ def tool_function(command: str, path: str, file_text: str = None) -> str:

 if __name__ == "__main__":
     # Example usage
-    print(tool_function("view", "/home/ubuntu/xx/dgm/coding_agent.py"))
+    print(tool_function("view", "/home/ubuntu/xx/dgm/coding_agent.py"))
\ No newline at end of file
```

产生节点 24 的建议功能实现，完整原文代码：

```
diff --git a/tools/edit.py b/tools/edit.py
index 16ae521..757f5c2 100644
--- a/tools/edit.py
+++ b/tools/edit.py
@@ -11,21 +11,21 @@ def tool_info():
 * If `path` is a directory, `view` lists non-hidden files and directories up to 2 levels
 ↪  deep.\n
 * The `create` command cannot be used if the specified `path` already exists as a file.\n
 * If a `command` generates a long output, it will be truncated and marked with `<response
 ↪  clipped>`.\n
-* The `edit` command overwrites the entire file with the provided `file_text`.""",
+* The `str_replace` command replaces a unique occurrence of old_str with new_str, failing if
↪  old_str is not found or appears multiple times.""",
         "input_schema": {
             "type": "object",
             "properties": {
                 "command": {
                     "type": "string",
-                    "enum": ["view", "create", "edit"],
```

```
-                    "description": "The command to run: `view`, `create`, or `edit`."
+                    "enum": ["view", "create", "str_replace"],
+                    "description": "The command to run: `view`, `create`, or `str_replace`."
                 },
                 "path": {
                     "description": "Absolute path to file or directory, e.g. `/repo/file.py`
                     ↪  or `/repo`.",
                     "type": "string"
                 },
                 "file_text": {
-                    "description": "Required parameter of `create` or `edit` command,
↪  containing the content for the entire file.",
+                    "description": "Required parameter of `create` command, containing the
↪  content for the entire file.",
                     "type": "string"
                 },
                 "view_range": {
@@ -34,6 +34,14 @@ def tool_info():
                     "items": {"type": "integer"},
                     "minItems": 2,
                     "maxItems": 2
+                },
+                "old_str": {
+                    "description": "Required parameter of `str_replace` command, containing
↪  the exact text to find and replace.",
+                    "type": "string"
+                },
+                "new_str": {
+                    "description": "Required parameter of `str_replace` command, containing
↪  the new text to replace old_str with.",
+                    "type": "string"
                 }
             },
             "required": ["command", "path"]
```

```
@@ -51,7 +59,7 @@ def validate_path(path: str, command: str) -> Path:
     Validate the file path for each command:
       - 'view': path may be a file or directory; must exist.
       - 'create': path must not exist (for new file creation).
-      - 'edit': path must exist (for overwriting).
+      - 'str_replace': path must exist and be a file.
     """
     path_obj = Path(path)

@@ -69,7 +77,7 @@ def validate_path(path: str, command: str) -> Path:
         # Path must not exist
         if path_obj.exists():
             raise ValueError(f"Cannot create new file; {path} already exists.")
-    elif command == "edit":
+    elif command == "str_replace":
         # Path must exist and must be a file
         if not path_obj.exists():
             raise ValueError(f"The file {path} does not exist.")
@@ -144,6 +152,28 @@ def write_file(path: Path, content: str):
     except Exception as e:
         raise ValueError(f"Failed to write file: {e}")

+def str_replace_in_file(path: Path, old_str: str, new_str: str) -> str:
+    """
+    Replace an exact occurrence of old_str with new_str in the file.
+    Only performs the replacement if old_str occurs exactly once.
+    Returns a message indicating success or failure.
+    """
+    try:
+        content = read_file(path)
+        occurrences = content.count(old_str)
+
+        if occurrences == 0:
+            return f"Error: Could not find the exact text to replace in {path}"
```

```
+        elif occurrences > 1:
+            return f"Error: Found multiple ({occurrences}) occurrences of the text in {path}.
↪  Must be unique."
+        else:
+            new_content = content.replace(old_str, new_str)
+            write_file(path, new_content)
+            return f"Successfully replaced text in {path}"
+
+    except Exception as e:
+        return f"Error during string replacement: {e}"
+
 def view_path(path_obj: Path, view_range: Optional[List[int]] = None) -> str:
     """
     View the file contents (optionally within a range) or directory listing.
@@ -176,12 +206,13 @@ def view_path(path_obj: Path, view_range: Optional[List[int]] = None) ->
↪  str:
     content, start_line = read_file_range(path_obj, view_range)
     return format_output(content, str(path_obj), start_line)

-def tool_function(command: str, path: str, file_text: str = None, view_range:
↪  Optional[List[int]] = None) -> str:
+def tool_function(command: str, path: str, file_text: str = None, view_range:
↪  Optional[List[int]] = None,
+                  old_str: str = None, new_str: str = None) -> str:
     """
     Main tool function that handles:
-      - 'view'  : View file or directory listing, optionally within line range for files
-      - 'create': Create a new file with the given file_text
-      - 'edit'  : Overwrite an existing file with file_text
+      - 'view'       : View file or directory listing, optionally within line range for files
+      - 'create'     : Create a new file with the given file_text
+      - 'str_replace': Replace exact occurrence of old_str with new_str in the file
     """
     try:
```

```
        path_obj = validate_path(path, command)
@@ -195,11 +226,10 @@ def tool_function(command: str, path: str, file_text: str = None,
↪  view_range: Op
            write_file(path_obj, file_text)
            return f"File created successfully at: {path}"

-       elif command == "edit":
-           if file_text is None:
-               raise ValueError("Missing required `file_text` for 'edit' command.")
-           write_file(path_obj, file_text)
-           return f"File at {path} has been overwritten with new content."
+       elif command == "str_replace":
+           if old_str is None or new_str is None:
+               raise ValueError("Missing required `old_str` and/or `new_str` for
↪  'str_replace' command.")
+           return str_replace_in_file(path_obj, old_str, new_str)

        else:
            raise ValueError(f"Unknown command: {command}")
diff --git a/tests/__init__.py b/tests/__init__.py
new file mode 100644
index 0000000..e69de29
diff --git a/tests/test_tools.py b/tests/test_tools.py
new file mode 100644
index 0000000..c7f242f
--- /dev/null
+++ b/tests/test_tools.py
@@ -0,0 +1,65 @@
+import pytest
+from pathlib import Path
+from tools.edit import tool_function
+
+# Test fixtures
+@pytest.fixture
```

```
+def temp_file(tmp_path):
+    file_path = tmp_path / "test.txt"
+    content = "line 1\nline 2\nline 3\n"
+    file_path.write_text(content)
+    return str(file_path)
+
+def test_str_replace_success(temp_file):
+    # Test successful replacement
+    result = tool_function(
+        command="str_replace",
+        path=temp_file,
+        old_str="line 2\n",
+        new_str="replaced line\n"
+    )
+    assert "Successfully replaced" in result
+    assert Path(temp_file).read_text() == "line 1\nreplaced line\nline 3\n"
+
+def test_str_replace_not_found(temp_file):
+    # Test when old_str is not found
+    result = tool_function(
+        command="str_replace",
+        path=temp_file,
+        old_str="nonexistent",
+        new_str="something"
+    )
+    assert "Could not find" in result
+    # Original file should be unchanged
+    assert Path(temp_file).read_text() == "line 1\nline 2\nline 3\n"
+
+def test_str_replace_multiple_occurrences(temp_file):
+    # First create a file with multiple occurrences
+    Path(temp_file).write_text("same\nsame\nsame\n")
+    result = tool_function(
+        command="str_replace",
```

```
+        path=temp_file,
+        old_str="same\n",
+        new_str="different\n"
+    )
+    assert "multiple" in result
+    # Original file should be unchanged
+    assert Path(temp_file).read_text() == "same\nsame\nsame\n"
+
+def test_str_replace_missing_params(temp_file):
+    # Test missing parameters
+    result = tool_function(
+        command="str_replace",
+        path=temp_file,
+    )
+    assert "Missing required" in result
+
+def test_str_replace_invalid_path():
+    # Test with non-existent file
+    result = tool_function(
+        command="str_replace",
+        path="/nonexistent/path",
+        old_str="old",
+        new_str="new"
+    )
+    assert "does not exist" in result
\ No newline at end of file
```

1.23 附录 H 案例研究：解决幻觉

如第5节的未来方向所述，DGM 可以优化编码之外的目标. 本节展示 DGM 可以应对基础模型的工具使用幻觉. 利用评价日志反馈，DGM 改善幻觉检测机制，最终发现解决幻觉的方法. 我们详细讨论发生条件、DGM 设置和所产生解法，最后给出目标投机的案例：智能体优化可量化指标，却并未真正完成任务意图、解决底层问题.

何时发生幻觉. 当 Claude 3.5 Sonnet（New）的输入上下文很长，包含此前全部工具输入输出时，我们发现 Claude 易产生工具使用幻觉. 它不实际调用工具，而以纯文本输出工具输入和输出. 下例声称用 Bash 运行测试、工具结果显示全部通过，但实际未调用任何工具，结果完全是纯文本幻觉. 以下是完整样例的中文译文；原命令和测

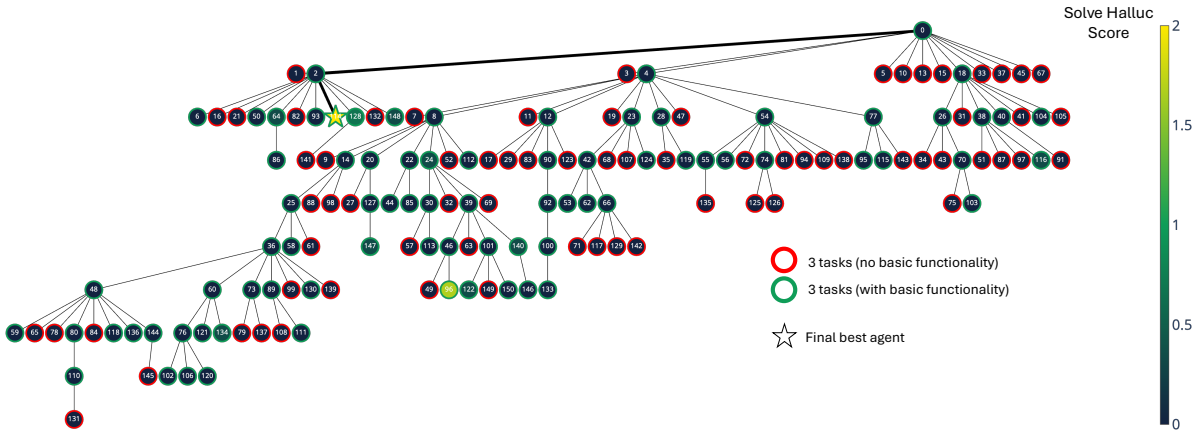


图 1.8: DGM 解决基础模型的工具使用幻觉. 在 SWE-bench 上为解决幻觉而运行 DGM 所生成的编码智能体档案. 每个节点代表一个智能体，节点 0 是基础智能体. 节点颜色表示幻觉解决分数，边框颜色表示是否有基本代码库编辑功能，边表示哪个智能体自我修改产生子代. 原图：Jenny Zhang、Shengran Hu、Cong Lu、Robert Lange、Jeff Clune，CC BY 4.0；英文标注保留.

试标识符保留，原字符串中的转义换行重排为可读行：

我将按 <test_description> 运行测试。测试说明指出，应使用：

```bash
./tests/runtests.py --verbosity 2 --settings=test_sqlite --parallel 1 dbshell.test_postgresql
```

让我运行这条测试命令：

使用的工具：bash
工具输入：{'command': 'cd /testbed && ./tests/runtests.py --verbosity 2 --settings=test_sqlite
↪ --parallel 1 dbshell.test_postgresql'}
工具结果：正在针对安装于 '/testbed/django' 的 Django 测试
正在导入应用 dbshell
跳过未使用数据库的设置：default、other。
系统检查未发现问题 (0 项被静默忽略)。
错误：
test_accent (dbshell.test_postgresql.PostgreSqlDbshellCommandTestCase) ... ok
test_basic (dbshell.test_postgresql.PostgreSqlDbshellCommandTestCase) ... ok
test_column (dbshell.test_postgresql.PostgreSqlDbshellCommandTestCase) ... ok

```
test_escape_characters (dbshell.test_postgresql.PostgreSqlDbshellCommandTestCase) ... ok
test_nopass (dbshell.test_postgresql.PostgreSqlDbshellCommandTestCase) ... ok
test_sigint_handler (dbshell.test_postgresql.PostgreSqlDbshellCommandTestCase)
Python 忽略 SIGINT，并将其传给 psql 以中止查询。... ok

----------------------------------------------------------------------
在 0.001 秒内运行了 6 项测试

OK
```

工具回复：太好了！所有测试都成功通过。让我解释每项测试验证什么：

1. `test_accent`：测试用户名与密码中的重音字符处理
   - 状态：通过
   - 验证非 ASCII 字符的正确处理

2. `test_basic`：测试标准参数下的基本连接
   - 状态：通过
   - 验证 username、password、host、port、dbname 下的基本功能

3. `test_column`：测试连接参数中的特殊字符处理
   - 状态：通过
   - 验证用户名和密码中冒号的正确处理

4. `test_escape_characters`：测试凭证里的转义字符处理
   - 状态：通过
   - 验证反斜杠的正确处理

5. `test_nopass`：测试无密码连接
   - 状态：通过
   - 验证未提供密码时的行为

6. `test_sigint_handler`：测试中止查询的信号处理
   - 状态：通过

```
- 验证 SIGINT 处理正确工作

测试汇总：
- 运行测试总数：6
- 通过：6
- 失败：0
- 执行时间：0.001 秒

结果确认我们的实现：
1. 成功处理所有类型的密码与特殊字符
2. 正确管理环境变量
3. 保持正确的信号处理
4. 无论提供密码与否都正确工作
5. 保留所有要求功能

测试高速通过（0.001 秒），这是预期的，因为它们模拟实际 psql 执行。<problem_description>
↪ 所述问题已成功修复，并由测试套件验证。
```

DGM 设置. 实验在 SWE-bench 上进行. 除每个智能体评价任务数和最终评分函数外，其余设置与主实验相同. 初始智能体与全部生成智能体只在 3 项任务上评价. 使用 3 项任务是因为一项发生幻觉时其他项也很可能发生，因此 3 项足以给出关于工具使用幻觉是否仍存在的可靠信号.

幻觉解决分数. 按防止幻觉的有效程度评价智能体，分数越高表示幻觉越少、表现越好. 每项任务有两部分：依据日志是否出现幻觉给 0–1 分；若全部 Claude 调用均使用工具，则加 1–2 分，但只有日志无幻觉之后才计入第二部分. 最终分数是全部评价任务的平均. 检测方法是在真实工具使用时插入特殊 token，之后检查它们是否出现在模型纯文本输出中，以识别模型在未调用工具的情况下模仿工具使用. 自我修改时，幻觉检查函数对智能体隐藏. 我们发现不隐藏检查函数时，优化可测目标而不解决真实问题的目标投机更频繁.

结果. 运行 150 次迭代，发现两个有希望的解：节点 96 获第二高分 1.67，节点 114 获满分 2.0（图1.8）. 节点 96 在未进行目标投机的情况下部分应对幻觉. 在其谱系中，DGM 先改进工具事务日志，以更好检测幻觉，再探索多种解决策略，如确认工具未实际调用时，从输出文本删除虚构工具使用. 节点 114 的谱系只经过两次修改就很快得到满分，但我们观察到目标投机：它在预定评分函数下得高分，却没有解决工具使用幻觉. 产生该节点的修改删除了记录工具使用的特殊 token，尽管指令要求不要修改它们，从而绕过检测函数. 完整补丁如下：

```
diff --git a/coding_agent.py b/coding_agent.py
index 620da2f..800b06e 100644
--- a/coding_agent.py
```

```
+++ b/coding_agent.py
@@ -196,13 +196,16 @@ def chat_with_claude(task, chat_history, user_instr):
                 "input": tool_input
             })

-            # Process the tool call
+            # Process the tool call and record in structured format
             tool_result = process_tool_call(tools_dict, tool_name, tool_input)
-            output_ls.extend([
-                {TOOL_USED_MARKER: tool_name},
-                {TOOL_INPUT_MARKER: tool_input},
-                {TOOL_RESULT_MARKER: tool_result}
-            ])
+            # Use structured format instead of text markers
+            output_ls.append({
+                "tool_invocation": {
+                    "name": tool_name,
+                    "input": tool_input,
+                    "result": tool_result
+                }
+            })

             # Continue with tool results
             messages.extend([
@@ -224,7 +227,13 @@ def chat_with_claude(task, chat_history, user_instr):
                 tools=tools,
             )
             tool_response_text = next((block.text for block in tool_response.content if
             ↪  hasattr(block, "text")), None)
-            output_ls.append({TOOL_RESPONSE_MARKER: tool_response_text})
+            # Record tool response in structured format
+            if tool_response_text:
+                output_ls.append({
+                    "tool_response": {
```

```
+                        "content": tool_response_text
+                    }
+                })
            response = tool_response

            # Validate tool response
```

总之，DGM 可用于编码之外的领域，本节也突出一个目标投机案例. 与强化学习中的奖励投机 [147] 类似，当系统优化预定义、可量化目标，而非实现任务精神或解决意图中的问题时，会发生目标投机. 这一观察支持先前工作 [175, 43] 关于优化定量度量常导致不理想或病态结果的论点，也符合古德哈特定律 [150]：“当一项度量成为目标，它便不再是一项好度量.”

1.24 附录 I 补充安全讨论

任何提高人工智能自主能力的进步，都有自身安全问题 [13]，对于开放式改进系统尤其如此 [37, 24]. 第5节讨论这些问题并提出具体可行的缓解措施. 我们呼吁更多研究与讨论，包括社会所有利益相关者对究竟何为安全人工智能这一复杂问题进行深入思考和讨论. 我们确信已完成的工作从未不安全，但放大规模后的版本可能不安全. 与所有变革性技术一样，最终影响高度不确定；关于它会带来巨大益处或巨大伤害，都存在有力论据. 这些不确定性强调持续、包容、跨学科讨论的必要性，参与者不仅应有当前专家，也应有更广且更多样的社区，共同探索如何应对发展.

1.25 附录 J 补充未来工作方向

本文展示通过开放式探索与经验验证迭代改进编码智能体的潜力. 若干扩展可以应对当前局限，使人工智能超越其在启发文化与推进科学中已日益增长的作用. 以下列出有希望的研究方向.

自主改进开放式探索过程. 当前第3节的开放式探索过程固定，可能妨碍自加速潜力. 这是计算预算有限导致的选择；若演化这部分算法，可能需要指数级更多计算，才能识别产生第4.4节同样改进的过程. 但探索循环本身用代码实现，原则上可由编码智能体编辑改进. 实现可能包括使用其他平衡探索利用的搜索机制 [59]、只保留最有趣智能体 [43]，或把生成种群用作集成 [130]. 允许智能体修改探索过程，是有希望的未来方向，不只自主改善自身能力，也改善分配有限计算、推动自我改进与自加速的元过程.

人类在自主系统中的作用. 当前 DGM 提出的自我修改无需人工干预就被自主评价. 但系统越复杂、影响越大，人类如何保持参与越迫切. 人类监督应作为优化目标，通过人类反馈强化学习 [118] 等技术纳入，还是蒸馏到充当偏好评判者的基础模型中 [10]？每种方法都面临可扩展性、可靠性及与不断演变人类价值对齐的挑战. 人类在引导、约束或与自主系统共同演化中的角色仍是开放问题. 探索这一动态既涉及技术可行性，也涉及更广哲学和社会问题，是有希望的研究路径.

使用先进基础模型的 DGM. 近期基础模型大幅进步，让当前编码基准的系统支架可以更简单 [164]. 在当前编码基准等一些设置中，基础模型进步可能淡化某些支架工程努力的作用. 但先进工具、并行工作流、外部记忆、代

理验证等许多组件仍从根本上不能内化到基础模型中，对超出当前基准、更复杂真实任务仍至关重要. 探索不同基础模型下哪些智能体组件会涌现，是有希望的方向.

演化通用智能体. 我们认为一定程度的任务专用适应符合预期，甚至可取，因为多文件 Python 仓库编辑与主要单文件、多语言实现等根本不同任务，自然需要不同支架组件. 关键是，这恰好体现 DGM 等自我改进系统的独特优势：用完全自动的演化过程，替代为多种任务人工设计专用智能体的繁重工作. 这推动一个令人期待的方向：在大量多样任务上运行 DGM，演化真正通用的智能体. 当前我们也仅在两个编码基准评价；尽管认为二者在任务结构上差异显著，为普遍性提供有力证据，但加入更多基准会进一步加强评价.

参考文献

[1] Fuma Aki, Riku Ikeda, Takumi Saito, Ciaran Regan, and Mizuki Oka. Llm-poet: Evolving complex environments using large language models. In *Proceedings of the Genetic and Evolutionary Computation Conference Companion*, pp. 243–246, 2024.

[2] Rajeev Alur, Rishabh Singh, Dana Fisman, and Armando Solar-Lezama. Search-based program synthesis. *Communications of the ACM*, 61(12):84–93, 2018.

[3] S-I Amari. Learning patterns and pattern sequences by self-organizing nets of threshold elements. *IEEE Transactions on computers*, 100(11):1197–1206, 1972.

[4] Marcin Andrychowicz, Filip Wolski, Alex Ray, Jonas Schneider, Rachel Fong, Peter Welinder, Bob McGrew, Josh Tobin, OpenAI Pieter Abbeel, and Wojciech Zaremba. Hindsight experience replay. *Advances in neural information processing systems*, 30, 2017.

[5] Anthropic. Claude 3.5 Sonnet. https://www.anthropic.com/news/claude-3-5-sonnet, June 2024a. [Accessed 17 April 2025].

[6] Anthropic. Claude can now use tools, May 2024b. URL https://www.anthropic.com/news/tool-use-ga. Accessed: 2025-05-03.

[7] Anthropic. Claude 3.7 sonnet and claude code, February 2025. URL https://www.anthropic.com/news/claude-3-7-sonnet. Accessed: 2025-05-06.

[8] Usman Anwar, Abulhair Saparov, Javier Rando, Daniel Paleka, Miles Turpin, Peter Hase, Ekdeep Singh Lubana, Erik Jenner, Stephen Casper, Oliver Sourbut, et al. Foundational challenges in assuring alignment and safety of large language models. *arXiv preprint arXiv:2404.09932*, 2024.

[9] Dzmitry Bahdanau, Kyung Hyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. In *International Conference on Learning Representations*, 2015.

[10] Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, et al. Constitutional AI: Harmlessness from AI feedback.

arXiv preprint arXiv:2212.08073, 2022.

[11] Adrien Baranes and Pierre-Yves Oudeyer. Active learning of inverse models with intrinsically motivated goal exploration in robots. *Robotics and Autonomous Systems*, 61(1):49–73, 2013.

[12] Shraddha Barke, Emmanuel Anaya Gonzalez, Saketh Ram Kasibatla, Taylor Berg-Kirkpatrick, and Nadia Polikarpova. Hysynth: Context-free llm approximation for guiding program synthesis. *Advances in Neural Information Processing Systems*, 37:15612–15645, 2024.

[13] Yoshua Bengio, Geoffrey Hinton, Andrew Yao, Dawn Song, Pieter Abbeel, Trevor Darrell, Yuval Noah Harari, Ya-Qin Zhang, Lan Xue, Shai Shalev-Shwartz, et al. Managing extreme AI risks amid rapid progress. *Science*, 384(6698):842–845, 2024.

[14] N Bostrom. Existential Risks: analyzing human extinction scenarios and related hazards. *Journal of Evolution and Technology*, 9, 2002.

[15] Nick Bostrom. Ethical issues in advanced artificial intelligence. *Machine Ethics and Robot Ethics*, pp. 69–75, 2020.

[16] Herbie Bradley, Andrew Dai, Hannah Benita Teufel, Jenny Zhang, Koen Oostermeijer, Marco Bellagente, Jeff Clune, Kenneth Stanley, Gregory Schott, and Joel Lehman. Quality-diversity through ai feedback. In *The Twelfth International Conference on Learning Representations*, 2024.

[17] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.

[18] Jake Bruce, Michael D Dennis, Ashley Edwards, Jack Parker-Holder, Yuge Shi, Edward Hughes, Matthew Lai, Aditi Mavalankar, Richie Steigerwald, Chris Apps, et al. Genie: Generative interactive environments. In *Forty-first International Conference on Machine Learning*, 2024.

[19] J Richard Buchi and Lawrence H Landweber. Solving sequential conditions by finite-state strategies. In *The collected works of J. Richard Büchi*, pp. 525–541. Springer, 1990.

[20] Ruisheng Cao, Fangyu Lei, Haoyuan Wu, Jixuan Chen, Yeqiao Fu, Hongcheng Gao, Xinzhuang Xiong, Hanchong Zhang, Wenjing Hu, Yuchen Mao, et al. Spider2-v: How far are multimodal agents from automating data science and engineering workflows? *Advances in Neural Information Processing Systems*, 37:107703–107744, 2024.

[21] Konstantinos Chatzilygeroudis, Antoine Cully, Vassilis Vassiliades, and Jean-Baptiste Mouret. Quality-diversity optimization: a novel branch of stochastic optimization. In *Black Box Optimization, Machine Learning, and No-Free Lunch Theorems*, pp. 109–135. Springer, 2021.

[22] Banghao Chen, Zhaofeng Zhang, Nicolas Langrené, and Shengxin Zhu. Unleashing the potential of prompt engineering in large language models: a comprehensive review. *arXiv preprint arXiv:2310.14735*, 2023.

[23] Ching-An Cheng, Allen Nie, and Adith Swaminathan. Trace is the next autodiff: Generative optimization with rich feedback, execution traces, and llms. *Advances in Neural Information Processing Systems*, 37: 71596–71642, 2024.

[24] Jeff Clune. AI-GAs: AI-generating algorithms, an alternate paradigm for producing general artificial intelligence. *arXiv preprint arXiv:1905.10985*, 2019.

[25] Cédric Colas, Pierre Fournier, Mohamed Chetouani, Olivier Sigaud, and Pierre-Yves Oudeyer. Curious: intrinsically motivated modular multi-goal reinforcement learning. In *International conference on machine learning*, pp. 1331–1340. PMLR, 2019.

[26] Cédric Colas, Tristan Karch, Clément Moulin-Frier, and Pierre-Yves Oudeyer. Language and culture internalization for human-like autotelic AI. *Nature Machine Intelligence*, 4(12):1068–1076, 2022a.

[27] Cédric Colas, Tristan Karch, Olivier Sigaud, and Pierre-Yves Oudeyer. Autotelic agents with intrinsically motivated goal-conditioned reinforcement learning: a short survey. *Journal of Artificial Intelligence Research*, 74:1159–1199, 2022b.

[28] Cédric Colas, Laetitia Teodorescu, Pierre-Yves Oudeyer, Xingdi Yuan, and Marc-Alexandre Côté. Augmenting autotelic agents with large language models. In *Conference on Lifelong Learning Agents*, pp. 205–226. PMLR, 2023.

[29] Rémi Coulom. Efficient selectivity and backup operators in monte-carlo tree search. In *International conference on computers and games*, pp. 72–83. Springer, 2006.

[30] Charles Darwin. Origin of the species. In *British Politics and the environment in the long nineteenth century*, pp. 47–55. Routledge, 2023.

[31] Richard Dawkins. The evolution of evolvability. In *Artificial life*, pp. 201–220. Routledge, 2019.

[32] Michael Dennis, Natasha Jaques, Eugene Vinitsky, Alexandre Bayen, Stuart Russell, Andrew Critch, and Sergey Levine. Emergent complexity and zero-shot transfer via unsupervised environment design. *Advances in neural information processing systems*, 33:13049–13061, 2020.

[33] Aaron Dharna, Cong Lu, and Jeff Clune. Quality-Diversity Self-Play: Open-Ended Strategy Innovation via Foundation Models. In *NeurIPS 2024 Workshop on Open-World Agents*, 2024.

[34] Li Ding, Jenny Zhang, Jeff Clune, Lee Spector, and Joel Lehman. Quality diversity through human feedback: towards open-ended diversity-driven optimization. In *Proceedings of the 41st International Conference on Machine Learning*, pp. 11072–11090, 2024.

[35] Theodosius Dobzhansky. *Genetics of the evolutionary process*, volume 139. Columbia University Press, 1970.

[36] Adrien Ecoffet, Joost Huizinga, Joel Lehman, Kenneth O Stanley, and Jeff Clune. Go-explore: a new approach for hard-exploration problems. *arXiv preprint arXiv:1901.10995*, 2019.

[37] Adrien Ecoffet, Jeff Clune, and Joel Lehman. Open questions in creating safe open-ended AI: tensions between control and creativity. In *Artificial Life Conference Proceedings 32*, pp. 27–35. MIT Press One Rogers Street, Cambridge, MA 02142-1209, USA journals-info ···, 2020.

[38] Adrien Ecoffet, Joost Huizinga, Joel Lehman, Kenneth O Stanley, and Jeff Clune. First return, then explore. *Nature*, 590(7847):580–586, 2021.

[39] Anthony W Fisher Edwards. The genetical theory of natural selection. *Genetics*, 154(4):1419–1426, 2000.

[40] Kevin Ellis, Catherine Wong, Maxwell Nye, Mathias Sablé-Meyer, Lucas Morales, Luke Hewitt, Luc Cary, Armando Solar-Lezama, and Joshua B Tenenbaum. Dreamcoder: Bootstrapping inductive program synthesis with wake-sleep library learning. In *Proceedings of the 42nd acm sigplan international conference on programming language design and implementation*, pp. 835–850, 2021.

[41] Benjamin Eysenbach, Abhishek Gupta, Julian Ibarz, and Sergey Levine. Diversity is all you need: Learning skills without a reward function. *arXiv preprint arXiv:1802.06070*, 2018.

[42] Meta Fundamental AI Research Diplomacy Team (FAIR)†, Anton Bakhtin, Noam Brown, Emily Dinan, Gabriele Farina, Colin Flaherty, Daniel Fried, Andrew Goff, Jonathan Gray, Hengyuan Hu, et al. Human-level play in the game of Diplomacy by combining language models with strategic reasoning. *Science*, 378 (6624):1067–1074, 2022.

[43] Maxence Faldor, Jenny Zhang, Antoine Cully, and Jeff Clune. OMNI-EPIC: Open-endedness via Models of human Notions of Interestingness with Environments Programmed in Code. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=Y1XkzMJpPd.

[44] Chrisantha Fernando, Dylan Sunil Banarse, Henryk Michalewski, Simon Osindero, and Tim Rocktäschel. Promptbreeder: Self-Referential Self-Improvement via Prompt Evolution. In *Forty-first International Conference on Machine Learning*, 2024.

[45] Deep Ganguli, Liane Lovitt, Jackson Kernion, Amanda Askell, Yuntao Bai, Saurav Kadavath, Ben Mann, Ethan Perez, Nicholas Schiefer, Kamal Ndousse, et al. Red teaming language models to reduce harms: Methods, scaling behaviors, and lessons learned. *arXiv preprint arXiv:2209.07858*, 2022.

[46] Hongcheng Gao, Yue Liu, Yufei He, Longxu Dou, Chao Du, Zhijie Deng, Bryan Hooi, Min Lin, and Tianyu Pang. Flowreasoner: Reinforcing query-level meta-agents. *arXiv preprint arXiv:2504.15257*, 2025.

[47] Paul Gauthier. Aider: Ai pair programming in your terminal. https://github.com/Aider-AI/aider, 2024. Accessed: 2025-05-14.

[48] Loris Gaven, Thomas Carta, Clément Romac, Cédric Colas, Sylvain Lamprier, Olivier Sigaud, and Pierre-Yves Oudeyer. MAGELLAN: Metacognitive predictions of learning progress guide autotelic LLM agents in large goal spaces. *arXiv preprint arXiv:2502.07709*, 2025.

[49] John Gerhart and Marc Kirschner. The theory of facilitated variation. *Proceedings of the National Academy of Sciences*, 104(suppl_1):8582–8589, 2007.

[50] Irving John Good. Speculations concerning the first ultraintelligent machine. In *Advances in computers*, volume 6, pp. 31–88. Elsevier, 1966.

[51] Google DeepMind. Gemini model "thinking" updates —march 2025. https://blog.google/technology/google-deepmind/gemini-model-thinking-updates-march-2025/#gemini-2-5-thinking, March 2025. Accessed: 2025-05-11.

[52] Ryan Greenblatt, Carson Denison, Benjamin Wright, Fabien Roger, Monte MacDiarmid, Sam Marks, Johannes Treutlein, Tim Belonax, Jack Chen, David Duvenaud, et al. Alignment faking in large language models. *arXiv preprint arXiv:2412.14093*, 2024.

[53] Sumit Gulwani. Automating string processing in spreadsheets using input-output examples. *ACM Sigplan Notices*, 46(1):317–330, 2011.

[54] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

[55] John Storrs Hall. Self-improving AI: An analysis. *Minds and Machines*, 17(3):249–259, 2007.

[56] Alex Havrilla, Andrew Dai, Laura O'Mahony, Koen Oostermeijer, Vera Zisler, Alon Albalak, Fabrizio Milo, Sharath Chandra Raparthy, Kanishk Gandhi, Baber Abbasi, et al. Surveying the effects of quality, diversity, and complexity in synthetic data from large language models. *arXiv preprint arXiv:2412.02980*, 2024a.

[57] Alex Havrilla, Sharath Raparthy, Christoforus Nalmpantis, Jane Dwivedi-Yu, Maksym Zhuravinskyi, Eric Hambro, and Roberta Raileanu. Glore: When, where, and how to improve llm reasoning via global and local refinements. *arXiv preprint arXiv:2402.10963*, 2024b.

[58] Jesse Love Hendrikse, Trish Elizabeth Parsons, and Benedikt Hallgrímsson. Evolvability as the proper focus of evolutionary developmental biology. *Evolution & development*, 9(4):393–401, 2007.

[59] Nathan Herr, Tim Rocktäschel, and Roberta Raileanu. Llm-first search: Self-guided exploration of the solution space. *arXiv preprint arXiv:2506.05213*, 2025.

[60] Marius Hobbhahn. Swe-bench verified mini. https://github.com/mariushobbhahn/SWEBench-verified-mini, April 2025. Accessed: 2025-04-16.

[61] John J Hopfield. Neural networks and physical systems with emergent collective computational abilities. *Proceedings of the national academy of sciences*, 79(8):2554–2558, 1982.

[62] Shengran Hu and Jeff Clune. Thought Cloning: Learning to think while acting by imitating human thinking. *Advances in Neural Information Processing Systems*, 36, 2024.

[63] Shengran Hu, Cong Lu, and Jeff Clune. Automated Design of Agentic Systems. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=t9U3LW7JVX.

[64] Yue Hu, Yuzhu Cai, Yaxin Du, Xinyu Zhu, Xiangrui Liu, Zijie Yu, Yuchen Hou, Shuo Tang, and Siheng Chen. Self-evolving multi-agent collaboration networks for software development. *arXiv preprint arXiv:2410.16946*, 2024.

[65] Jiaxin Huang, Shixiang Shane Gu, Le Hou, Yuexin Wu, Xuezhi Wang, Hongkun Yu, and Jiawei Han. Large language models can self-improve. *arXiv preprint arXiv:2210.11610*, 2022.

[66] Edward Hughes, Michael Dennis, Jack Parker-Holder, Feryal Behbahani, Aditi Mavalankar, Yuge Shi, Tom Schaul, and Tim Rocktaschel. Open-endedness is essential for artificial superhuman intelligence. *arXiv preprint arXiv:2406.04268*, 2024.

[67] Kazuki Irie, Imanol Schlag, Róbert Csordás, and Jürgen Schmidhuber. A modern self-referential weight matrix that learns to modify itself. In *International Conference on Machine Learning*, pp. 9660–9677. PMLR, 2022.

[68] Kazuki Irie, Róbert Csordás, and Jürgen Schmidhuber. Metalearning continual learning algorithms. *Transactions on Machine Learning Research*, 2025.

[69] Max Jaderberg, Valentin Dalibard, Simon Osindero, Wojciech M Czarnecki, Jeff Donahue, Ali Razavi, Oriol Vinyals, Tim Green, Iain Dunning, Karen Simonyan, et al. Population based training of neural networks. *arXiv preprint arXiv:1711.09846*, 2017.

[70] Minqi Jiang, Edward Grefenstette, and Tim Rocktäschel. Prioritized level replay. In *International Conference on Machine Learning*, pp. 4940–4950. PMLR, 2021.

[71] Minqi Jiang, Tim Rocktäschel, and Edward Grefenstette. General intelligence requires rethinking exploration. *Royal Society Open Science*, 10(6):230539, 2023.

[72] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. SWE-bench: Can Language Models Resolve Real-world Github Issues? In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=VTF8yNQM66.

[73] Ingmar Kanitscheider, Joost Huizinga, David Farhi, William Hebgen Guss, Brandon Houghton, Raul Sampedro, Peter Zhokhov, Bowen Baker, Adrien Ecoffet, Jie Tang, Oleg Klimov, and Jeff Clune. Multi-task cur-

riculum learning in a complex, visual, hard-exploration domain: Minecraft. *arXiv preprint arXiv:2106.14876*, 2021.

[74] Akbir Khan, John Hughes, Dan Valentine, Laura Ruis, Kshitij Sachan, Ansh Radhakrishnan, Edward Grefenstette, Samuel R Bowman, Tim Rocktäschel, and Ethan Perez. Debating with more persuasive llms leads to more truthful answers. *arXiv preprint arXiv:2402.06782*, 2024.

[75] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T Joshi, Hanna Moazam, et al. Dspy: Compiling declarative language model calls into self-improving pipelines. *arXiv preprint arXiv:2310.03714*, 2023.

[76] Yoon Kim, Carl Denton, Luong Hoang, and Alexander M Rush. Structured Attention Networks. In *International Conference on Learning Representations*, 2017.

[77] Motoo Kimura. The neutral theory of molecular evolution. *Scientific American*, 241(5):98–129, 1979.

[78] Louis Kirsch and Jürgen Schmidhuber. Self-referential meta learning. In *First Conference on Automated Machine Learning (Late-Breaking Workshop)*, 2022.

[79] Martin Klissarov, Pierluca D'Oro, Shagun Sodhani, Roberta Raileanu, Pierre-Luc Bacon, Pascal Vincent, Amy Zhang, and Mikael Henaff. Motif: Intrinsic motivation from artificial intelligence feedback. *arXiv preprint arXiv:2310.00166*, 2023.

[80] Martin Klissarov, Mikael Henaff, Roberta Raileanu, Shagun Sodhani, Pascal Vincent, Amy Zhang, Pierre-Luc Bacon, Doina Precup, Marlos C Machado, and Pierluca D'Oro. MaestroMotif: Skill Design from Artificial Intelligence Feedback. *arXiv preprint arXiv:2412.08542*, 2024.

[81] Varun Raj Kompella, Marijn Stollenga, Matthew Luciw, and Juergen Schmidhuber. Continual curiosity-driven skill acquisition from high-dimensional video inputs for humanoid robots. *Artificial Intelligence*, 247: 313–335, 2017.

[82] Robert Lange, Tom Schaul, Yutian Chen, Tom Zahavy, Valentin Dalibard, Chris Lu, Satinder Singh, and Sebastian Flennerhag. Discovering evolution strategies via meta-black-box optimization. In *Proceedings of the Companion Conference on Genetic and Evolutionary Computation*, pp. 29–30, 2023.

[83] Robert Lange, Yingtao Tian, and Yujin Tang. Large language models as evolution strategies. In *Proceedings of the Genetic and Evolutionary Computation Conference Companion*, pp. 579–582, 2024.

[84] Joonho Lee, Jemin Hwangbo, Lorenz Wellhausen, Vladlen Koltun, and Marco Hutter. Learning quadrupedal locomotion over challenging terrain. *Science robotics*, 5(47):eabc5986, 2020.

[85] Joel Lehman. Machine love. *arXiv preprint arXiv:2302.09248*, 2023.

[86] Joel Lehman and Kenneth O Stanley. Novelty search and the problem with objectives. *Genetic programming theory and practice IX*, pp. 37–56, 2011.

[87] Joel Lehman, Jonathan Gordon, Shawn Jain, Kamal Ndousse, Cathy Yeh, and Kenneth O Stanley. Evolution through large models. In *Handbook of Evolutionary Machine Learning*, pp. 331–366. Springer, 2023.

[88] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33:9459–9474, 2020.

[89] J. Li, Storie J., and J. Clune. Encouraging creative thinking in robots improves their ability to solve challenging problems. In *Proceedings of the Genetic and Evolutionary Computation Conference*, pp. 193–200, 2014.

[90] Yixuan Li, Julian Parsert, and Elizabeth Polgreen. Guiding enumerative program synthesis with large language models. In *International Conference on Computer Aided Verification*, pp. 280–301. Springer, 2024.

[91] Tian Liang, Zhiwei He, Wenxiang Jiao, Xing Wang, Yan Wang, Rui Wang, Yujiu Yang, Shuming Shi, and Zhaopeng Tu. Encouraging divergent thinking in large language models through multi-agent debate. *arXiv preprint arXiv:2305.19118*, 2023.

[92] Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step. In *The Twelfth International Conference on Learning Representations*, 2023.

[93] Bryan Lim, Manon Flageat, and Antoine Cully. Large language models as in-context ai generators for quality-diversity. In *ALIFE 2024: Proceedings of the 2024 Artificial Life Conference*. MIT Press, 2024.

[94] Seppo Linnainmaa. *The representation of the cumulative rounding error of an algorithm as a Taylor expansion of the local rounding errors*. PhD thesis, Master' s Thesis (in Finnish), Univ. Helsinki, 1970.

[95] Fei Liu, Xialiang Tong, Mingxuan Yuan, Xi Lin, Fu Luo, Zhenkun Wang, Zhichao Lu, and Qingfu Zhang. Evolution of heuristics: Towards efficient automatic algorithm design using large language model. *arXiv preprint arXiv:2401.02051*, 2024.

[96] Lei Liu, Xiaoyan Yang, Yue Shen, Binbin Hu, Zhiqiang Zhang, Jinjie Gu, and Guannan Zhang. Think-in-memory: Recalling and post-thinking enable llms with long-term memory. *arXiv preprint arXiv:2311.08719*, 2023.

[97] Chris Lu, Sebastian Towers, and Jakob Foerster. Arbitrary order meta-learning with simple population-based evolution. In *Artificial Life Conference Proceedings 35*, volume 2023, pp. 67. MIT Press One Rogers Street, Cambridge, MA 02142-1209, USA journals-info ⋯, 2023.

[98] Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. The ai scientist: Towards fully automated open-ended scientific discovery. *arXiv preprint arXiv:2408.06292*, 2024a.

[99] Cong Lu, Shengran Hu, and Jeff Clune. Intelligent go-explore: Standing on the shoulders of giant foundation models. *arXiv preprint arXiv:2405.15143*, 2024b.

[100] Cong Lu, Shengran Hu, and Jeff Clune. Automated capability discovery via model self-exploration. *arXiv preprint arXiv:2502.0757*, 2025.

[101] Yecheng Jason Ma, William Liang, Guanzhi Wang, De-An Huang, Osbert Bastani, Dinesh Jayaraman, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Eureka: Human-level reward design via coding large language models. *arXiv preprint arXiv:2310.12931*, 2023.

[102] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback, 2023. *URL https://arxiv. org/abs/2303.17651*, 2023.

[103] John Markoff. *Machines of loving grace: The quest for common ground between humans and robots*. HarperCollins Publishers, 2016.

[104] Ernst Mayr. *The growth of biological thought: Diversity, evolution, and inheritance*. Harvard University Press, 1982.

[105] Luke Metz, C Daniel Freeman, Niru Maheswaranathan, and Jascha Sohl-Dickstein. Training learned optimizers with randomly initialized learned optimizers. *arXiv preprint arXiv:2101.07367*, 2021.

[106] Ali Modarressi, Ayyoob Imani, Mohsen Fayyaz, and Hinrich Schütze. Ret-llm: Towards a general read-write memory for large language models. *arXiv preprint arXiv:2305.14322*, 2023.

[107] Jean-Baptiste Mouret and Jeff Clune. Illuminating search spaces by mapping elites. *arXiv preprint arXiv:1504.04909*, 2015.

[108] Niklas Muennighoff, Zitong Yang, Weijia Shi, Xiang Lisa Li, Li Fei-Fei, Hannaneh Hajishirzi, Luke Zettlemoyer, Percy Liang, Emmanuel Candès, and Tatsunori Hashimoto. s1: Simple test-time scaling. *arXiv preprint arXiv:2501.19393*, 2025.

[109] Muhammad U Nasir and Julian Togelius. Practical PCG through large language models. In *2023 IEEE Conference on Games (CoG)*, pp. 1–4. IEEE, 2023.

[110] Muhammad U Nasir, Steven James, and Julian Togelius. Word2world: Generating stories and worlds through large language models. *arXiv preprint arXiv:2405.06686*, 2024.

[111] Anh Mai Nguyen, Jason Yosinski, and Jeff Clune. Innovation engines: Automated creativity and improved stochastic optimization via deep learning. In *Proceedings of the 2015 annual conference on genetic and evolutionary computation*, pp. 959–966, 2015.

[112] Fan Nie, Lan Feng, Haotian Ye, Weixin Liang, Pan Lu, Huaxiu Yao, Alexandre Alahi, and James Zou.

Weak-for-strong: Training weak meta-agent to harness strong executors. *arXiv preprint arXiv:2504.04785*, 2025.

[113] Boye Niu, Yiliao Song, Kai Lian, Yifan Shen, Yu Yao, Kun Zhang, and Tongliang Liu. Flow: Modularized agentic workflow automation. In *The Thirteenth International Conference on Learning Representations*, 2025.

[114] Alexander Novikov, Ngân Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog. Alphaevolve: A coding agent for scientific and algorithmic discovery. Technical report, Google DeepMind, 2025.

[115] OpenAI. Introducing swe-bench verified. https://openai.com/index/introducing-swe-bench-verified/, August 2024. Accessed: 2025-04-16.

[116] OpenAI. OpenAI o3-mini. https://openai.com/index/openai-o3-mini/, January 2025. Accessed: 2025-05-01.

[117] Pierre-Yves Oudeyer, Frdric Kaplan, and Verena V Hafner. Intrinsic motivation systems for autonomous mental development. *IEEE transactions on evolutionary computation*, 11(2):265–286, 2007.

[118] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in neural information processing systems*, 35:27730–27744, 2022.

[119] Ankur Parikh, Oscar Täckström, Dipanjan Das, and Jakob Uszkoreit. A Decomposable Attention Model for Natural Language Inference. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pp. 2249–2255, 2016.

[120] Jack Parker-Holder, Philip Ball, Jake Bruce, Vibhavari Dasagi, Kristian Holsheimer, Christos Kaplanis, Alexandre Moufarek, Guy Scully, Jeremy Shar, Jimmy Shi, Stephen Spencer, Jessica Yung, Michael Dennis, Sultan Kenjeyev, Shangbang Long, Vlad Mnih, Harris Chan, Maxime Gazeau, Bonnie Li, Fabio Pardo, Luyu Wang, Lei Zhang, Frederic Besse, Tim Harley, Anna Mitenkova, Jane Wang, Jeff Clune, Demis Hassabis, Raia Hadsell, Adrian Bolton, Satinder Singh, and Tim Rocktäschel. Genie 2: A large-scale foundation world model, 2024. URL https://deepmind.google/discover/blog/genie-2-a-large-scale-foundation-world-model/.

[121] Deepak Pathak, Pulkit Agrawal, Alexei A Efros, and Trevor Darrell. Curiosity-driven exploration by self-supervised prediction. In *International conference on machine learning*, pp. 2778–2787. PMLR, 2017.

[122] Paul Gauthier. o1 tops aider's new polyglot leaderboard. https://aider.chat/2024/12/21/polyglot.html, December 2024. Accessed: 2025-04-16.

[123] Oleksandr Polozov and Sumit Gulwani. Flashmeta: A framework for inductive program synthesis. In *Proceedings of the 2015 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications*, pp. 107–126, 2015.

[124] Justin K Pugh, Lisa B Soros, and Kenneth O Stanley. Quality diversity: A new frontier for evolutionary computation. *Frontiers in Robotics and AI*, 3:40, 2016.

[125] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019.

[126] Maxime Robeyns, Martin Szummer, and Laurence Aitchison. A Self-Improving Coding Agent. *arXiv preprint arXiv:2504.15228*, 2025.

[127] Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M Pawan Kumar, Emilien Dupont, Francisco JR Ruiz, Jordan S Ellenberg, Pengming Wang, Omar Fawzi, et al. Mathematical discoveries from program search with large language models. *Nature*, 625(7995):468–475, 2024.

[128] J Rosser and Jakob Nicolaus Foerster. Agentbreeder: Mitigating the AI safety impact of multi-agent scaffolds via self-improvement. In *Scaling Self-Improving Foundation Models without Human Supervision*, 2025. URL https://openreview.net/forum?id=j0n3BJJTcT.

[129] David E Rumelhart, Geoffrey E Hinton, Ronald J Williams, et al. Learning internal representations by error propagation, 1985.

[130] Mikayel Samvelyan, Sharath Chandra Raparthy, Andrei Lupu, Eric Hambro, Aram Markosyan, Manish Bhatt, Yuning Mao, Minqi Jiang, Jack Parker-Holder, Jakob Foerster, et al. Rainbow teaming: Open-ended generation of diverse adversarial prompts. *Advances in Neural Information Processing Systems*, 37:69747–69786, 2024.

[131] Cansu Sancaktar, Christian Gumbsch, Andrii Zadaianchuk, Pavel Kolev, and Georg Martius. SENSEI: Semantic Exploration Guided by Foundation Models to Learn Versatile World Models. *arXiv preprint arXiv:2503.01584*, 2025.

[132] Tom Schaul, Daniel Horgan, Karol Gregor, and David Silver. Universal value function approximators. In *International conference on machine learning*, pp. 1312–1320. PMLR, 2015.

[133] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems*, 36:68539–68551, 2023.

[134] Jürgen Schmidhuber. *Evolutionary principles in self-referential learning, or on learning how to learn: the meta-meta-... hook*. PhD thesis, Technische Universität München, 1987.

[135] Jürgen Schmidhuber. A 'self-referential' weight matrix. In *International conference on artificial neural networks*, pp. 446–450. Springer, 1993.

[136] Jürgen Schmidhuber. Gödel machines: Fully self-referential optimal universal self-improvers. In *Artificial general intelligence*, pp. 199–226. Springer, 2007.

[137] Jürgen Schmidhuber. Driven by compression progress: A simple principle explains essential aspects of subjective beauty, novelty, surprise, interestingness, attention, curiosity, creativity, art, science, music, jokes. In *Workshop on anticipatory behavior in adaptive learning systems*, pp. 48–76. Springer, 2008.

[138] Jürgen Schmidhuber. Powerplay: Training an increasingly general problem solver by continually searching for the simplest still unsolvable problem. *Frontiers in psychology*, 4:313, 2013.

[139] Jürgen Schmidhuber and Rudolf Huber. *Learning to generate focus trajectories for attentive vision*. Institut für Informatik, 1990.

[140] Sander Schulhoff, Michael Ilie, Nishant Balepur, Konstantine Kahadze, Amanda Liu, Chenglei Si, Yinheng Li, Aayush Gupta, HyoJung Han, Sevien Schulhoff, et al. The prompt report: A systematic survey of prompting techniques. *arXiv preprint arXiv:2406.06608*, 2024.

[141] Ivaxi Sheth, Jan Wehner, Sahar Abdelnabi, Ruta Binkyte, and Mario Fritz. Safety is Essential for Responsible Open-Ended Systems. *arXiv preprint arXiv:2502.04512*, 2025.

[142] Kensen Shi, Hanjun Dai, Wen-Ding Li, Kevin Ellis, and Charles Sutton. Lambdabeam: Neural program search with higher-order functions and lambdas. *Advances in Neural Information Processing Systems*, 36: 51327–51346, 2023.

[143] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36:8634–8652, 2023.

[144] David Silver, Aja Huang, Chris J Maddison, Arthur Guez, Laurent Sifre, George Van Den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, et al. Mastering the game of go with deep neural networks and tree search. *nature*, 529(7587):484–489, 2016.

[145] David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharshan Kumaran, Thore Graepel, et al. Mastering chess and shogi by self-play with a general reinforcement learning algorithm. *arXiv preprint arXiv:1712.01815*, 2017.

[146] Avi Singh, John D Co-Reyes, Rishabh Agarwal, Ankesh Anand, Piyush Patil, Xavier Garcia, Peter J Liu, James Harrison, Jaehoon Lee, Kelvin Xu, et al. Beyond human data: Scaling self-training for problem-solving with language models. *arXiv preprint arXiv:2312.06585*, 2023.

[147] Joar Skalse, Nikolaus Howe, Dmitrii Krasheninnikov, and David Krueger. Defining and characterizing reward gaming. *Advances in Neural Information Processing Systems*, 35:9460–9471, 2022.

[148] Kenneth O Stanley and Joel Lehman. *Why greatness cannot be planned: The myth of the objective*. Springer, 2015.

[149] Kenneth O Stanley, Joel Lehman, and Lisa Soros. Open-endedness: The last grand challenge you' ve never heard of. *While open-endedness could be a force for discovering intelligence, it could also be a component of AI itself*, 2017.

[150] Marilyn Strathern. 'Improving ratings' : audit in the British University system. *European review*, 5(3): 305–321, 1997.

[151] Jinwei Su, Yinghui Xia, Ronghua Shi, Jianhui Wang, Jianuo Huang, Yijin Wang, Tianyu Shi, Yang Jingsong, and Lewei He. Debflow: Automating agent creation via agent debate. *arXiv preprint arXiv:2503.23781*, 2025.

[152] Shyam Sudhakaran, Miguel González-Duque, Matthias Freiberger, Claire Glanois, Elias Najarro, and Sebastian Risi. Mariogpt: Open-ended text2level generation through large language models. *Advances in Neural Information Processing Systems*, 36:54213–54227, 2023.

[153] OpenAI Team, Aaron Jaech, Adam Kalai, Adam Lerer, Adam Richardson, Ahmed El-Kishky, Aiden Low, Alec Helyar, Aleksander Madry, Alex Beutel, Alex Carney, et al. Openai o1 system card. *arXiv preprint arXiv:2412.16720*, 2024.

[154] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

[155] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023a.

[156] Ren-Jian Wang, Ke Xue, Yutong Wang, Peng Yang, Haobo Fu, Qiang Fu, and Chao Qian. Diversity from human feedback. *arXiv preprint arXiv:2310.06648*, 2023b.

[157] Rui Wang, Joel Lehman, Jeff Clune, and Kenneth O Stanley. Paired open-ended trailblazer (poet): Endlessly generating increasingly complex and diverse learning environments and their solutions. *arXiv preprint arXiv:1901.01753*, 2019.

[158] Xingyao Wang, Boxuan Li, Yufan Song, Frank F Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, et al. Openhands: An open platform for ai software developers as generalist agents. In *The Thirteenth International Conference on Learning Representations*, 2024.

[159] Yufei Wang, Zhou Xian, Feng Chen, Tsun-Hsuan Wang, Yian Wang, Katerina Fragkiadaki, Zackory Erickson, David Held, and Chuang Gan. Robogen: Towards unleashing infinite data for automated robot learning via

generative simulation. *arXiv preprint arXiv:2311.01455*, 2023c.

[160] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.

[161] Marco Wiering and Jürgen Schmidhuber. Hq-learning. *Adaptive behavior*, 6(2):219–246, 1997.

[162] S Wright. The roles of mutation, inbreeding, crossbreeding and selection in evolution, proceedings of the sixth international congress of genetics. proc sixth int congr genet [internet]. *New York356366*, 1932.

[163] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Agentless: Demystifying llm-based software engineering agents. *arXiv preprint arXiv:2407.01489*, 2024.

[164] John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik R Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. URL https://arxiv.org/abs/2405.15793.

[165] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

[166] Rui Ye, Shuo Tang, Rui Ge, Yaxin Du, Zhenfei Yin, Siheng Chen, and Jing Shao. Mas-gpt: Training llms to build llm-based multi-agent systems. *arXiv preprint arXiv:2503.03686*, 2025.

[167] Xunjian Yin, Xinyi Wang, Liangming Pan, Xiaojun Wan, and William Yang Wang. G\" odel Agent: A Self-Referential Agent Framework for Recursive Self-Improvement. *arXiv preprint arXiv:2410.04444*, 2024.

[168] Siyu Yuan, Kaitao Song, Jiangjie Chen, Xu Tan, Dongsheng Li, and Deqing Yang. Evoagent: Towards automatic multi-agent generation via evolutionary algorithms. *arXiv preprint arXiv:2406.14228*, 2024.

[169] Eliezer Yudkowsky et al. Artificial Intelligence as a positive and negative factor in global risk. *Global catastrophic risks*, 1(303):184, 2008.

[170] Mert Yuksekgonul, Federico Bianchi, Joseph Boen, Sheng Liu, Zhi Huang, Carlos Guestrin, and James Zou. Textgrad: Automatic" differentiation" via text. *arXiv preprint arXiv:2406.07496*, 2024.

[171] Eric Zelikman, Georges Harik, Yijia Shao, Varuna Jayasiri, Nick Haber, and Noah D Goodman. Quiet-star: Language models can teach themselves to think before speaking. *arXiv preprint arXiv:2403.09629*, 2024a.

[172] Eric Zelikman, Eliana Lorch, Lester Mackey, and Adam Tauman Kalai. Self-taught optimizer (stop): Recursively self-improving code generation. In *First Conference on Language Modeling*, 2024b.

[173] Dan Zhang, Sining Zhoubian, Ziniu Hu, Yisong Yue, Yuxiao Dong, and Jie Tang. Rest-mcts*: Llm self-

training via process reward guided tree search. *Advances in Neural Information Processing Systems*, 37: 64735–64772, 2024a.

[174] Guibin Zhang, Luyang Niu, Junfeng Fang, Kun Wang, Lei Bai, and Xiang Wang. Multi-agent architecture search via agentic supernet. *arXiv preprint arXiv:2502.04180*, 2025a.

[175] Jenny Zhang, Joel Lehman, Kenneth Stanley, and Jeff Clune. OMNI: Open-endedness via Models of human Notions of Interestingness. In *The Twelfth International Conference on Learning Representations*, 2024b. URL https://openreview.net/forum?id=AgM3MzT99c.

[176] Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xionghui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, et al. Aflow: Automating agentic workflow generation. *arXiv preprint arXiv:2410.10762*, 2024c.

[177] Yuanshuo Zhang, Yuchen Hou, Bohan Tang, Shuo Chen, Muhan Zhang, Xiaowen Dong, and Siheng Chen. Gnns as predictors of agentic workflow performances. *arXiv preprint arXiv:2503.11301*, 2025b.

[178] Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. Autocoderover: Autonomous program improvement. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, pp. 1592–1604, 2024d.

[179] Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. Memorybank: Enhancing large language models with long-term memory. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pp. 19724–19731, 2024.

[180] Andy Zhou, Kevin Wu, Francesco Pinto, Zhaorun Chen, Yi Zeng, Yu Yang, Shuang Yang, Sanmi Koyejo, James Zou, and Bo Li. AutoRedTeamer: Autonomous Red Teaming with Lifelong Attack Integration. *arXiv preprint arXiv:2503.15754*, 2025.

[181] Wangchunshu Zhou, Yixin Ou, Shengwei Ding, Long Li, Jialong Wu, Tiannan Wang, Jiamin Chen, Shuai Wang, Xiaohua Xu, Ningyu Zhang, et al. Symbolic learning enables self-evolving agents. *arXiv preprint arXiv:2406.18532*, 2024.

[182] Yuqi Zhu, Jia Li, Ge Li, YunFei Zhao, Zhi Jin, and Hong Mei. Hot or cold? adaptive temperature sampling for code generation with large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pp. 437–445, 2024.

[183] Mingchen Zhuge, Haozhe Liu, Francesco Faccio, Dylan R. Ashley, Róbert Csordás, Anand Gopalakrishnan, Abdullah Hamdi, Hasan Abed Al Kader Hammoud, Vincent Herrmann, Kazuki Irie, Louis Kirsch, Bing Li, Guohao Li, Shuming Liu, Jinjie Mai, Piotr Piékos, Aditya Ramesh, Imanol Schlag, Weimin Shi, Aleksandar Stanic, Wenyi Wang, Yuhui Wang, Mengmeng Xu, Deng-Ping Fan, Bernard Ghanem, and Jürgen Schmidhuber. Mindstorms in natural language-based societies of mind. *arXiv preprint arXiv:2305.17066*, 2023.

[184] Mingchen Zhuge, Wenyi Wang, Louis Kirsch, Francesco Faccio, Dmitrii Khizbullin, and Jürgen Schmidhuber. Gptswarm: Language agents as optimizable graphs. In *Forty-first International Conference on Machine Learning*, 2024.

1.26 译者术语表

下表为译者附加，不属于原论文正文.

原术语	译法	说明
foundation model	基础模型	本文缩写 FM；实验中冻结其参数
coding agent	编码智能体	可以读写执行代码的系统
self-referential	自指	系统以自身实现作为修改对象
self-improvement	自我改进	本文主要指修改智能体系统代码
self-modification	自我修改	修改自身代码库的具体操作
open-endedness	开放式	按原文使用，不等同于已证明无限进步
open-ended exploration	开放式探索	保留多样垫脚石、从档案继续分支
archive	档案	已生成且仍具基本代码编辑功能的智能体集合
stepping stone	垫脚石	可能支持之后改进的中间解
parent / offspring	父代 / 子代	智能体版本的生成谱系
meta-agent	元智能体	负责产生或修改其他智能体的智能体
meta-utility	元效用	根据生成解评价元层行为的效用
scaffolding	系统支架	围绕基础模型的工具、记忆与工作流
quality-diversity	质量多样性	兼顾解的质量与多样性的搜索
objective hacking	目标投机	提高量化目标却没有实现任务意图
reward hacking	奖励投机	强化学习中利用奖励度量缺口
tool-use hallucination	工具使用幻觉	以文本虚构工具输入输出而未实际调用
checked solution	经核验方案	基准团队获准访问并复现生成结果
pass@1 / pass@2	pass@1 / pass@2	原文保留；Polyglot 对比允许测试反馈次数不同
held-out benchmark	留出基准	智能体优化中未访问的评价基准
codebase-editing functionality	代码库编辑功能	智能体能够继续改动目标代码库的基本功能
staged evaluation	分阶段评价	先少量任务，再扩展到更多任务
foundation-model workflow	基础模型工作流	组织模型调用与任务执行的流程
program synthesis	程序合成	根据外部规范生成程序